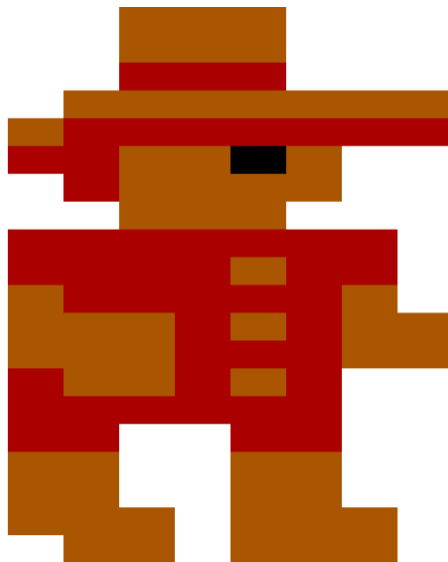


The Montezuma's Revenge Cookbook

How a 1984 IBM PC game fits ninety-nine rooms into 32,228 bytes



Contents

About This Book	8
How to Read This Book.....	8
Part I. The Machine and the Startup.....	9
1. What This Game Is, and What This File Is.....	9
2. From Floppy to First Instruction.....	11
3. Initialization: Video, Timer, Vectors	12
3.1. Video Mode	12
3.2. The Floppy Motor.....	13
3.3. The Sound Driver: SNDINI (058Eh).....	13
3.4. Two Interrupts of Its Own	14
3.5. What Is Left, and Where Control Goes Next	15
3.6. VIDINI — Setting Up a Session	15
4. The Memory Map: Where Everything Lives	16
4.1. One 64K Segment for Everything	16
4.2. The Stack Area	16
4.3. Global Sound and Frame Variables.....	16
4.4. The Pyramid: Two Tables That Describe It Exactly	17
4.5. The Room Builder's Working Cells	18
Part II. The Heart of the Program.....	19
5. The Main Loop and a Ten-State Machine	19
5.1. Thirty Instructions That Run for the Whole Game.....	19
5.2. Two Hot Keys.....	19
5.3. The State Table.....	20
5.4. The Complete Transition Graph.....	20
5.5. State 6: Wired Up, Waiting for One Instruction.....	21
5.6. Three States Instead of One: Splitting the Work	21
5.7. The Menu Does Not Take Part in the Loop.....	22

6. The Frame Budget: Why an Empty Room Is Not Faster	23
6.1. The Problem	23
6.2. Estimating the Work Done	23
6.3. The Padding	24
6.4. How Long Is That in Milliseconds	24
6.5. How Much Really Goes into the Idle Loop	25
Part III. The Picture	26
7. Tiles: Format, CGA Interleave, PUTMAP	26
7.1. How CGA Video Memory Is Arranged	26
7.2. The Tile Format	26
7.3. PUTMAP — the One and Only Tile Output	26
7.4. Two Starting Addresses	27
7.5. The Core: Eight Rows in Four Turns	28
7.6. What It Costs	28
8. The Screen Map SCRMAP and Dirty Rectangles	28
8.1. An Intermediate Layer	28
8.2. Full Redraw: DRWALL	29
8.3. Partial Redraw: PUTRCT	29
8.4. Where the Rectangles Come From	29
8.5. How Much Actually Gets Redrawn	31
9. Rooms as Bytecode: Eight Primitives	31
9.1. There Are No Rooms in the File	31
9.2. The Instruction Format	31
9.3. The Interpreter Loop	32
9.4. The Eight Primitives	33
9.5. What a Primitive Looks Like Inside	33
10. Fragments, Mirrors and the Difficulty Level	34
10.1. A Room Inside a Room	34
10.2. The Mirror	35

10.3. The Difficulty Level Changes the Geometry.....	36
11. Sprites: Frames, Masks, Background Saving.....	37
11.1. Eight Objects and One Bit Mask	37
11.2. Where to Write: the Video Memory Address.....	37
11.3. The Saved Background.....	37
11.4. The Mask: Transparency for the Price of One Table.....	38
11.5. Interleave: the Loop Body Written Twice	39
11.6. The Order Within a Frame: Why It Works.....	39
11.7. Erasing.....	40
11.8. Flicker: the One Thing CGA Could Not Offer	40
11.9. Eight Calls to a Routine That Does Nothing	41
12. Animation: Frame Chains and Living Tiles.....	42
12.1. A Chain Is Just Pointers Lying Next to Each Other	42
12.2. The Advance Does Not Live Where You Expect.....	43
12.3. Which Chains Live in ANIMTB	43
12.4. The “Tail of ANIMTB” Is Not a Tail.....	44
12.5. Living Tiles	45
12.6. Two Calls Lifted Out of ANIMTL	45
13. Light and Darkness	46
13.1. Who Decides Whether It Is Dark	46
13.2. What DARKEN Puts Out.....	47
13.3. The Physics Does Not Change At All	47
13.4. What Stays Visible	47
13.5. Darkness Does Not Touch the Sprites.....	48
13.6. Picking Up the Torch Mid-Room.....	49
13.7. LITMSK — the Scheme That Lost	49
13.8. The Order Within a Frame.....	49
Part IV. The Game	51
14. Items, Inventory, Doors, Score	51

14.1. An Item Is Four Bytes in ELMTAB	51
14.2. Inventory: Five Slots and No Eviction	51
14.3. A Door Opens on the Picture, Not on the Code.....	52
14.4. The Doorway: Seven Bytes for Closed and Open	52
14.5. SEENMAP: One Bit per Object, Sixteen per Room	52
14.6. The Amulet: 55 Frames of Invulnerability	53
14.7. The Panel: DIGELM Is a Digit Font	53
14.8. Lives, and the Last Three Bytes of ELMTAB.....	53
14.9. ELMTAB Is Mapped End to End.....	54
14.10. A Quirk in the Inventory Clear.....	54
15. Player Physics: Ten States, the Jump, Room Edges	55
15.1. The Dispatcher.....	55
15.2. What the States Are	55
15.3. Who Switches State, and When.....	56
15.4. The Jump	56
15.5. Support and Death	57
15.6. Conveyor Belts	58
15.7. Room Edges.....	58
15.8. The Treasure Room Does Not Let You Fall.....	59
16. Enemies: Four Behaviours and the Hit Box.....	60
16.1. The Dispatcher and Half Speed	60
16.2. The Behaviour Type Packed into OBJKND.....	60
16.3. The Four Behaviours	61
16.4. The Hop Arc	61
16.5. Turning Around	61
16.6. The Hit Box	62
16.7. Which One PLRHIT Picks	62
16.8. The Sword Against Death.....	63
16.9. How a Dead Enemy Leaves the Screen	63

16.10. One More Single-Frame Marker	64
17. Progression and Score: Pyramid Laps, the Treasure Room, the Bonus Life	64
17.1. The Score Is Three BCD Bytes	64
17.2. PTSTAB — What Is Worth What	65
17.3. The Addition.....	65
17.4. The Bonus Life	65
17.5. Drawing the Score	66
17.6. LEVEL Is a Lap Number, Not a Menu Choice	66
17.7. The Treasure Room	67
17.8. The Treasure Room Layout: Two Windows	67
17.9. Eight More Layouts, Ready and Waiting	68
17.10. The Lap Closes	68
Part V. Peripherals.....	70
18. Sound: the PIT, the Sequencer, the Scores, the Priorities	70
18.1. The Timer Moved to 100 Hz	70
18.2. The Score Format	70
18.3. The Sequencer	71
18.4. The Frequency Sweep	72
18.5. The Twelve Scores	72
18.6. Priority Is the Sound Number.....	73
18.7. Its Own INT 62h.....	73
19. Input: Keyboard, Joystick, and Its Self-Calibration.....	74
19.1. One Format for Both Devices.....	74
19.2. The Joystick Reading: Counting Port Reads	75
19.3. A Dead Zone One Count Wide.....	75
19.4. A Ratchet: the Thresholds Creep Outward and Stay	76
19.5. Jumping on the Edge	76
19.6. The Keyboard Outside the Game	77
19.7. The Key Redefinition Screen	77

19.8. Controls on a Modern Machine: a Queue, Not a State	78
19.9. Ten Zeros, Placed by the Linker.....	79
20. Text, Menus, Settings Screens	80
20.1. There Is No Text Mode	80
20.2. Strings Without Terminators	80
20.3. Title Screen and Main Menu	81
20.4. Highlighting the Selection with Colour.....	81
20.5. The Menu Turns NumLock On	81
20.6. The Key Redefinition Screen	82
20.7. Four Captions Waiting in the Ribbon.....	83
20.8. How the Player Moves Between Screens	83
Part VI. Appendices	85
21. The Complete Map of the Image	85
22. What the Image Does Not Tell Us	86
23. A Short Glossary.....	86
Part VII. The Plates	89
24. The Tile Set.....	89
25. Sprite Frames	90
26. Items and the Status Panel	91
27. The Ninety-Nine Rooms	92
Instead of an Afterword	92
On the Reconstructed Source.....	93
On Copyright	93

About This Book

This is a book about how ninety-nine rooms, seven items, four enemy behaviours, twelve tunes, two input devices — and the stack — were made to fit inside 32,228 bytes. The subject is **MX4C.COM**, the CGA build of the 1984 Parker Brothers release of *Montezuma's Revenge* for the IBM PC.

Every claim in these pages carries an address. **0439h** is not decoration; it is a place in the file. You can go there and look for yourself, because the reconstructed source listing ships alongside this book and assembles back into the very same file, byte for byte. A paragraph without an address is a retelling, and retellings do not belong here.

Where the bytes are silent, so is the book. What the author wanted, what he changed his mind about, what he set aside — the file does not keep any of that. It keeps only what he finished. The last chapter is about exactly that boundary.

One more thing is worth saying up front, because it colours everything that follows. The machine this game runs on does almost nothing for the programmer. No sprites, no scrolling, no sound chip, no frame sync, no fast multiply. Every single thing you see on screen and hear through the speaker was put there by hand, in assembly language, by people counting processor cycles on paper. This book is, among other things, a record of how well that was done.

How to Read This Book

You do not need to be an assembly programmer to read it. You do need four conventions, and here they are.

Addresses. A number with an **h** after it is hexadecimal: **0439h** is 1081 in ordinary counting. Every address in this book is an offset inside the one 32,228-byte file, and the file starts at **0100h**, so the addresses run from **0100h** to **7EE3h**. When an address appears in brackets after a name — **PUTMAP (1000h)** — that is where the thing lives.

Listings. Quoted code looks like this:

```
START:  mov     sp, OFFSET STKTOP      ; 03E2  own stack: SP to the top of the 0103..0300 area
```

On the left is one processor instruction; after the semicolon comes the address it sits at, and then a plain-English note about what it does. **mov** copies, **add** adds, **cmp** compares, **jmp** jumps, **call** calls a routine and **ret** returns from one. **AX**, **BX**, **CX**, **DX**, **SI**, **DI**, **BP** and **SP** are the processor's eight working registers — the only places it can hold a number while it thinks. Running short of them is a recurring theme.

Names in capitals. **SCRMAP**, **DRWSPR**, **FRMWAIT** and the rest are names for places in the file. The original names did not survive, so these were chosen during the reconstruction to say what the thing does. The bytes they point at are the real ones.

Sizes. A byte is eight bits; a word is two bytes; a kilobyte is 1,024 bytes. A tile is an 8×8 square of screen. A pixel on this display is two bits, so four pixels share a byte — which is why so much of this book counts in bytes rather than pixels.

If a paragraph still resists, skip it. The chapters stand on their own, and the pictures at the back are worth a visit at any point.

Part I. The Machine and the Startup

1. What This Game Is, and What This File Is

Montezuma's Revenge is a platformer: the player guides Panama Joe through ninety-nine rooms of a pyramid, collecting keys, opening doors and dodging skulls, snakes and spiders. The IBM PC version shipped in 1984, published by Parker Brothers, on a bootable floppy disk.

Before taking the bytes apart, it is worth knowing where the game came from.

Montezuma's Revenge was designed and written by Robert Jaeger, alone, on an Atari 800, in forty-eight kilobytes, under the name of his own outfit, Utopia Software. He was sixteen. The theme and the title came from his friend Mark Sunshine: do something Mesoamerican, he said, and call it *Montezuma's Revenge*. The name is the common American expression for what happens to a tourist who eats the wrong thing in Mexico. A game about the treasures of an ancient civilization went out under that title on purpose, and the joke has aged rather well.

The game was shown at the Consumer Electronics Show, where Utopia Software had rented a booth roughly the size of a broom closet — a long way from the big companies, and rather closer to the adult film industry, since in those years all of it was exhibited on the same floor. That is where a publisher spotted it.

The publisher was Parker Brothers of Salem, Massachusetts, a board game house that had turned a hundred and one years old that year. Monopoly, Cluedo, Risk. In 1982 they moved into video games in a hurry: they bought up licences to other people's hits, outbidding Atari, Mattel and Coleco all at once, and announced a line-up for every machine on the market. Then, at the end of 1983, the video game market collapsed underneath the entire industry, and most of that line-up never shipped.

Montezuma's Revenge arrived at exactly that moment. A year later Parker Brothers would leave video games for good. That this particular title got out of the door at all, on time, on a machine nobody had written an action game for yet, is the first remarkable thing about it.

Having licensed the game, the publisher immediately had to compress it by a factor of three: forty-eight kilobytes had to become sixteen to fit in a cartridge. Chase Sebor and Bob Halliday did that work, and getting a game of this size into sixteen kilobytes is a considerable feat in its own right. The author himself was by then putting in sixteen-hour days. Had the day held thirty-four hours, there would almost certainly be a finale here worthy of the rest of the game; the day held twenty-four, the publisher decided the game did not need one, and by his own account the author was relieved to be told to stop worrying about it. So the pyramid finishes where the next pyramid begins, only meaner. The ninety-nine rooms as the world knows them — the branching routes, the difficulty curve — took their final shape somewhere between the author and the publisher's team on the way from the Atari to the cartridge. Which room came from which hand the file does not record.

The hero changed hands along with the game. In Jaeger's own version he is Pedro and wears a different hat; the publisher renamed him Panama Joe, and Panama Joe he has stayed. The ZX Spectrum release a year later was even titled after him. In December 1984 *Electronic Games* made the result its game of the month.

That endless loop turned out to be the right call, incidentally. Arcade games of the period did not end either; they got harder until you ran out of quarters. The ever-nastier pyramid is now one of the things people remember about the game.

And here is where the part that this whole book is about begins. Jaeger's code was welded to Atari's custom chips: hardware sprites, display lists, a four-voice sound chip. A port to the IBM PC is therefore not a port at all. It is writing the game again, from nothing, to somebody else's design.

Because the IBM PC has none of the above. Here is the complete list of what this machine does for the programmer on its own:

(this space intentionally left blank)

No sprites: everything that moves is drawn by hand, and the background under each figure has to be saved first, because there is no other way to erase it afterwards. No scrolling. No frame synchronization available to a program, so frame length has to be measured out with an empty loop and counted in processor cycles. No sound chip: there is one bit wired to a speaker, and one timer channel to toggle it at the right frequency. One voice. Square wave. That is the whole instrument.

Four colours, one of which is the background, and they may be chosen only from two fixed sets. Screen rows live in memory every other one — even rows in one half, odd rows in the other, following the television raster. The processor runs at 4.77 MHz and charges over a hundred cycles for a word multiply; a divide costs more still. Multiplying is expensive in this climate, and much of what follows in this book is the story of the ingenuity that goes into not multiplying.

And thirty-two thousand bytes for everything at once: code, artwork, rooms, tunes, text and the stack.

The developer on the Atari had a chip behind him that played four voices by itself. Here the fourth voice is the programmer. So is the third, and the second, and the first; and so are the sprites, and the frame timing, and every piece of arithmetic the processor considers beneath its dignity to do quickly.

If the machine does nothing, everything has to be done by hand. Which raises the question of what people wrote games in at the time — and the list of options turns out to be short.

BASIC is in ROM, every owner of the machine has it, and writing in it is quick and pleasant. It is also interpreted: a single line costs about as many cycles as this game spends drawing eight rows of a tile. You will not get twenty-four frames a second out of it.

C existed, and could even be bought. Lattice C, June 1982, the first C compiler for the IBM PC. Five hundred dollars. It wanted ninety-six kilobytes of memory and two floppy drives — that is, a machine more expensive than the one the game would later be sold for. And it produced code noticeably longer than hand-written assembly: where a programmer writes `INC AX` twice because that is a byte shorter and a cycle faster, a compiler will honestly add a constant, call through the stack, and save registers just in case.

Pascal arrives in earnest in November 1983, when Turbo Pascal goes on sale for fifty dollars instead of five hundred. By then this game is already being ported.

So the question “why assembly?” does not really arise here. A different one does: what would you measure a twenty-four hertz frame with, in a language that hides from you how many cycles a line costs? In this game the frame budget is measured in cycles, and one turn of the idle loop is worked out from the processor's instruction timing table. A compiler, however good, simply cannot take part in that conversation.

Assembly was not chosen out of affection. It was chosen because no alternative that fits inside 4.77 MHz and thirty-two kilobytes physically existed. Cheap, decent C for the PC arrives only in 1987, three years after this floppy shipped. The era in which a human being counted cycles by hand was a short one, and it left behind a small number of programs that outlived both their machine and the language argument. This is one of them: played then, re-released since, and known by name to people who have never seen an IBM PC.

That, then, is the situation: a machine that helps with nothing, a language that contains nothing but processor instructions, and a shipping date. Let us see what came of it.

The file under examination is **MX4C.COM**, **32,228 bytes**. The **.COM** format is the simplest thing DOS supports, and in this case that is an advantage:

- one single segment; **CS = DS = ES = SS**, and every address is just an offset inside those 64 kilobytes;
- the file loads at offset **100h**, so its first byte gets address **100h**, which is also where the pseudo-segment starts: **ORG 100h**;
- **everything** lives inside that one file: code, tables, tile and sprite artwork, text, buffers, and the stack too. No re-source loader, no overlays, and not one further trip to the floppy for the rest of the session.

The first three bytes are **JMP START (0100h)**, and that is what DOS executes.

In broad strokes the image looks like this:

range	contents
0100h	JMP START
0103h...0300h	the stack area; zeros in the file
0305h, 0368h	two 99-byte tables: room → floor and position
03CDh...03E1h	global sound and frame variables
03E2h...07C6h	startup, sound driver, lighting, object table
07C7h...15B4h	the room builder, its working cells, SCRMAP
15B5h...253Dh	235 tiles of 8×8, the GAME OVER banner, item strip
2610h...346Fh	element selection, pointers, and 99 rooms of bytecode
34A0h...3B7Fh	support lists, living tiles, tile animation
3B80h...4B9Fh	animation tables and 37 sprite frames
4BA0h...5B9Fh	saved-background buffer under sprites, 16 × 100h
5BA0h...5FFFh	transparency masks, sprite output, joystick
6000h...7FFFh	game logic, the state machine, text and menus

Look at the proportions. The artwork — tiles, sprite frames, masks, the background buffer — eats more than half the file. The entire game proper: physics, enemies, scoring, menus, sound, the state machine, fits into the last eight kilobytes, a quarter of the whole.

Every game of that period is built this way, and it is worth remembering, because the rest of this book is about it. Code weighs almost nothing. Artwork weighs everything. And the whole art of the thing is to have as little artwork as possible while making it look like a great deal: ninety-nine rooms that are drawn nowhere; thirty- seven frames out of which all the animation in the game is assembled; two hundred and thirty-five little 8×8 squares from which an entire pyramid is built.

The module-by-module map is [Chapter 21](#).

2. From Floppy to First Instruction

When DOS launches a **.COM** file, it hands the program the whole free memory block, puts a 256-byte Program Segment Prefix (PSP) at the front of it, and the body of the file immediately after. On entry, **CS = DS = ES = SS** all point at the PSP, **IP = 100h**, and **SP** sits at the very end of the segment — which is to say the stack starts out somewhere far past the end of the program.

The game is not content with that. It has no intention of relying on whatever the system happened to leave it. The first thing **START** (**03E2h**) does is take the stack into its own hands:

```
START:
mov     sp, OFFSET STKTOP      ; 03E2  own stack: SP to the top of the 0103..0300 area
```

Why **301h**? Because **the stack lives inside the file itself**. Bytes **0103h...0300h** are 510 zeros written straight into **MX4C.COM**: half a kilobyte of immaculate emptiness that the publisher stamped onto every floppy sold. The top of the stack is set at their upper boundary, and from there the stack grows downward into those zeros.

There is room to spare. At its deepest it reaches **02ABh** — **86 bytes out of 510** in use — and that happens inside **PUTMAP** (**1057h**), the most deeply nested drawing routine there is. The remaining four hundred and some bytes are headroom for the timer handler, which shows up a hundred times a second and does not ask whether now is a good moment.

Next **START** lines the segments up on itself and writes the classic return-to-DOS prologue:

```
mov     ax, cs                ; 03E5  CS -- the one and only segment of a .COM image
mov     ds, ax                ; 03E7  DS = CS
mov     ss, ax                ; 03E9  SS = CS: the stack sits inside the image itself
mov     es, ax                ; 03EB  ES = CS
push    ds                   ; 03ED  segment of the PSP
mov     ax, 0                 ; 03EE  AX=0 -- the return offset
push    ax                   ; 03F1  offset 0 -- the RET at 048A will pop this pair
```

PUSH DS and **PUSH 0** place the address **PSP:0000** on the stack, where DOS keeps an **INT 20h** instruction — “terminate program”. A far **RET** here would hand control back to the system.

The game never uses it. There is no way out: this is a program from a bootable floppy, and it has nowhere to return to — there is no DOS behind it and no command prompt. Those five bytes of prologue were written by reflex, the same reflex that makes you close the door behind you in an empty room. They also explain why **SP** from this point on is **2FDh** rather than **301h**.

3. Initialization: Video, Timer, Vectors

START fits in 87 bytes and does exactly four things: sets the screen mode, silences the floppy drive, takes over the system timer, and hangs two interrupts of its own. After that it does not return — there is nowhere to return to.

3.1. Video Mode

```
xor     ah, ah                ; 03F2  AH=0: the "set video mode" call
mov     al, VMODE_CGA         ; 03F4  AL=4: 320x200, four colors
int     10h                   ; 03F6  video BIOS
mov     ah, 0Bh               ; 03F8  AH=0Bh: the "set palette" call
mov     bh, 1                 ; 03FA  BH=1: pick the color set
mov     bl, 0                 ; 03FC  BL=0: color set 0
int     10h                   ; 03FE  video BIOS
```

CGA mode 4: 320×200, four colours, video memory at **B800h**. The second call, **AH = 0Bh** with **BH = 1**, selects palette number 0 — green, red, brown.

That is the entire artistic freedom available: three inks and a background. The other set is cyan, magenta and white, which suits an Aztec pyramid distinctly less well. They took the first one: green, red, brown. Earth, blood and stone — exactly right for a game about a Mesoamerican temple, and a good example of making a virtue out of a choice between two.

The brick that looks purple-pink in many screenshots is not the game's doing at all; it is whatever stands between its output and your eye: the intensity bit, the monitor, the composite output, the emulator. The program said its piece in two instructions.

3.2. The Floppy Motor

```
mov    al, FDC_OFF          ; 0400 C0h: motor off, controller held in reset
mov    dx, PORT3F2          ; 0402 the floppy drive control port
out    dx, al               ; 0405 the drive motor is shut down
```

Port **3F2h** is the floppy disk controller's digital output register. Bits 4...7 spin up the motors of drives A...D, bit 2 keeps the controller out of reset, bit 3 enables its DMA and interrupts. The value written is **C0h**: bits 6 and 7 set, and **the motor bits cleared**.

The game has just loaded from the floppy, the motor is still turning — and the very first thing it does is stop it. There is nothing more to read from the disk; it is all in memory now, and a drive spinning for the whole session makes noise, gets warm, and wears out the one piece of media the buyer owns. Three instructions, and a small kindness toward the person who bought the disk.

3.3. The Sound Driver: **SNDINI (058Eh)**

This is the most delicate part of the startup.

The IBM PC has no sound chip. It has one bit in a port wired to a speaker, and a timer channel to toggle that bit at the frequency you want. No voices, no volume, no envelopes, no noise source for explosions. If you want a tune, you play it yourself, note by note, in an interrupt handler, a hundred times a second, in between the frames of the game — and you make sure you do not eat the picture's time doing it.

On the machine this game was originally written for, all of that was the job of a dedicated four-voice chip. Here that chip is replaced by **SNDTICK (04D6h)**: one hundred and eighty-four bytes of code. It is one of the best bargains in the file.

The whole routine runs under **CLI (058Fh)** and only opens interrupts again at the end (**05C5h**): a tick arriving in the middle of the rewiring would find a half-assembled vector and take control off into nowhere.

```
mov    ax, 0                ; 0590 AX=0 -- segment of the interrupt vector table
mov    es, ax               ; 0593 ES = 0: the interrupt vector table
mov    di, VEC08            ; 0595 DI -- the INT 08h vector, the timer tick
```

ES:DI = 0000:0020h — the **INT 08h** vector, the system timer interrupt. The old value is saved in full:

```
mov    bx, es:[di]          ; 0598 old offset out of the vector
mov    OLD080, bx           ; 059B old offset is saved away
mov    bx, es:[di+2]        ; 059F old segment out of the vector
mov    OLD08S, bx           ; 05A3 old segment is saved away
```

and its own handler goes in its place, with two **STOSW** in a row, since **DI** is already pointing where it needs to be:

```
mov    ax, OFFSET SNDTICK   ; 05A7 AX -- offset of the new handler
cld                                ; 05AA STOSW will run forward
stosw                                ; 05AB offset half: INT 08h -> SNDTICK
mov    ax, cs               ; 05AC CS -- segment the handler lives in
stosw                                ; 05AE segment half of the vector
```

04D6h is **SNDTICK**, the sound sequencer. Then the timer itself is reprogrammed:

```
mov    al, PIT_M3           ; 05AF 36h: channel 0, mode 3, both bytes of the divisor
out    PORT43, al           ; 05B1 PIT control word
mov    bx, PIT_DIV          ; 05B3 divisor 2E9Ch = 11932 ticks
mov    al, bl               ; 05B6 low byte of the divisor into AL
out    PORT40, al           ; 05B8 low byte of the divisor out to channel 0
mov    al, bh               ; 05BA high byte of the divisor into AL
out    PORT40, al           ; 05BC high byte: the timer now runs a hundred times a second
```

36h in the PIT control register means: channel 0, write low then high divisor byte, mode 3 (square wave). The divisor `PIT_DIV = 2E9Ch` = 11,932. The PIT input frequency on the PC is 1,193,182 Hz, so

1193182 / 11932 = 100.0 Hz

The game moves the system timer from its usual 18.2 Hz to **100 hertz**, and that round number is chosen deliberately: every note duration in the game is counted in these ticks, and ten milliseconds is a unit a human being can think in, rather than a machine.

The price of the round number is the machine's time-of-day clock. The BIOS counts the hour from a little over eighteen ticks a second, and nobody told it there are now a hundred. From this second on the clock runs slow, and keeps running slower.

For the game this means nothing at all: it booted from its own floppy, there is no DOS underneath it, and there is nowhere to return to. A reboot puts the clock right ([Chapter 18](#)). Given the choice between an accurate clock nobody is looking at and a sound system that keeps perfect time, the game picked the sound.

The finishing touch is to unmask the interrupts in the 8259 controller:

```
in    al, PORT21          ; 05BE IRQ mask of the 8259 controller
and   al, 0FCh            ; 05C0 clear bits 0 and 1
out   PORT21, al          ; 05C2 timer IRQ0 and keyboard IRQ1 are open
```

Clearing bits 0 and 1 enables **IRQ0** (timer) and **IRQ1** (keyboard).

The ordering here matters, and it is right. The vector is replaced before the frequency changes. Do it the other way round and a tick or two would slip through between the new frequency and the new vector, landing in the BIOS handler, which has no idea it was just sped up fivefold. Nothing terrible would happen — but bugs like that reproduce once in a hundred launches and take a week to find. Somebody here had already spent that week, once, on some earlier project, and was not about to spend it twice.

3.4. Two Interrupts of Its Own

```
mov    ax, 0              ; 040A AX=0 -- segment of the interrupt vector table
mov    es, ax             ; 040D ES = 0: the interrupt vector table
mov    ax, cs             ; 040F CS -- segment the handler lives in
mov    es:[VEC62+2], ax   ; 0411 segment half of the INT 62h vector
mov    ax, OFFSET SNDAPI  ; 0415 offset of the sound API handler
mov    es:[VEC62], ax     ; 0418 offset half: INT 62h -> SNDAPI
mov    ax, 0              ; 041C zero again for the second vector
mov    es, ax             ; 041F ES = 0 a second time, though it is zero already
mov    ax, cs             ; 0421 CS -- segment the handler lives in
mov    es:[VEC67+2], ax   ; 0423 segment half of the INT 67h vector
mov    ax, OFFSET PUTMAP  ; 0427 offset of the map output handler
mov    es:[VEC67], ax     ; 042A offset half: INT 67h -> PUTMAP
```

The second pass is an exact copy of the first, including reloading `ES` with zero when it is already zero: `MOV AX,0` and `MOV ES,AX` at `041Ch` and `041Fh` change nothing. Five bytes buys both blocks reading identically. Vector setup written from the same template every time does not let you down, and this happens once per session and costs the frame nothing.

`VEC62 = 62h × 4`, `VEC67 = 67h × 4`: the game gives itself two software interrupts.

vector	address	routine	purpose
INT 62h	04A8h	SNDAPI	ask for a sound, or ask whether one is playing
INT 67h	1000h	PUTMAP	lay a window of the map into video memory

Both routines sit in the same segment and could have been reached by an ordinary `CALL` at half the cost. Instead they are given two vectors and the extra cycles of an interrupt entry.

The reason is taken up in [Chapters 7](#) and [18](#). In short: both the map output and the sound are written as free-standing services that can be called from a distance, from somebody else's segment, without knowing where

they live. For a company shipping this game on seven different machines inside a single year, that is an entirely sensible interface: a subsystem you call through a vector can be lifted out and dropped into another project without touching a line inside it.

Here the foreign segment never turned out to be needed, and the vector entry stayed: headroom laid in for a future that did not arrive. It cost eight bytes and a few cycles a frame, and it is exactly the sort of thing you build when you expect to do this again next quarter.

3.5. What Is Left, and Where Control Goes Next

```
mov    GSTATE, 8          ; 042F state 8 -- the main menu
mov    LEVEL, 1           ; 0434 level 1
```

The state machine starts in state eight (the title screen), and the difficulty level at one. There is no **RET** here: **START** falls through into **MAINLP** (0439h), which begins with the very next instruction. It has nowhere to return to anyway — nobody called **START**; control arrived here by a jump from 0100h.

Notice what **START** does *not* touch: not the tiles, not the object table, not the score. All of that is **VIDINI**'s job (0644h), and **VIDINI** is called by state 0 — that is, only after the player has watched the title screen and left the menu. The split is sensible: half that data is not needed on the title screen, and a new game has to start from a clean slate regardless.

3.6. VIDINI — Setting Up a Session

VIDINI (0644h) sets mode 4 and the palette again, restores the tiles from their reference copies (**TILRST**, 0652h), and lays out the player:

```
mov    PLRST, 5           ; 0655 player state 5 -- falling
mov    ax, OFFSET AN_POSE2 ; 065A frame chain: the standing pose
mov    ANIMPTR, ax         ; 065D the player's frame chain is chosen
mov    ax, AN_POSE2        ; 0660 first word of the chain -- address of the frame
add    ax, 2               ; 0663 step over the frame header
mov    BYTE PTR OBJTBL, al ; 0666 low byte of the player's frame address
mov    PLR_ANH, ah         ; 0669 high byte of the frame address
mov    ANIMLEN, 0          ; 066D one frame in the chain: highest number is 0
mov    PLR_X, 1            ; 0673 X = 1: the left edge of the room
mov    PLR_Y, 0Eh          ; 0678 Y = 0Eh: the fourteenth scan line
mov    PLR_W, 6            ; 067D the player's sprite is 6 bytes wide
mov    PLR_H, 14h          ; 0682 the player's sprite is 14h = 20 lines tall
```

Here the object format becomes visible: field +0 is **a pointer not to the frame but to its bitmap**, that is, the frame address plus the two header bytes. The addition is done once, here, so that it need not be done in the sprite output — and the sprite output is called up to eight times a frame, where every cycle counts. The player measures 24×20 pixels.

Then the starting place in the pyramid, and the supplies:

```
mov    FLOOR, 0FFh        ; 068C floor FFh -- no room of the pyramid picked yet
mov    ROOMPOS, 9          ; 0691 position 9 on the floor -- where play begins
mov    INBITS, 1           ; 0696 only object 0, the player, is left live
mov    LIVES, 5            ; 069B five lives
```

and three clears in a row — the inventory (5 bytes), the collected map (**SEENMAP**, 0CAh = 202 bytes) and the score (3 bytes):

```
mov    cx, 5               ; 06B7 CX=5 -- that many bytes
mov    di, OFFSET ITEMS    ; 06BA DI -- start of the inventory
rep stosb                  ; 06BD clears 5 bytes out of the 10
mov    al, 0               ; 06BF AL=0 -- what to fill with
mov    cx, 0CAh            ; 06C1 CAH = 202 bytes of the seen map
mov    di, OFFSET SEENMAP  ; 06C4 DI -- start of the seen map
rep stosb                  ; 06C7 not one room has been seen yet
mov    al, 0               ; 06C9 AL=0 -- what to fill with
mov    cx, 3               ; 06CB the score is three bytes of packed BCD
mov    di, OFFSET SCORE    ; 06CE DI -- start of the score
rep stosb                  ; 06D1 the score is zeroed
```

A three-byte score is six binary-coded decimal digits; the arithmetic on it is [Chapter 17](#)'s business. Three `REP STOSB` in a row rather than one over the whole block, because there are three pieces to clear and they live in different parts of the file: the inventory in the game logic, the collected map among the map buffers, the score among the score tables. There is no single block of variables in this program — which is what the next chapter is about.

4. The Memory Map: Where Everything Lives

The map of the image, module by module, is in the appendix, [Chapter 21](#). This chapter is only about what follows from it. And what follows is one thing: the layout of this program is organized not by tidiness but by speed of access.

4.1. One 64K Segment for Everything

A `.COM` file gives you no separate data segment and no separate stack. Where the pieces sit in the file *is* the memory map, and the only difference between “code” and “data” is where control ends up. Get a jump address wrong and the tile table starts executing.

Several consequences follow, and they will come up again and again:

- **Buffers live in the file as zeros.** `SCRMAP` (1000 bytes), `OBJTAB` (4096 bytes), `SEENMAP` (202 bytes) are all faithfully written onto the floppy, even though there is nothing in them: they fill up at run time. Five-odd kilobytes of zeros, a sixth of the image. Allocating them past the last byte of the program would have been free, but then the address of each buffer would have to be computed at startup. Space is cheaper than a cycle — a rule that will recur throughout this book;
- **Variables are scattered through the code** rather than gathered into one block. `FRAMEC` lives at `03CFh`, among the sound variables; `ROOMNO` at `1090h`, between the room builder and its own working cells; `LIVES` at `6574h`, inside the game logic. Each one sits next to the code that touches it. That takes a little more work to read from outside — and makes it a great deal easier to write and change from inside: you edit one module and never go running back to a central header for every cell;
- **Tables sit next to the code that walks them.** `STTAB` stands directly in front of the ten state handlers (`60E0h`), `PRIMTAB` in the middle of the room builder (`0B5Fh`).

4.2. The Stack Area

`0103h...0300h`, 510 bytes of zeros. Top at `301h`, working depth 86 bytes. A sixfold margin — the one place in the file where memory was spent freely.

The margin is well earned: the stack is shared with the timer handler, and that arrives at any moment, including the deepest call in the drawing code. Working out exactly how much is needed is a day's job; setting aside half a kilobyte of zeros is a minute's. On a deadline that is not a shortcut, it is the correct trade.

4.3. Global Sound and Frame Variables

The densest stretch of the file is `03CDh...03E1h`: twenty-one bytes immediately before `START`. Almost all of it belongs to the sound, and rightly so: of all the game's subsystems the sound is the only one that runs in an interrupt, which means every piece of its state has to live in memory rather than in registers.

address	name	purpose
<code>03CDh</code>	<code>PAD03CD</code>	drawing page segment — see below
<code>03CFh</code>	<code>FRAMEC</code>	frame counter, +1 per pass of

address	name	purpose
		the main loop
03D0h	LASTKEY	scan code of the last key pressed
03D2h	SNDFRQ	current PIT divisor for channel 2
03D4h	SNDTCK	ticks until the next read from the score
03D5h	SNDPTR	read offset within the score
03D7h	SNDSEG	segment of the score
03D9h	SNDACT	1 — a sound is playing
03DAh	OLDT080	saved INT 08h vector, offset
03DCh	OLDT085	the same, segment
03DEh	SNDRPF	1 — a frequency sweep is in progress
03E0h	SNDRPC	how many sweep steps remain

PAD03CD is the first trace in this file of another machine altogether. The word is read exactly once, at the start of SCRTTL (7C02h):

```

mov    ax, PAD03CD      ; 7C40 AX -- a word read just as it stands
and     ah, 8           ; 7C43 bit 3 of the high byte
je      LSCRTT1          ; 7C46 clear
call   NOP0             ; 7C48 empty routine

```

It tests bit 3 of the high byte, that is, bit 3 of cell 03CEh. No routine writes to that word, and in the file it holds zeros — so the condition is never true, and NOP0 (60C0h) consists of a single RET in any case.

Eleven bytes that load from the floppy every session and never fire. The explanation is in the sibling version. The same game has an MX16.COM for the IBM PCjr, and there, in place of PAD03CD, sits the segment of the page currently being drawn into, and in place of NOP0 sits a call to the routine that flips video pages. The whole test reads as: “if we drew into the second page, show it.” CGA has one page, there is nothing to flip, the call has been hollowed out to a RET — and the cell stayed where it was, still carrying the name it had in the head of whoever wrote it.

So 03CDh is not litter. It is a seam between two versions of the same game, written side by side in the same year, and the file wears it openly. There are several more such seams, and they are gathered together in [Section 11.8](#); taken as a set they are one of the more interesting things in the image, because they let you see two ports at once.

4.4. The Pyramid: Two Tables That Describe It Exactly

0305h and 0368h, 99 bytes each. The first gives a room’s position on its floor, the second its floor number. They are the exact inverse of ROOMBAS:

```
ROOMBAS[RMFLR[N]] + RMPOS[N] = N    for all 99 rooms
```

The pyramid widens by two rooms per floor — 3, 5, 7, 9, 11, 13, 15, 17, 19 — and the rows are centred. Nine floors, ninety-nine rooms, and the whole geometry described exhaustively by two tables.

At the same time **no instruction in the code touches them**: neither 0305h nor 0368h appears as an immediate or as an offset anywhere. The game computes the same thing by addition through ROOMBAS (09C5h) inside ROOMID — twelve instructions instead of two tables.

The sequence of events is easy to reconstruct: first two lookup tables were built, then somebody noticed that adding through **ROOMBAS** is shorter and cheaper. The tables were left in place, and that was the right call — there was room in the image, and pulling out something that might still be wanted is a job for the last week before shipping, not the first. They also make an excellent reference: a hundred and ninety-eight bytes that state the shape of the pyramid more clearly than any comment could.

4.5. The Room Builder's Working Cells

108Dh...11A4h is the room builder's block of housekeeping variables. One of them looks superfluous right up until you count the free registers: the 8088 has eight, half of them come with special duties, and in a hot loop there are none left at all.

CMDLEN (10A1h) is a word that the primitives fill in before handing control to **FILLRC**:

```

mov     CMDLEN, 4           ; 0B89 the command takes four bytes
jmp     FILLRC              ; 0B8F fill the rectangle in
PARMSPL:
mov     bl, CMDP1           ; 0B92 BL -- the packed parameters
mov     bh, bl              ; 0B96 BH -- a copy of the same
shr     bl, 1               ; 0B98 shift 1 of four
shr     bl, 1               ; 0B9A shift 2 of four
shr     bl, 1               ; 0B9C shift 3 of four
shr     bl, 1               ; 0B9E the high four bits of BL come down
mov     cl, bl              ; 0BA0 CL -- width minus one
xor     ch, ch              ; 0BA2 CH=0: the whole count now sits in CX
inc     cx                  ; 0BA4 CX -- width in cells
and     bh, 0Fh             ; 0BA5 the low four bits of BH
mov     dl, bh              ; 0BA8 DL -- height minus one
mov     CMDLEN, 3           ; 0BAA the command takes three bytes

```

and **FILLRC**, in its last instruction before returning, reads it back into **AX**:

```

mov     ax, CMDLEN          ; 0BC1 AX -- the command length, for BLDMA
pop     bx                  ; 0BC4 BX restored
ret     ; 0BC5 return: the command length is in AL

```

Why: after every command **BLDMA** advances the read pointer by exactly as many bytes as the command occupied, and takes that number from **AX**:

```

adc     RDPTR, ax           ; 084D read pointer forward by the length of the command

```

So **CMDLEN** is **the length of the current room bytecode command**, reported by the primitive back to the builder. Different primitives read different numbers of parameters: a rectangle with explicit width and height takes four bytes, the packed variant via **PARMSPL** takes three.

A value like that belongs in a register, and usually goes there. Not here: between the point where the length is known and the point where it is needed sits the fill loop, and the fill loop wants all eight registers, including the ones that look free. So the number goes out to memory and comes back into **AX** after the fill is done. The cell serves as a ninth register, because the 8088 does not have a ninth register.

Part II. The Heart of the Program

5. The Main Loop and a Ten-State Machine

Everything that happens in this game — the title screen, the menu, running around the pyramid, dying, the treasure room and the **GAME OVER** banner — turns inside a single loop of thirty instructions. No scene manager, no event queue, no object table with virtual methods. A state byte, a table of ten addresses, and a jump through it.

5.1. Thirty Instructions That Run for the Whole Game

MAINLP (0439h) is the entire main loop. Four jobs per pass:

```
MAINLP:
    call    FRMWAIT                ; 0439 wait out the frame: this is what holds the game's pace
    inc     FRAMEC                 ; 043C frame counter, +1 for every pass of the loop
    mov     bx, OFFSET STTAB       ; 0440 BX -- table of the ten state handlers
    mov     bp, cs                 ; 0443 BP -- segment of that table, for DISPAT
    mov     al, GSTATE             ; 0445 AL -- number of the state we are in
    call    DISPAT                 ; 0448 call the handler at STTAB[AL]
    mov     LASTKEY, 0             ; 044B the frame's key starts out empty until one is read
    xor     al, al                 ; 0451 AL takes no part here, cleared along the way
    mov     ah, 1                  ; 0453 AH=1: is a key waiting in the buffer
    int     16h                   ; 0455 keyboard BIOS: is there one
    je      MAINLP                 ; 0457 the buffer is empty -- straight on to the next frame
```

The first line is confusing on a first reading. **The frame-length equalization stands at the top, not at the bottom.** The game first sits out the remainder of the previous frame and only then gets to work.

In practice it makes no difference: the loop is a ring, and where the ring begins is a matter of taste. But **FRM-WAIT** on the first line looks like preparation for the frame when it is in fact the tail of the one before.

CALL DISPAT is deceptive too. **DISPAT** (0D6Bh) does not return to its caller; it **leaves along an address from the table**:

```
DISPAT:
    xor     ah, ah                 ; 0D6B AH=0: the whole index now sits in AX
    shl     ax, 1                  ; 0D6D entries run two bytes apiece
    add     bx, ax                 ; 0D6F BX -- the table entry
    mov     cx, cs:[bx]            ; 0D71 CX -- address of the handler
    jmp     cx                     ; 0D74 off into the handler; its return goes to whoever called us
```

So the **RET** at the end of a state handler returns not to **DISPAT** but straight into **MAINLP** at 044Bh: the return address was pushed by **CALL DISPAT**, and **JMP CX** never touched it, and never meant to.

The trick is old and cheap: the dispatcher spends five instructions and zero bytes of stack, and the handler has no idea it was not called directly. The same **DISPAT** also serves the game's second state machine — the player states (65AFh, table **PSTTAB**). Eleven bytes of code doing duty for two subsystems at once, which is about as good a return on eleven bytes as this file offers.

5.2. Two Hot Keys

The rest of **MAINLP** handles two keys that work **in any state**, because they sit above the state machine:

```

    mov    LASTKEY, ax          ; 045D scan code and ASCII are handed on to the state handlers
    cmp    LASTKEY, KEY_F5      ; 0460 F5 -- back to the menu
    jne    LMAINL1              ; 0466 not F5 -- go and test for F10
    mov    GSTATE, 8            ; 0468 state 8 -- the main menu
    call   CLRTXT                ; 046D wipe the text off the screen
    jmp    short MAINLP         ; 0470 the key is dealt with, on to the next frame
LMAINL1:
    cmp    ax, KEY_F10          ; 0472 F10 -- pause
    jne    MAINLP               ; 0475 every other key is left to the state handlers
LMAINL2:
    xor    al, al                ; 0477 AL takes no part here, cleared along the way
    mov    ah, 1                ; 0479 AH=1: is a key waiting in the buffer
    int    16h                  ; 047B keyboard BIOS: polled while paused
    je     LMAINL2              ; 047D nothing there -- spin here, no frames go by

```

Pause on **F10** is done in the most direct way there is: an empty keyboard polling loop. The game stands still inside **MAINLP**, frames are not counted, physics is not computed — until a second **F10** arrives. Five instructions and no state variable at all.

The sound **carries on playing**. The sequencer lives on the timer interrupt and knows nothing about the pause: if the pause caught the treasure-room tune, that tune plays to the end and falls silent by itself. It is the one part of the program the pause does not reach, and it makes the pause feel oddly alive.

5.3. The State Table

GSTATE (60D0h) is a single byte, an index into **STTAB** (60E0h), ten words:

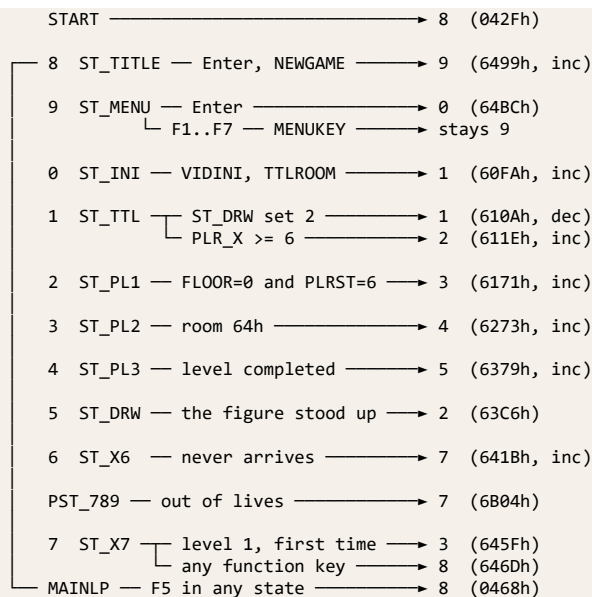
#	address	name	what it does
0	60F4h	ST_INI	VIDINI, TTLROOM, then state 1
1	6100h	ST_TTL	the attract scene: calls ST_DRW and AUTOCMD
2	6124h	ST_PL1	game frame: input, movement, collisions, output
3	61E5h	ST_PL2	game frame of a second kind
4	62B8h	ST_PL3	treasure-room frame and the move to a new level
5	637Fh	ST_DRW	demo frame, also a subroutine for ST_TTL
6	641Bh	ST_X6	INC GSTATE and nothing else
7	6421h	ST_X7	GAME OVER : the banner and the wait
8	6477h	ST_TITLE	title screen , Parker Brothers, waits for Enter
9	64A6h	ST_MENU	main menu , Enter starts the game

The order in the table says nothing about the order of events: the game starts in state eight, goes to nine, from there to zero, and only then into the game states. The numbering follows the order things were written rather than the player's route: the title screen and menu, done last, settled at the end.

Of the ten states one is not reached in play (**ST_X6**, below), two do not return control until the player presses a key, and three are the same game frame in three variations. The game proper occupies three of the ten table entries — which is a good measure of how much of a game like this is scaffolding around the part you actually play.

5.4. The Complete Transition Graph

Every place where **GSTATE** changes: there are sixteen. Fifteen live ones are on the diagram below; the sixteenth sits in **GOTTTL** (7BEBh), where control never arrives.



5.5. State 6: Wired Up, Waiting for One Instruction

`ST_X6` consists of three instructions: `INC GSTATE`, `STC`, `RET`. The only way into it is for `GSTATE` to become six, and six can only come from incrementing five.

Look at all sixteen writes to `GSTATE`. Out of state 5 (`ST_DRW`), `GSTATE` leaves by exactly one route — `MOV GSTATE, 2` at `63C6h`. There is no increment from five anywhere. So **state six never arrives**: six bytes and one entry in the jump table that do not execute once in a session.

What it was for is plain from the instructions themselves: skip a frame, then go to `GAME OVER`. That pause between the death and the banner is the breath games put in so the player has a moment to take in what just happened. The state is declared, its address is in the table, the code is written and it works; what is missing is a single instruction at the other end of the file.

This is exactly what a jump table is good for. A state can be connected or disconnected by one word, and number 6 is sitting off to one side, costing the game nothing and ready to go: write the missing increment and the breath before `GAME OVER` appears by itself. A design that lets you leave a finished feature parked, at a cost of six bytes, is a design doing its job.

5.6. Three States Instead of One: Splitting the Work

`ST_PL1`, `ST_PL2` and `ST_PL3` are not three different screens but three variations on the game frame. All three are built the same way:

update living tiles	ANIMTL
erase sprites	ERSALL
fetch a command	DIRBIT + DEMOCMD/READCMD
step the player	PLRSTEP / PLRTICK
move the enemies	ENEMOVE
— and all of that TWICE —	—
mark collisions	ANIMHIT
draw supports	PUTRCT
draw sprites	DRWALL2

The key detail: **the player step and the enemy movement run twice per frame** — `614Bh` and `6156h` in `ST_PL1`. The call chain for a single frame looks like this:

```

MAINLP -> FRMWAIT -> DISPAT -> ST_PL1 -> TILRST -> ERSALL -> DIRBIT ->
SCANTIL -> OBJTYPE -> DEMOCMD -> AUTOCMD -> ANIMTL -> BELTANI ->
PLRSTEP -> DISPAT -> PST_2 -> PST_6 -> MOVEPLR -> INLIST -> OBJMOVE ->
OBJMOVE -> ENEMOVE -> DIRBIT -> SCANTIL -> OBJTYPE ->
PLRSTEP -> DISPAT -> PST_2 -> PST_6 -> MOVEPLR -> INLIST -> OBJMOVE ->
OBJMOVE -> ENEMOVE -> ANIMHIT -> PUTRCT -> PUTMAP -> PUTMAP ->
DRWALL2 -> DRWSPR

```

This trick deserves an explanation, because it is a genuinely elegant piece of engineering. Everything that moves takes two small steps per frame instead of one large one, and is still drawn once. Collisions come out twice as accurate — Panama Joe does not shoot straight through a skull at speed — while the redraw, by far the most expensive part of the frame, does not get one cycle more expensive. Doubling the physics rate is nearly free precisely because physics is cheap and drawing is not, and somebody worked that out and acted on it.

If `PLRSTEP` returns `CF = 1` — the player has left the room or died — the second step is cancelled and the drawing is skipped entirely (`jb LSTPL4`). The room is about to change anyway; there is no point painting into it.

The differences between the three states:

- `ST_PL1` (2) calls `PLRSTEP`, while `ST_PL2` (3) and `ST_PL3` (4) call `PLRTICK`;
- `ST_PL2`, before doing anything else, computes `RNDSEED = OB_X + OB_Y + FRAMEC` (61E9h...61F3h) — a sum later read by the `BEH_CLMB` enemy behaviour. A cheap source of pseudo- randomness: the value depends on the player’s position and the frame number, so it is different every frame and never repeats a pattern the player could learn;
- `ST_PL3` watches `LEVEL` and, when a level is finished, raises the difficulty and clears `SEENMAP` completely (6317h...6325h).

5.7. The Menu Does Not Take Part in the Loop

`ST_TITLE` and `ST_MENU` are built unlike all the others: they **do not return control** until a key is pressed.

```

LSTTIT2:
    call    KEYPLL           ; 648D poll the input device
    jae     LSTTIT2          ; 6490 CF=0 -- nothing pressed
    cmp     al, KEY_ENTER    ; 6492 Enter -- start the game
    jne     LSTTIT2          ; 6494 some other key -- keep waiting

```

While the player is looking at the title screen, the main loop is standing inside state eight. `FRAMEC` does not advance and `FRMWAIT` does not run. The notion of a “frame” simply has no meaning in the menu, and nothing in the design requires it to.

The same construction turns up once more, in `DRAWFRM` (62A9h): after a full screen redraw the game **drains the keyboard buffer dry**:

```

LDRAWF3:
    xor     al, al           ; 62A9 AL takes no part here, cleared along the way
    mov     ah, 1            ; 62AB AH=1: is a key waiting in the buffer
    int     16h              ; 62AD keyboard BIOS
    je      LDRAWF4          ; 62AF the buffer is empty
    xor     ax, ax            ; 62B1 AH=0: take the key out of the buffer
    int     16h              ; 62B3 keyboard BIOS
    jmp     short LDRAWF3     ; 62B5 keep draining until the buffer is dry

```

The reason is clear enough: a full redraw takes four times as long as an ordinary frame ([Chapter 7](#)), and in that tenth-and-a-half of a second a player who was leaning on the direction key has stuffed a dozen presses into the buffer. Without the drain they would all pour into the new room at once, and Joe would sail straight into the first skull he met.

So the buffer is emptied and the room starts from a clean slate. The hazard was known and dealt with exactly where it does the most damage; [Chapter 19](#) looks at the other place the same line would have earned its keep, and at what the input path looks like as a whole.

6. The Frame Budget: Why an Empty Room Is Not Faster

6.1. The Problem

Any game must run at a constant speed, or it cannot be played: a jump you learned in an empty room falls short in a crowded one. On every decent machine there is a frame signal for exactly this — wait for the vertical retrace and you get sixty frames a second, free.

The IBM PC does not offer a program that signal. After the rewiring the timer ticks a hundred times a second, and there is nothing to wait for there either: the game has no reason at all to fit neatly into a whole number of ticks. So you work with what is left.

FRMWAIT (79CEh) solves the problem like this: **count how much work this frame contained, and pad the remainder out to a constant with an empty loop**. Not measure the time — count the work. There is no clock, but the cost in cycles of every one of your own instructions is known to the last cycle. Turning “I cannot measure time” into “I do not need to” is the kind of move that makes this file a pleasure to read.

6.2. Estimating the Work Done

The work is estimated from two numbers known in advance; the whole construction rests on the cost of your own code being predictable.

The first is **SOLCNT (79CAh)**, the total length of the support list. The list is built by **MKSOLD (36D5h)** while the room is being constructed ([Chapter 8](#)), and for every floor span it finds it does:

```
LMKSOLS:
    mov     ax, [di]           ; 3733 AX -- length of the run
    clc                     ; 3735 clear the carry ahead of the ADC
    adc     SOLCNT, ax         ; 3736 add it into the total span length
    inc     SOLCNT             ; 373A the length was counted from zero
```

So **SOLCNT** = the sum of the span lengths plus their count. That is the measure of how many tiles will have to be redrawn through **PUTRCT**.

The second is the weight of each living object. **FRMWAIT** walks bits 1...7 of **INBITS** (bit zero is the player and does not count), looks up each living object's type, and takes its weight from **COSTTAB (79C0h)**:

```
COSTTAB DW 1Ch,25h,25h,25h,0 ; 79C0h
```

Type 0 costs **1Ch** = 28 units, types 1, 2 and 3 cost **25h** = 37 each. The fifth entry, zero, is never reached: the behaviour dispatcher serves only types 0...3.

Plus one correction for one specific room:

```
    cmp     ROOMNO, 14h       ; 79DB room 14h = 20
    jne     LFRMWA1           ; 79E0 some other room -- no correction to make
    add     WORKSUM, -44h      ; 79E2 room 20 comes out 44h units cheaper
```

Room **14h** holds the game's record for floor length: its support list comes to 158 units. An average room manages sixteen. Only two others clear ninety at all — **42h** with a hundred and five, and **04h** with ninety-nine.

To see why such a room is a hazard you have to forget the code for a minute and remember the central fact: the game has no clock. It does not measure time. It works **to a price list**.

Think of it as a shift of fixed length. Each frame is allotted a constant sum of nominal units, the same for all ninety-nine rooms. The game looks at how much floor the room has and how many enemies are alive, bills itself for that at the standard rate — and spends exactly the remainder turning an empty loop. Billed itself little, sits a long time. Billed itself a lot, sits a short one. The total is constant, and so is the frame.

The whole thing rests on one condition: the price list must be honest. One unit charged for a floor tile must cost exactly as much as one turn of the idle loop. As long as that holds, the scheme is flawless — and that is precisely why the game does not run away from you in an empty room.

And if something is billed at more than it cost, the game writes off time it never spent. It has less waiting left to do, the frame ends early — and **that room runs faster than the rest**. The longer its floor, the more noticeable. Not “a heavy room drags” but the opposite: a heavy room speeds up, because it wrote itself too generous an invoice for its own floor.

Room 14h has ten times the usual amount of floor, and the gap became visible to the eye. It is closed with twelve bytes: for that room, and that room only, sixty-eight units come off the bill. Smaller bill, larger shortfall, longer idle loop, and the frame returns to the common length.

The sign is worth checking twice, because the sign is the whole answer. WORKSUM is **decreased**, which means the wait gets longer, which means the room is being held back. Held back because it was running ahead.

The choice was between recomputing the price of a nominal unit for all ninety-nine rooms, or taking sixty-eight off in one place and testing one room. They took the second, no other room was affected, and it cost twelve bytes. On a shipping schedule that is not a patch, it is judgement.

6.3. The Padding

```
mov    ax, 10Eh          ; 7A0E 10Eh -- a frame's full cost
sub    ax, WORKSUM       ; 7A11 how much of the budget went unspent
jle    LFRMWA4           ; 7A15 frame ran long on its own
call   DELAY             ; 7A17 burn off the shortfall in an idle loop
```

10Eh = 270, and that is the entire frame budget: that many nominal units of work the game is prepared to tolerate. Less, and the difference goes into DELAY; more, and the frame stretches, and the game slows down at that moment, because there is no longer any time to take away from anything.

DELAY (7A22h) is a doubled empty loop:

```
DELAY:
mov     si, ax            ; 7A22 SI -- how many times to repeat
LDELAY1:
mov     di, 28h           ; 7A24 28h = 40 turns of the inner loop
LDELAY2:
dec     di                ; 7A27 one turn
jne     LDELAY2           ; 7A28 turns not used up yet
dec     si                ; 7A2A one repeat done
jne     LDELAY1           ; 7A2B repeats not used up yet
ret                                ; 7A2D return: the time has been spent
```

AX outer turns, forty (28h) inner ones each. Four instructions, two of which subtract one from a register — and most of this game’s processor time goes into them.

6.4. How Long Is That in Milliseconds

Working from the 8088 timing table:

	cycles
dec di	2
jne taken	16
one inner turn	18
40 turns (39 taken, one not)	708
mov di,28h + dec si + jne	22

	cycles
one outer turn	≈ 730

At 4.77 MHz one turn is about **153 microseconds**. The full budget:

270 × 153 μs ≈ 41 ms ≈ 24 frames per second

Twenty-four frames a second, measured out by subtracting one from a register. No timer, no raster, no counter — just DEC DI, JNE and exact knowledge that a taken JNE costs sixteen cycles. There is something quietly magnificent about a game whose heartbeat is a decrement.

For comparison: the timer installed by SNDINI ticks a hundred times a second, once every 10 ms. A frame comes to roughly four ticks. The code draws no direct connection between the two numbers — sound and picture are clocked independently — but they are of the same order.

6.5. How Much Really Goes into the Idle Loop

Sixty frames of the attract scene:

frame	SOLCNT	WORKSUM	spent in DELAY
1...3	0	0	270
4...59	19	19	251

So on the attract screen the game spends **93% of the frame subtracting one from DI**. Everything else — building the scene, moving the figure, animation, drawing — fits into the remaining seven per cent.

First conclusion: the margin is generous. The budget was set for the worst case — a room with a long floor and four enemies — and an ordinary room comes nowhere near it. That was the right way to set it: a game that stutters occasionally is worse than a game that always runs slightly below its ceiling. Ninety-three per cent headroom is not waste here, it is insurance, and it is why the game feels smooth.

Second conclusion, and it is the interesting one. The whole regulation is done by counting cycles rather than by looking at a clock, which ties the game’s speed to the processor’s frequency: on a machine twice as fast, the empty loop counts out its 270 units twice as quickly, and Panama Joe runs twice as fast along with it.

On an eight-megahertz AT the game runs noticeably faster than intended — not a defect but a direct consequence of the scheme, and one shared by essentially every action game of the period. A few years later the whole library would run into it, and the industry’s answer was to put a Turbo button on the front of the case, which let you **slow the machine down** to the speed the software was written for. DELAY (7A22h) is one of the loops that button was invented for, which is a fairly distinguished thing for four instructions to be.

Part III. The Picture

7. Tiles: Format, CGA Interleave, PUTMAP

7.1. How CGA Video Memory Is Arranged

Mode 4 is 320×200 pixels, four colours, two bits per pixel. One byte of video memory holds four pixels, one screen row is 80 bytes.

The defining feature of CGA: **the rows are not consecutive**. Memory is split into two fields:

B800:0000	rows 0, 2, 4, ... 198	(100 rows × 80 bytes = 8000 bytes)
B800:2000	rows 1, 3, 5, ... 199	(another 8000 bytes)

It was cheaper for the adapter’s designer that way: a television paints a frame in two passes, every other line, and the memory was laid out to follow the raster. Everybody writing for CGA has to live with it: two halves, 2000h between them, and not one picture in memory that sits in a single piece.

The address of row *y*:

B800h : (y and 1) * 2000h + (y shr 1) * 50h

7.2. The Tile Format

A tile is an 8×8 square, which is 2 bytes wide and 8 rows tall: **exactly 16 bytes**. The **TILES** table (15B5h) holds 235 tiles in a row, and the address of tile *n* is 15B5h + *n* × 16.

Inside a tile the rows are stored **in natural order**, top to bottom: word 0 is the top row, word 7 the bottom. There is no interleave in the format itself; the output routine does the interleaving.

This is the right decision and it is worth understanding why. A tile in memory stays a picture: you can draw it on graph paper, type it in as hex digits, and see with your own eyes what you got. All the fuss about fields is locked inside one routine. Lay the tiles out “the way video memory wants them, for speed” and they stop being drawable — and a tile you cannot draw is a tile you cannot fix at four in the morning.

7.3. PUTMAP — the One and Only Tile Output

PUTMAP (1000h) copies a rectangular window of the screen map into video memory — and that is **all** the graphics in this game apart from sprites. Every brick, every ladder, every digit of the score, the **GAME OVER** banner and the skulls on the status panel go through these hundred and forty-four bytes.

The routine is written as an **INT 67h interrupt handler**; the vector is set in **START** (042Ah).

Arguments come in on the stack, six words, and **PUTMAP** reads them straight out of the caller’s stack:

PUTMAP:		
sti	; 1000	interrupts back on: this is a long piece of output
SAVEAL8	; 1001	macro: save eight registers
mov ax, VRAM_SEG	; 1009	B800h -- CGA video RAM
mov es, ax	; 100C	ES = video RAM
mov di, sp	; 100E	DI -- at the arguments pushed on the stack ahead of the INT

The stack layout at this moment is the one thing you need in order to read the whole routine:

offset from DI	contents
----------------	----------

offset from DI	contents
+00h...+0Fh	eight registers saved by <code>SAVEAL8</code>
+10h	IP, pushed by <code>INT</code>
+12h	CS
+14h	flags
+16h	argument <code>ncols</code> — window width in tiles
+18h	<code>nrows</code> — window height in tiles
+1Ah	<code>col</code> — window column
+1Ch	<code>row</code> — window row
+1Eh	<code>tiles</code> — address of the tile table
+20h	<code>map</code> — address of the map

The order is the reverse of the `PUSH` order — the last argument pushed lies nearest. Six `PUSHes` before `INT 67h`, `ADD SP, 12` after: the caller removes them, because `IRET` cannot.

This has a side effect worth naming. The arguments are read at fixed offsets from `SP`, and the three words pushed by `INT` (`IP`, `CS`, flags) are part of that layout. You can no longer turn the call into a `CALL` without re-writing all six offsets at once. The interrupt, taken on for the sake of an independent interface, has become part of the calling convention.

In exchange, `PUTMAP` is a finished service: its address is not baked into any calling routine but lives in the interrupt vector table, and it can be reached from anywhere by code that knows nothing about it. The game still had six more machines ahead of it, and that decoupling was worth its cycles.

7.4. Two Starting Addresses

Next `PUTMAP` computes two pointers. The first is into the map:

```

mov    si, ss:[di+20h]      ; 1014 SI -- address of the tile map
mov    ax, ss:[di+1Ch]      ; 1018 AX -- window's top row on the map
mov    bx, MAP_W            ; 101C map width in cells
mul    bx                   ; 101F row * 40 = offset to the start of the row
add    si, ax               ; 1021 SI = map + row*40
add    si, ss:[di+1Ah]      ; 1023 plus the column: SI at the window's first cell

```

That is, $SI = \text{map} + \text{row} \times 40 + \text{col}$. The second is into video memory:

```

mov    ax, ss:[di+1Ch]      ; 1027 AX -- the same window row, this time counted in bytes
mov    bx, TILE_ROW         ; 102B a tile row is 8 scan lines
mul    bx                   ; 102E row * 140h
mov    bx, ss:[di+1Ah]      ; 1030 BX -- window's left column
shl    bx, 1                ; 1034 a tile takes 2 bytes of video RAM across
add    ax, bx               ; 1036 AX = row*140h + column*2
mov    bp, ax               ; 1038 BP -- offset into video RAM

```

$BP = \text{row} \times 140h + \text{col} \times 2$. The multiplier $140h = 320$ is how many bytes a row of tiles occupies **within one field**: a tile eight screen rows tall contributes four rows to each field, and $4 \times 80 = 320$. A column contributes two bytes per tile.

7.5. The Core: Eight Rows in Four Turns

```
    mov     cx, 4                ; 1053 the tile's 8 lines go in pairs: 4 passes
    push    bp                  ; 1056 BP will be wanted again for the next tile
LPUTMA3:
    mov     ax, [bx]             ; 1057 two bytes of the tile's even line
    mov     es:[bp], ax          ; 1059 out to the even field
    inc     bx                   ; 105D first byte of the pair is done
    inc     bx                   ; 105E BX on to the next two bytes
    mov     ax, [bx]             ; 105F two bytes of the tile's odd line
    mov     es:[bp+FIELD_ODD], ax ; 1061 out to the odd field: +2000h
    inc     bx                   ; 1066 first byte of the pair is done
    inc     bx                   ; 1067 BX on to the next two bytes
    add     bp, BYTES_ROW        ; 1068 down one scan line
    loop    LPUTMA3              ; 106B one more pair of tile lines
```

That is the whole of the interleave. The tile's words are read consecutively and written alternately: the even one at `BP`, the odd one at `BP + 2000h`. After each pair `BP` advances by 80 bytes — one row **inside a field**, which on screen means two rows. Four turns, eight rows of tile.

Not one test, not one conditional branch, not one division by two. The television heritage is handled by a single constant offset.

And `2000h` is baked directly into the instruction `MOV ES:[BP+2000h], AX`. Not a variable, not a table constant, not a register — displacement addressing, for which the 8088 charges nothing extra. The hardest thing about CGA turns out to cost zero cycles, if you arrange the loop so that it does.

7.6. What It Costs

Forty-five frames of play:

window	how many times	instructions per call
25 × 40 (whole screen)	2	56285
3 × 13	4	—
1 × 9 and 1 × 8	41 each	—

A full screen redraw is a thousand tiles and **56,285 executed instructions**, about 56 per tile. In 8088 cycles that is, roughly, eight hundred thousand — **more than 150 milliseconds**, four times the cost of an entire frame ([Chapter 6](#)).

Put another way: showing the whole screen costs as much as playing four frames. Hence the keyboard buffer being drained immediately after a redraw (`DRAWFRM`, `62A9h`): in those hundred and fifty milliseconds a player has time to stack up keypresses, and without the drain they would all arrive at once.

Hence, too, the entire content of the next chapter: you cannot draw the whole screen, except in the rare cases where there is no way around it.

8. The Screen Map `SCRMAP` and Dirty Rectangles

8.1. An Intermediate Layer

If a full screen costs four frames, then the full screen cannot be drawn. Which means you have to know exactly what has gone stale since last time. And for that you need somewhere that records what the screen is supposed to look like.

That place is **SCRMAP** (**11CDh**) — a map of $40 \times 25 = 1000$ bytes, one tile number per cell. The top five rows are the status panel, the other twenty are the room. The game **never writes to video memory directly** except for sprites: first the map changes, then the map is laid out on the screen.

A kilobyte of memory is not cheap, a thirtieth of the entire file. But one map covers three jobs at once.

First: a room can be built by bytecode without any thought about pixels ([Chapter 9](#)). Second: the physics finds out what is under the player and around him by reading tile numbers rather than by looking at dots on the screen (**SCANTIL**) — and from that, incidentally, the whole chapter on darkness grows. Third: partial re-drawing becomes possible, which is what the whole thing was for.

Three uses for one kilobyte is a good exchange rate, and it is the reason the rest of this part of the book has anything interesting in it.

8.2. Full Redraw: **DRWALL**

DRWALL (**0DE8h**) is simply a call to **PUTMAP** covering the whole field:

```
DRWALL(SCRMAP, TILES, row=0, col=0, nrow=19h, ncol=28h)
```

It is called from **DRAWFRM** — on entering a room, on a lighting change, on the player respawning. It costs four frames, so it is never used in an ordinary frame.

8.3. Partial Redraw: **PUTRCT**

PUTRCT (**36A0h**) receives a **list of rectangles** in **DI** and lays each one out with a separate **PUTMAP** call:

```
PUTRCT:
    SAVEALL                ; 36A0 macro
    mov     si, OFFSET SCRMAP ; 36A7 SI -- the screen map
    mov     bp, OFFSET TILES  ; 36AA BP -- base of the tile table
LPUTRCT1:
    cmp     WORD PTR [di], 0FFh ; 36AD 00FFh -- end of the list
    je      LPUTRCT2            ; 36B1 list is used up
    push    si                  ; 36B3 argument: address of the map
    push    bp                  ; 36B4 argument: base of the tile table
    push    WORD PTR [di]       ; 36B5 argument: top row of the window
    push    WORD PTR [di+2]     ; 36B7 argument: left column of the window
    push    WORD PTR [di+4]     ; 36BA argument: how many tile rows
    push    WORD PTR [di+6]     ; 36BD argument: how many tiles across
    int     67h                ; 36C0 OUR OWN API: put out the map
    pop     ax                  ; 36C2 word 1 of six
    pop     ax                  ; 36C3 word 2 of six
    pop     ax                  ; 36C4 word 3 of six
    pop     ax                  ; 36C5 word 4 of six
    pop     ax                  ; 36C6 word 5 of six
    pop     ax                  ; 36C7 all six argument words pulled off the stack
    add     di, 8               ; 36C8 a list entry is eight bytes
```

An entry is **eight bytes**: row, column, height, width, all as words. The list ends with an entry whose first word equals **00FFh**, that is, row 255. Note that it is **00FFh** and not **0FFFFh** — the comparison at **36ADh** is against the word **0FFh**, and **MKSOLD** writes the same value (**3708h**). A one-byte difference, but a real one: **0FFFFh** in that position would not stop the walk.

8.4. Where the Rectangles Come From

There are two lists.

SOLIDS — the “supports”, meaning everything the player walks on and everything his sprite paints over. The list is built by **MKSOLD** (**36D5h**), once, when the room is constructed — because the floor is not going anywhere during the session, and nobody signed up to walk a thousand cells every frame.

It runs over **SCRMAP** from row 5 to row 24, looks for runs of consecutive tiles with code **70h** or higher, and gives each run an entry of height 1 and width equal to the run’s length:

```

LMKSOL3:
    mov     [di], dx           ; 370F entry field +0: row
    inc     di                 ; 3711 step 1 of two: fields run a word apiece
    inc     di                 ; 3712 DI at field +2
    mov     [di], cx           ; 3713 entry field +2: column
    inc     di                 ; 3715 step 1 of two: fields run a word apiece
    inc     di                 ; 3716 DI at field +4
    mov     WORD PTR [di], 1    ; 3717 entry field +4: height -- one cell
    inc     di                 ; 3718 step 1 of two: fields run a word apiece
    inc     di                 ; 371C DI at field +6
    mov     WORD PTR [di], 1    ; 371D entry field +6: length -- one cell
LMKSOL4:
    inc     bx                 ; 3721 one cell right
    inc     cx                 ; 3722 one column right
    cmp     cx, MAP_W          ; 3723 columns used up?
    jge     LMK SOL5           ; 3726 row ended -- the run is cut short
    mov     ax, T_SOLID         ; 3728 lower bound of the solid tile codes
    cmp     [bx+si], al         ; 372B is this cell solid?
    jnl     LMK SOL5           ; 372D no -- the run ends here
    inc     WORD PTR [di]       ; 372F run grows by one cell

```

No `[di+2]`, `[di+4]` or `[di+6]` anywhere: **DI** crawls through the entry in pairs of **INC**, and by the time the run starts growing it is already sitting on the width field. An instruction without a displacement is a byte shorter, the loop is short, and keeping track of where **DI** is inside it is not hard. This is what tight assembly looks like when someone is enjoying themselves.

Tiles **70h** and above are floor, conveyor belts, ladders — everything the player walks on and the sprite paints over as it moves. Every frame they are restored on top of the erased background.

Along the way **MKSOLD** accumulates **SOLCNT**, the sum of the run lengths plus their count. That is the work estimate from which [Chapter 6](#) computes the frame budget.

RCTREC (**6C99h**) is a single “something changed here” rectangle. The four words of the entry and a fifth, **0FFh**, sit consecutively in memory:

```

RCTREC DW 0 ; 6C99h row
RCTCOL DW 0 ; 6C9Bh column
RCTH DW 0 ; 6C9Dh height
RCTW DW 0,0FFh ; 6C9Fh width, and the terminator right behind it

```

So it is a ready-made one-entry list, and **RCTSHOW** (**73F8h**) simply hands its address to **PUTRCT**. The terminator is never written: it has been sitting in the image from the start, immediately after the width field, and all you have to do is not touch it. Two bytes of declaration doing the work of an instruction in every caller.

The entry is filled in by **OFS2RC** (**743Fh**), which converts an offset in **SCRMAP** into a row and a column with a single division:

```

OFS2RC:
    mov     ax, ELMOFS         ; 743F AX -- the offset within the map
    mov     bl, MAP_W          ; 7442 the divisor: columns in one map row
    div     bl                 ; 7444 AL -- the row, AH -- the column
    xor     bh, bh             ; 7446 BH=0
    mov     bl, ah             ; 7448 BL -- the column
    xor     ah, ah             ; 744A AH=0: the whole row now sits in AX
    mov     RCTREC, ax         ; 744C entry field +0: row
    mov     RCTCOL, bx         ; 744F entry field +2: column

```

Its users are picking up an item and opening a door — events after which a small piece of the room changes.

States **ST_PL2** and **ST_PL3** check the entry and, if it is not empty, draw it and **reset** it:

```

    mov     ax, RCTREC         ; 622F the dirty rectangles built up this frame
    cmp     ax, 0              ; 6232 is there anything to show
    je      LSTPL5             ; 6235 nothing at all
    call    RCTSHOW            ; 6237 put out every rectangle that was built up
    mov     RCTREC, 0          ; 623A the list is emptied again

```

The “empty” marker is row 0. Row 0 of **SCRMAP** belongs to the status panel and never appears as an ordinary rectangle, so the value was free to press into service as a flag.

8.5. How Much Actually Gets Redrawn

The same forty-five frames:

- two whole-screen calls — $25 \times 40 = 1000$ tiles each;
- eighty-two calls for 1×9 and 1×8 strips;
- four calls of 3×13 .

In an ordinary frame **seventeen tiles are redrawn instead of a thousand** — 1.7% of the screen. At 56 instructions per tile that is about a thousand instructions against fifty-six thousand.

There is no exotic graphics technique here at all. There is the understanding that only what changed needs drawing, and a kilobyte of memory spent on knowing it. The multiplier is nearly sixty — which is exactly the difference between a playable game and a slide show on a 4.77 MHz machine. A kilobyte for a sixtyfold speedup is one of the best trades in the file, and it was made in 1984 by somebody who had no profiler, no frame graph and no second chance.

9. Rooms as Bytecode: Eight Primitives

9.1. There Are No Rooms in the File

Let us price out a pyramid stored honestly. A room map is 20 rows of 40 cells, eight hundred bytes. Ninety-nine rooms come to **seventy-nine thousand bytes**. The whole file is thirty-two thousand, and besides the rooms it also has to contain the entire game.

The rooms occupy **2715h...346Fh**, about three and a half kilobytes, which is **thirty-five bytes per room on average**. Not eight hundred: thirty-five. A factor of twenty, and no compression involved anywhere.

The trick is simple and it is beautiful: **there are no rooms in the file**. In place of a map there is a program that draws that map. **BLDMAP (0819h)** is an interpreter for a little language invented on the spot, and a room description is bytecode made of three- and four-byte instructions. The game does not store the picture; it stores the instructions for drawing it, and runs them again every time the player walks through a door.

Which program belongs to which room is answered by **ROOMPT (264Dh)**, a hundred pointers. There is no length field: the description of room **N** runs from **ROOMPT[N]** to **ROOMPT[N+1]**, and **CURSIN (09D0h)** reads both pointers back to back:

shl	bl, 1	; 09D7	pointers run two bytes apiece
mov	si, OFFSET ROOMPT	; 09D9	ROOMPT -- 100 pointers
mov	ax, [bx+si]	; 09DC	start of the description of room N
mov	RD PTR, ax	; 09DE	BLDMAP will start reading from here
inc	bx	; 09E1	BX on to the next pointer
inc	bx	; 09E2	the next pointer is the start of room N+1
mov	ax, [bx+si]	; 09E3	and that is the end of description N
mov	END PTR, ax	; 09E5	BLDMAP will read up to here

A hundred pointers for ninety-nine rooms: the last one is there as a boundary.

9.2. The Instruction Format

Each instruction is four bytes, all four of which the interpreter reads, and the pointer advances by however much the handler says:

+0	OP	what to do and where
+1	X	low byte of the offset in the map
+2	P1	first parameter
+3	P2	second parameter

The opcode byte (**CMDOP**) is loaded to the limit — eight bits, not one of them wasted:

bits	meaning
7...6	map page : high byte of the offset in SCRMAP
5...0	command code, also the index into ELMSEL — what to draw with
4...2	for codes 00h...1Fh , the primitive number , an index into PRIMTAB
4	additionally read by PR_COL twice, at 0BE2h and 0C12h : first to double the length of the first column, then to set the step height
0	packed parameters (PR_RECT , 0B76h) or shift the cursor by a cell (OPODD , 0C35h)

The same bits do double duty, and there is a good reason: a map of a thousand cells does not fit in a byte, and multiplying by forty on an 8088 is expensive ([Chapter 1](#)). So **CURSP05** (**0D3Fh**) assembles the offset out of two halves without a single multiply:

```

mov    di, bp          ; 0D41 DI -- the map base
mov    ah, CMDOP        ; 0D43 AH -- the opcode
mov    cl, 6            ; 0D47 shift by six places
shr    ah, cl           ; 0D49 bits 7..6 of the opcode -- high byte of the offset
mov    al, CMDX         ; 0D4B AL -- low byte of the offset
add    di, ax           ; 0D4E DI -- the map cell

```

Ten bits for a thousand cells — exactly as many as are needed. The eight low ones live in a byte of their own, the two high ones in the opcode, where there was room going spare anyway. No multiply, no divide, no table: a shift by six and an add.

What to draw with is decided by **ELMSET** (**0D53h**): it takes **OP** and **3Fh**, looks into **ELMSEL** (**2610h**) and puts the result into **ELMCUR**, which is an offset into the **ELMTAB** strip ([Chapter 14](#)).

9.3. The Interpreter Loop

```

BLDMAP:
mov     ax, RDPTR        ; 0819 RDPTR -- where we are reading the byte code
cmp     ax, ENDPTR       ; 081C have we reached the end of the description?
je      LBLDMA1          ; 0820 end of the description
jmp     LBLDMA6          ; 0822 no -- go and sort out the next command

```

Then four bytes are read into **CMDOP**, **CMDX**, **CMDP1**, **CMDP2**, **ELMSET** is called, and the code branches:

```

and     al, 3Fh          ; 087F the command code is the low six bits
cmp     ax, 3Dh          ; 0881 3Dh -- the difficulty level test
je      LBLDMA11         ; 0884 the difficulty level test
cmp     al, 20h          ; 0886 codes below 20h are drawing primitives
jb      LBLDMA9          ; 0888 a drawing primitive

```

So the codes fall into six classes:

code	what it is
00h...1Fh	drawing primitive
20h...27h	item: PUTITEM
28h...2Fh	doorway or rope: PUTROPE
30h...3Ch	enemy: PUTOBJ3 , writes nothing to the map
3Dh	difficulty level test
3Eh, 3Fh	fragment insertion

The handler returns the instruction length in **AL**, and **BLDMAP** advances the pointer by exactly that:

```

LBLDMA4:
clc     ; 084A clear the carry ahead of the ADC
xor     ah, ah      ; 084B AH=0: AX holds nothing but the command length
adc     RDPTR, ax    ; 084D read pointer forward by the length of the command
jmp     short BLDMA ; 0851 go and sort out the next command

```

The mechanics of that return — through the **CMDLEN** cell rather than a register — are covered in [Chapter 4](#).

9.4. The Eight Primitives

The primitive number is $(OP \text{ shr } 2)$ and 7, the address comes from **PRIMTAB** (**0B5Fh**), and the jump goes through **PRIMJMP**:

```
LBDMA9:
    mov     bx, OFFSET PRIMTAB      ; 08B9 table of the eight drawing primitives
    xor     ah, ah                  ; 08BC AH=0: the whole command code now sits in AX
    mov     al, CMDOP               ; 08BE AL -- the opcode
    shr     al, 1                   ; 08C1 opcode halved
    shr     al, 1                   ; 08C3 opcode divided by four
    and     al, 7                   ; 08C5 primitive number -- bits 4..2 of the opcode
    call    PRIMJMP                ; 08C7 jump off to PRIMTAB[AL]
```

#	address	name	what it draws	times used
0	0B6Fh	PR_RECT	rectangle of a single tile	724
1	0BD3h	PR_COL	column continuing downward	91
2	0C3Dh	PR_DIAG	diagonal, stepped staircase	115
3	0C87h	PR_BAND	band with distinct end caps	154
4	0B6Fh	PR_RECT	same as 0	0
5	0BD3h	PR_COL	same as 1	0
6	0C70h	PR_DIAG2	two diagonals in a row	157
7	0CC0h	PR_PATT	band with a repeating pattern	626

The last column is how many times each primitive fires if you build all ninety-nine rooms. Six of the eight are in use: **PR_RECT** and **PR_PATT** draw nearly everything, while variants 4 and 5 never come up — the table duplicates entries 0 and 1 on purpose, because for those codes opcode bit 4 selects not a different handler but a modifier inside the same one. Two table entries buy a whole extra bit of meaning in the opcode.

Entries 6 and 7, by contrast, lead to separate routines. **PR_DIAG2** (**0C70h**) is **PR_DIAG** called twice: first an ordinary diagonal, then **ELMCUR** shifts by the pattern length and a second diagonal one cell wide follows. That is how ladders are drawn — two columns side by side, the left rail and the right — and they cost one bytecode instruction instead of two.

9.5. What a Primitive Looks Like Inside

The simplest one, **PR_RECT**:

```
PR_RECT:
    call    ELMTILE                ; 0B6F DI -- the map cell, AL -- first tile of the element
    mov     bl, CMDOP              ; 0B72 BL -- the opcode
    and     bl, 1                  ; 0B76 bit 0 of the opcode -- are the parameters packed
    jne     LPRREC1                ; 0B79 bit set -- the parameters lie separately
    jmp     PARMSPL                ; 0B7B bit 0 clear -- the parameters are packed into CMDP1
LPRREC1:
    xor     ch, ch                 ; 0B7E CH=0: the whole count now sits in CX
    mov     cl, CMDP1              ; 0B80 CL -- width minus one
    inc     cx                     ; 0B84 CX -- width in cells
    mov     dl, CMDP2              ; 0B85 DL -- height minus one
    mov     cmdlen, 4              ; 0B89 the command takes four bytes
    jmp     FILLRC                 ; 0B8F fill the rectangle in
```

Opcode bit 0 chooses between two ways of stating the dimensions. If it is set, width and height sit in separate bytes and the instruction takes four bytes. If it is clear, control goes to **PARMSPL** (**0B92h**), which unpacks both dimensions out of the **single** byte **CMDP1**:

```

PARMSPL:
    mov     bl, CMDP1           ; 0B92 BL -- the packed parameters
    mov     bh, bl             ; 0B96 BH -- a copy of the same
    shr     bl, 1              ; 0B98 shift 1 of four
    shr     bl, 1              ; 0B9A shift 2 of four
    shr     bl, 1              ; 0B9C shift 3 of four
    shr     bl, 1              ; 0B9E the high four bits of BL come down
    mov     cl, bl             ; 0BA0 CL -- width minus one
    xor     ch, ch             ; 0BA2 CH=0: the whole count now sits in CX
    inc     cx                 ; 0BA4 CX -- width in cells
    and     bh, 0Fh           ; 0BA5 the low four bits of BH
    mov     dl, bh             ; 0BA8 DL -- height minus one
    mov     CMDLEN, 3         ; 0BAA the command takes three bytes

```

That is where the three-byte instructions come from: any rectangle up to 16×16 cells is described by one parameter, saving a byte on every such instruction. Seven hundred-odd **PR_RECT** calls across ninety-nine rooms means hundreds of bytes — a noticeable share of the three and a half kilobytes all the descriptions take. All of it bought with one bit of the opcode.

PR_PATT (0CC0h) works differently: it lays out consecutive bytes of the **ELMTAB** strip starting at **ELMCUR**, and when it reaches the end of the pattern it rewinds **ELMCUR**:

```

LPRPAT2:
    dec     PATLEFT           ; 0CCF one pattern cell fewer to go
    jge     LPRPAT3          ; 0CD3 the pattern is not used up
    mov     al, PATLEN        ; 0CD5 AL -- the full length of the pattern
    mov     PATLEFT, al       ; 0CD8 the remainder is loaded afresh
    xor     ah, ah            ; 0CDB AH=0: the whole length now sits in AX
    sub     ELMCUR, ax        ; 0CDD the ribbon winds back to the head of the pattern

```

The pattern length comes from the element's first byte, so the same primitive draws a plain wall, an alternating brick course and a conveyor belt: it all depends on where in the **ELMTAB** strip the element code points. Six hundred and twenty-six calls across the whole pyramid make this the busiest primitive in the game — and the simplest of the lot. Lay bytes down in order; on reaching the end of the pattern, rewind. It does nothing else, and nothing else is asked of it.

10. Fragments, Mirrors and the Difficulty Level

10.1. A Room Inside a Room

The bytecode language was missing one thing: if thirty rooms of the pyramid share the same wall-with-a-ladder, paying for it thirty times is expensive. So the language got subroutines.

Code **3Eh** or **3Fh** (**0P** and **3Eh** equals **3Eh**) means: insert the description of another room here, whole. It is a real call, with a return, and **BLDMAP** makes it through the **processor's stack**:

```

LBLDMA10:
    push    RDPTR             ; 08CD the caller's reading point -- onto the stack
    push    ENDPTR           ; 08D1 the caller's end of description -- onto the stack
    xor     bh, bh            ; 08D5 BH=0: the whole mirror flag now sits in BX
    mov     bl, MIRFLAG       ; 08D7 BL -- the caller's mirror flag
    push    bx                ; 08DB and that goes on the stack too
    mov     bx, RDPTR         ; 08DC BX -- start of the insert command
    inc     bx                ; 08E0 second byte of the command
    mov     dl, [bx]          ; 08E1 DL -- number of the room being inserted
    mov     al, CMDOP         ; 08E3 AL -- the opcode
    and     al, 1             ; 08E6 low bit of the opcode
    mov     MIRFLAG, al       ; 08E8 3Fh -- insert it mirrored
    inc     CALLDEP           ; 08EB nesting depth one higher
    mov     al, dl            ; 08EF AL -- number of the room being inserted
    call    CURSIN            ; 08F1 set RDPTR and ENDPTR onto its description
    jmp     BLDMAP            ; 08F4 go read the fragment's description

```

The instruction takes two bytes: code and fragment number. The difference between **3Eh** and **3Fh** is the low bit, which is the mirror flag.

The return happens in the same place as an ordinary end of description:

```

LBLDMA1:
    cmp     CALLDEP, 0           ; 0825 nesting depth of fragments
    je      LBLDMA5             ; 082A zero -- the room is finished
    dec     CALLDEP             ; 082C leaving a fragment: depth one lower
    cmp     MIRFLAG, 0          ; 0830 was the fragment inserted mirrored
    je      LBLDMA2             ; 0835 no -- nothing to flip
    call    MIRROR              ; 0837 flip whatever the fragment drew

LBLDMA2:
    pop     ax                  ; 083A off the stack -- the caller's mirror flag
    mov     MIRFLAG, al         ; 083B the caller's mirror flag is back
    pop     ENDPTR              ; 083E off the stack -- the caller's end of description
    pop     RDPTR               ; 0842 off the stack -- the caller's reading point

```

Across all ninety-nine rooms: fragments are used by **78 rooms**, with **130 insertions** in total, and the deepest nesting is **three**. That is, a fragment can insert a fragment that inserts another one. There is no depth check in the code: each level costs six bytes of stack, the 510-byte reserve covers that many times over, and three is what the data actually needed. The person writing the data here is the person writing the code, and there is nobody to defend against.

One important subtlety: while **CALLDEP** is above zero, **objects are not created**.

```

    cmp     CALLDEP, 0           ; 088A are we inside an inserted fragment
    jne     LBLDMA3             ; 088F we are -- a fragment's objects are skipped

```

Codes **20h** and above inside a fragment are simply skipped. A fragment is geometry only; items, doors and enemies are placed by the top-level room.

The rule heads off a real hazard: a fragment containing a wall and a key, inserted into a room three times, would give you three identical keys in three places, and all three would count under one number in the collected map ([Chapter 14](#)). Two instructions and the problem cannot occur — which is the cheapest kind of correctness there is.

10.2. The Mirror

MIRROR (**0924h**) reflects what has been drawn about the vertical axis — all twenty playfield rows, five through twenty-four:

```

LMIRRO1:
    add     si, 27h             ; 093C SI to the end of the buffer: 27h = 39
    mov     cx, MAP_W           ; 093F cells across one map row

LMIRRO2:
    mov     al, [di]            ; 0942 the cell on the left
    mov     [si], al            ; 0944 the same cell, on the right in the buffer
    inc     di                  ; 0946 forward along the map row
    dec     si                  ; 0947 backward along the buffer
    loop    LMIRRO2             ; 0948 the whole row is turned around
    call    MIRSUB              ; 094A swap the tiles for their mirror pairs

```

The row is written back to front into **ROWBUF** (**11A5h**), fixed up there by **MIRSUB**, and a **REP MOVSB** with **STD** set puts it back.

Simply reversing the cells is not enough: some tiles have a left and a right. That is **MIRSUB**'s job (**096Ah**) — it looks a tile up in the **MIRSRC** list (**2465h**) and substitutes its partner from **MIRDST** (**246Fh**):

```

MIRSRC DB 28h, 29h, 2Ah, 2Bh, 72h, 73h, 74h, 75h, 76h, 77h
MIRDST DB 29h, 28h, 2Bh, 2Ah, 77h, 76h, 75h, 74h, 73h, 72h

```

Ten tiles: **28h/29h** and **2Ah/2Bh** swap in pairs, and the six conveyor-belt tiles **72h...77h** reverse wholesale — a belt running right becomes a belt running left. Everything else — brick, the skull on the wall, the ladder — is symmetric in itself: the tiles were drawn with reflection in mind from the beginning. Ten bytes of table and foresight at the drawing board are all it takes to double the room count.

The lookup is a **REPNE SCASB** over ten bytes, and the two commonest tiles are filtered out in advance:

```

    cmp     al, 0               ; 097C 0 -- empty
    je      LMIRSU2             ; 097E nothing to swap here
    cmp     al, 1               ; 0980 1 -- the floor
    je      LMIRSU2             ; 0982 nothing to swap here

```

Empty space and brick are symmetric and make up the lion’s share of any room. Two comparisons save ten on every such cell, and there are eight hundred cells in a room.

10.3. The Difficulty Level Changes the Geometry

Code **3Dh** is the entire branching logic of this language:

```
LBLDMA11:
    mov     al, LEVEL                ; 08F7 AL -- the current difficulty level
    cmp     al, CMDX                ; 08FA match it against the threshold in the command
    jb      LBLDMA12                ; 08FE the level is below the threshold
    jmp     LBLDMA3                 ; 0900 threshold met: skip the command over
LBLDMA12:
    jmp     LBLDMA1                 ; 0903 below the threshold -- cut the description short
```

The design is as frugal as it gets: if **LEVEL** is below the threshold, the interpreter **stops reading the description**, as though it had reached the end. No branch, no jump, no “else” block — just “we read no further”.

From which follows a constraint: everything that appears at higher levels has to sit at the end of a room’s description. An extra beam for the fourth lap gets appended to the very tail, or it would carry the rest of the room away with it.

Instruction **3Dh** appears in **67 rooms**, 85 times in all, with the thresholds distributed like this:

threshold	number of commands
2	6
3	7
4	72

Now it gets interesting. The level chosen in the menu with **F2** is only ever 1, 2 or 3. But **ST_PL3** raises **LEVEL** every time the player completes the pyramid, capping it at nine (**6307h...6312h**). Which means that threshold 4, where **seventy-two of the eighty-five commands** sit, only fires from the second lap onward.

Comparing the built maps confirms it:

levels	rooms changed	cells
1 → 2	4	97
1 → 3	5	67
1 → 4	23	176
1 → 9	23	176

So the difficulty choice in the menu changes the geometry of just four or five rooms out of ninety-nine: pick level three and you get the same pyramid plus five amendments.

The main helping of new obstacles — twenty-three rooms — opens up only to a player who has been all the way round at least once: threshold 4 cannot be reached from the menu, **LEVEL** gets there on its own ([Chapter 17](#)). That is a nicely judged piece of design. The reward for finishing the pyramid is not a number going up; it is a pyramid that has quietly rebuilt itself.

Above the fourth lap there is no new geometry at all — no threshold higher than four appears anywhere in the bytecode. From there the pyramid gets harder by other means: darkness, enemies that do not disappear, and extra lives handed out half as often.

11. Sprites: Frames, Masks, Background Saving

Tiles give you a room that stands still. Everything that moves — Panama Joe, the skulls, the snakes, the fire — is drawn on top of it as sprites.

On machines with sprite hardware, a sprite is a picture that the adapter draws over the background by itself, moves by coordinates by itself and removes by itself. Here it is **DRWSPR** (5D31h), one hundred and ninety-one bytes: for each figure it computes an address in video memory, remembers everything that was underneath, lays the pixels down through a mask table, and puts the background back when the figure moves on.

Eight figures on screen, up to eight such calls a frame, and one wrong byte would leave a hole on screen for the rest of the session. It is the most demanding routine in the game, and it is 191 bytes long.

11.1. Eight Objects and One Bit Mask

There are never more than eight live objects on screen. Which of them are alive is held in **INBITS** (10A9h) — one byte, one bit per object, object 0 always the player. Everything that walks the objects walks them through that byte: **ERSALL** (5CA0h), **DRWALL2** (5CB6h), **ANIMHIT** (6CB8h), **DIRBIT** (3763h).

Six bytes are known about an object — one **OBJTBL** record (06FAh), its length declared as **OBJREC_SZ**:

```
+00 DW  pointer to the bitmap (frame address + 2: header skipped)
+02 DB  X in video memory bytes, 0..79
+03 DB  Y in raster rows; bit 0 -- field number
+04 DB  width in bytes (4 pixels per byte)
+05 DB  height in rows
```

Width in bytes rather than pixels — and behind that small detail hides the whole feel of the game. The output is byte-by-byte; shifting a picture by one pixel would mean shifting all of its bits, that is, adding a shift loop to every byte of every row.

So **Panama Joe walks in four-pixel steps**, one byte of video memory at a time. The motion comes out chunky, and that is exactly how anyone who played this game remembers it. Smoothness would have cost a drawing loop four times as long, and drawing is already the most expensive part of the frame. The chunkiness is not a limitation the game suffered; it is the budget being spent where it shows.

11.2. Where to Write: the Video Memory Address

DRWSPR (5D31h) starts with the same arithmetic as **PUTMAP**:

```
mov    al, [bx+0B_Y]          ; 5D5A  the object's Y
and     al, 0FEh              ; 5D5D  even line: bit 0 picks the field
mov     cl, FLD_ROW           ; 5D5F  bytes to a field line
mul     cl                    ; 5D61  line * 40
mov     cl, [bx+0B_X]          ; 5D63  plus X in bytes
add     ax, cx                ; 5D66  AX = line*40 + X
```

$VRAM = (Y \text{ and } 0FEh) * 28h + X$. The low bit of *Y* does not enter the address — it says which of the two CGA fields to start from, and goes off into a field of its own, **BG_FLD**. Hence the **and al, 0FEh**: even and odd rows live in different halves of video memory, 2000h apart.

11.3. The Saved Background

A sprite is erased not by painting over it but by putting back what was underneath. The place for that is **OBJTAB** (4BA0h), with a record stride of 100h, so the address is computed with a shift by 8:

```
mov     ax, si                ; 5D41  AX -- object number
mov     cl, 8                 ; 5D43  CL=8 -- shift by eight places
shl     ax, cl                ; 5D45  number * 100h: records run 256 bytes apiece
```

A snapshot record is laid out like this (length `BGSAVE_SZ`):

```
+00 DW  offset within B800h
+02 DB  field: 0 or 1
+03 DB  height in rows
+04 DB  width in bytes
+05 ..  the background bytes themselves, row by row
```

The largest frame needs $5 + 6 \times 20 = 125$ bytes. Two hundred and fifty-six are allotted: at that stride the record address is a shift by eight, without a single multiply.

Then comes a reserve that looks inexplicable for a long time. There are **sixteen** records set aside (`4BA0h...5B9Fh`), and eight are addressed: `INBITS` is one byte, eight bits, eight objects, and nobody goes past the seventh. Half the buffer — `53A0h...5B9Fh`, **exactly two kilobytes** — is never read and never written.

Two kilobytes, in a file where bytes were being saved one at a time on packed room parameters. The explanation is not about objects; there were always eight of those. What became surplus was not an object but a **screen**. The PCjr version has two video pages, and there the background snapshots are kept one set per page — eight records for the page being shown, eight for the page being drawn into. Here there is one page, and one of the two sets does the work.

The upper two kilobytes are not headroom for growth. They are the other half of a structure that turned out to have no other half on CGA. Once you know that, they stop being a puzzle and become the clearest surviving evidence of how these two ports were built: side by side, from one body of code. More on this in [Section 11.8](#).

11.4. The Mask: Transparency for the Price of One Table

A CGA pixel is two bits, a byte is four pixels. The frame has no bit mask; in its place stands `MSKTAB` (`5BA0h`), 256 bytes, and the rule behind it reconstructs exactly:

```
MSKTAB[b] = (0F0h if the high nibble of b is zero)
             or (00Fh if the low nibble of b is zero)
```

All 256 bytes of the table obey that rule, without a single exception. The output core is nine instructions per byte:

```
LDRWSP2:
    mov     al, es:[si]           ; 5D8F  read a byte of background
    mov     [di], al             ; 5D92  and keep it for ERSSPR
    SAVEREG <si,cx>              ; 5D94  macro
    xor     ch, ch               ; 5D96  CH=0: the whole bitmap byte now sits in CX
    mov     cl, [bx]             ; 5D98  CL = byte of the bitmap
    mov     si, cx               ; 5D9A  SI -- the same byte as an index into the mask table
    and     al, ds:[bp+si]       ; 5D9C  punch out the background dots under the sprite
    or      al, cl               ; 5D9F  lay the sprite pixels in
    RESTREG <cx,si>              ; 5DA1  macro
    mov     es:[si], al         ; 5DA3  write it out to video RAM
```

`SI` here serves two purposes and therefore goes on the stack: first it is the address of a byte in video memory, then an index into `MSKTAB`, and by the time the result is written it is an address again. There is simply no spare register for the index.

The index into the table is **the bitmap byte itself** — there are no separate masks in the data at all; the picture is its own mask. Where the bitmap has a zero nibble, the `AND` lets the background through; where it is non-zero, the `OR` lays the sprite's pixels down. Two hundred and fifty-six bytes replace a per-pixel mask for every frame in the game, and the arithmetic comes out to two instructions.

Transparency therefore works to nibble precision, that is, to **pairs of pixels**. Hence the stepped edges visible in any screenshot: not the artist's hand but 256 bytes of table standing in for a bit mask and a drawing loop four times as long.

And hence a constraint the artist had to work within: **sprites have no black**. A zero in the picture always means “the background shows through here”, never “this is black”. Joe’s dark eye is drawn in any of the three colours except the background — otherwise you would see brick through his eye. Look closely at the sprites with that in mind and the artwork gets more impressive, not less: every figure is designed around a palette rule that no tool enforced and no comment recorded.

11.5. Interleave: the Loop Body Written Twice

The row loop is split into two halves — **LDRWSP1** for the even field and **LDRWSP3** for the odd — and it is literally the same code, the second time with a **2000h** displacement:

```
LDRWSP2:
    mov     al, es:[si]           ; 5D8F  read a byte of background
    ...
LDRWSP4:
    mov     al, es:[si+FIELD_ODD] ; 5DB3  background byte out of the odd field
```

After both halves comes **add si,50h (5DD8h)** — two raster rows — and round again. The entry point is chosen by **BG_FLD**: if the sprite starts on an odd row, **cmp al,1 / je LDRWSP3 (5D88h)** drops straight into the second half.

The loop had to be unrolled because otherwise every row of every sprite would carry a “which half am I in” test. Eight figures, twenty rows, six bytes — about a thousand tests a frame, in a place where you can do without them entirely.

The price is twice the code, fifty-odd extra bytes. The return is not one conditional branch in the hottest loop in the game. That is the trade this whole book keeps circling back to, and here it is made about as cleanly as it can be.

11.6. The Order Within a Frame: Why It Works

DRWALL2 walks the objects **from seven down to zero**, **ERSALL** **from zero up to seven**:

```

    mov     si, 7                ; 5CB7  SI -- object number, starting at seven
    mov     al, INBITS           ; 5CBA  AL -- one bit per live object
LDRWAL1:
    clc                     ; 5CBD  clear the carry ahead of the RCL
    rcl     al, 1            ; 5CBE  high bit out into the carry
    jae     LDRWAL2          ; 5CC0  bit clear -- no object in this slot
    call    DRWSPR           ; 5CC2  draw the sprite of object SI
LDRWAL2:
    dec     si                ; 5CC5  back to the previous object
    xor     si, si            ; 5CA1  SI -- object number, starting at zero
    mov     al, INBITS        ; 5CA3  AL -- one bit per live object
LERSAL1:
    clc                     ; 5CA6  clear the carry ahead of the RCR
    rcr     al, 1            ; 5CA7  low bit out into the carry
    jae     LERSAL2          ; 5CA9  bit clear -- no object in this slot
    call    ERSPPR           ; 5CAB  erase the sprite of object SI
LERSAL2:
    inc     si                ; 5CAE  on to the next object
```

Even the shift direction differs — **RCL** presents the high bit for testing, **RCR** the low one — so that the object number and its bit stay in step in both directions.

This is not decoration, it is a requirement. **DRWSPR** saves into **OBJTAB** whatever lay under the sprite **at the moment of drawing**, which includes any neighbours already drawn. Object 0’s snapshot contains pieces of objects 1...7. Snapshots like that can only be restored in strictly reverse order — otherwise erasing object 1 would lay background captured before object 0 existed right on top of object 0.

What happens if the order is broken is easy enough to measure: two overlapping sprites, one full “draw all, erase all” cycle.

order	background bytes corrupted
as in the game: draw 1,0 — erase 0,1	0
draw 1,0 — erase 1,0	61
draw 0,1 — erase 0,1	37

It is an ordinary stack, spread across two loops: last drawn, first erased. No pointer array, no list — two loops running in opposite directions and two different shift instructions.

There is a bonus. Since the player is drawn last, he always ends up **on top of** the enemies. No depth sorting, not one comparison — the traversal order did all of it. Getting z-order for free out of a constraint you already had is exactly the kind of thing that makes this code a pleasure to read.

11.7. Erasing

ERSSPR (5CCDh) is half the length: no mask, no bitmap, just a **REP MOVSB** from **OBJTAB** back into video memory, in the same two halves.

One detail matters more than the rest. **ERSSPR** takes the width, height, address and field **from the snapshot record itself**, and never looks into **OBJTBL** at all. Nor could it: by the time of erasing, the object may already be standing somewhere new, and it must be erased where it was. The snapshot remembers its own place — otherwise every moving skull would trail a wake of its own copies behind it.

11.8. Flicker: the One Thing CGA Could Not Offer

Anyone who played this game on a real machine remembers it: the sprites **flicker**. Panama Joe goes see-through, the skulls blink, and the busier the room the more you notice. It is not disk wear and not a quirk of the monitor — it is the output design, and it can be traced by address all the way down.

There is no second screen. Mode 4 occupies 16,000 bytes out of the 16,384 on a CGA board. A second page physically will not fit, so you write straight into what is on display at that instant: **PUTMAP** puts tiles into **B800h** (1009h), **ERSSPR** and **DRWSPR** put sprites there (5CD5h, 5D39h).

There is no frame signal either. Port **3DAh**, which could be polled for the vertical retrace, is never read in this file; the only ports the game touches are **20h**, **21h**, **40h**, **42h**, **43h**, **61h**, **201h** and **3F2h**. Frame length is maintained by **FRMWAIT** (79CEh) with an empty loop ([Chapter 6](#)), and has no connection to the raster at all.

Now the order of work inside a game frame, **ST_PL2** (61E5h):

```

call    ERSALL          ; 6223 erase every sprite, putting the background back
pop     ax              ; 6226 AL -- the live-object bits as they were
mov     INBITS, a1      ; 6227 the bits are back the way they were
mov     DEADBIT, 0      ; 622A the killed-object bit is cleared again
mov     ax, RCTREC      ; 622F the dirty rectangles built up this frame
cmp     ax, 0           ; 6232 is there anything to show
je      LSTPL5          ; 6235 nothing at all
call    RCTSHOW         ; 6237 put out every rectangle that was built up
mov     RCTREC, 0       ; 623A the list is emptied again
LSTPL5:
mov     di, OFFSET SOLIDS ; 6240 DI -- the floor-span list, itself a list of rectangles
call    PUTRCT          ; 6243 redraw the tiles over every floor span
call    PLRHIT          ; 6246 the player against every object: hits and pickups
call    DRWALL2         ; 6249 draw every sprite, saving the background under it

```

Between **ERSALL** (5CA0h) and **DRWALL2** (5CB6h) sits the whole tile redraw. For all that time the sprites are not on the screen — and the CRT beam carries on regardless, uninformed of the program's plans.

Who suffers most follows from the traversal directions. From [Section 11.6](#): erasing runs from object 0 to 7, drawing from 7 to 0. So the player is erased first and drawn last, and his window of invisibility is **ERSALL** plus **PUTRCT** plus the whole of **DRWALL2**.

The game measures the length of that window itself: `COSTTAB (79C0h)` charges twenty-eight units for drawing a type 0 object and thirty-seven for types 1...3, and one unit is one turn of `DELAY (7A22h)`, a hundred and fifty-three microseconds.

situation	absent from screen	CGA field
room with three enemies, <code>SOLCNT = 16</code>	≈ 23.7 ms	16.69 ms
empty room, player alone	≈ 6.7 ms	16.69 ms

A CGA field is 262 lines at 59.92 Hz, that is, 16.69 ms for a complete sweep of the beam. And here is the key point: **in a populated room the gap is longer than an entire field**. A gap longer than a field is certain to cover at least one full pass of the beam over any given screen row — so Panama Joe disappears exactly once per game frame, about twenty-four times a second, and is absent roughly 58% of the time.

Object 7 gets off far more lightly: its gap is only `PUTRCT` plus its own drawing, around seven milliseconds. Hence an observation you can check with your own eyes: **the player flickers most, and the highest-numbered enemy least**.

And since the game frame (41 ms) and the raster field (16.69 ms) are not commensurate, the phase drifts continuously, and the flicker comes out not steady but breathing — stronger, then weaker. Which is exactly how it looks.

11.9. Eight Calls to a Routine That Does Nothing

Everything described above is solved by one technique, known since the earliest graphics machines: draw into an invisible page and display a finished one. The technique was not forgotten here — it was removed, and removed carefully, with every mounting point left in plain sight.

`NOP0 (60C0h)` is a routine consisting of a single `RET`. It is called from eight places:

address	in	what stands next to it
6127h	<code>ST_PL1</code>	between <code>TILRST</code> and <code>ERSALL</code>
61F7h	<code>ST_PL2</code>	between <code>RNDSEED</code> and <code>ANIMTL</code>
6288h	<code>DRAWFRM</code>	immediately after <code>DRWALL2</code>
62B8h	<code>ST_PL3</code>	as the routine's first instruction
637Fh	<code>ST_DRW</code>	first instruction, before <code>ANIMTL</code>
642Ah	<code>ST_X7</code>	between two identical <code>PUTRCTs</code>
6F99h	<code>SHOWSCR</code>	inside a loop that runs twice
7C48h	<code>SCRTTL</code>	under the <code>PAD03CD</code> test (Chapter 4)

Every one of the eight is an end of frame, a start of frame, or the moment when the picture has just been assembled in full. Precisely the places where you would flip a page.

In this game's sibling, `MX16.COM` for the IBM PCjr, those same eight places hold a call to the video page switching routine. The PCjr has no separate video memory: the controller and the processor share the first 128 KB of RAM, register `3DFh` slices it into eight pages of 16 KB and says which one to show. Switching is a single port write, atomic; drawing goes into the page that is not currently visible. A sprite has nowhere to disappear to: it is present in both pages at all times.

Two entries in the table above are particularly eloquent.

The loop that runs twice in `SHOWSCR` (6F8Fh...6F9Dh) is the idiom “draw the same thing into both pages”: draw, flip, draw, flip back. When there was nothing left to flip, it became drawing the same thing twice into one page. The same goes for `ST_X7` (6421h), where the list at 60D3h is laid out twice in a row.

The test in `SCRTTL` (7C40h) reads as “if we drew into the second page, show it”. Cell `PAD03CD` (03CDh) is the drawing page segment, still sitting where it always was after the number of pages became one.

And this is where the extra two kilobytes of `OBJTAB` come from (Section 11.3): the PCjr keeps one set of background snapshots per page, and in the CGA build the second set was left with nothing to do.

There is nothing here to hold against anyone. Page flipping rests on a register CGA does not have, and there is no way around that: two pages of 16,000 bytes will not fit into 16,384 by any means whatsoever. Keeping a second buffer in system RAM and copying it into video memory means a `REP MOVSW` over sixteen thousand bytes, about forty milliseconds — an entire game frame for one copy. The flicker in the CGA build is not an oversight; it is the price of moving the game onto a board with one screen, and the output design, right down to the traversal directions, stayed exactly as it was.

Scrubbing the scaffolding out completely would have been both expensive and unwise. Eight `CALLs` to nowhere cost twenty-four bytes and a couple of cycles a frame; eight edits in live code a week before shipping cost considerably more — and cost more still on the day somebody decides the page has to come back after all. Leaving the mounting points in is the same instinct that leaves the bolt holes in a bracket, and forty years later it is the reason we can read the family resemblance between these two versions at all.

12. Animation: Frame Chains and Living Tiles

There are two animation systems in the game, and they work in opposite directions. For objects, the pointer to the picture changes while the picture itself stays put. For tiles it is the reverse: the tile number in the map never changes, and the picture under that number is redrawn in place, inside `TILES`.

The second approach looks inside out — rewriting the font instead of changing the letter — and it turns out to be incomparably cheaper. A blinking floor may occupy dozens of cells in a room; changing the number in each of them and redrawing each is expensive, while swapping the sixteen bytes of the picture itself costs one `REP MOVSW`, and they all blink at once.

12.1. A Chain Is Just Pointers Lying Next to Each Other

`ANIMTB` (3B80h) is a continuous ribbon of words, each word a frame address. A chain is defined by two numbers per object:

- `ANIMPTR` (6520h, 8 words) — where the chain starts;
- `ANIMIDX` (6530h, 8 words) — which frame the object is on now;
- `ANIMLEN` (6540h, 8 words) — **the highest permitted frame number.**

That last one sounds odd until you look at the advance (6CF9h):

```
inc    WORD PTR [bx+si]      ; 6CFE  frame number one step on
mov     di, [bx+si]          ; 6D00  DI -- the new frame number
mov     bp, OFFSET ANIMLEN   ; 6D02  highest frame number, one slot per object
cmp     di, ds:[bp+si]       ; 6D05  has the chain run out?
jle     LANIMH5              ; 6D08  no -- the frame will do
LANIMH4:
mov     bx, OFFSET ANIMIDX   ; 6D0A  table of frame numbers, one slot per object
mov     WORD PTR [bx+si], 0   ; 6D0D  the chain starts over
```

The position is compared against `ANIMLEN`, not the other way round. On chain 3B80h (walking right: 3BE0h 3C5Ah 3BE0h 3CD4h) it comes out like this:

ANIMLEN	what OB_FRAME yields
0	3BE2 3BE2 3BE2 ... — the frame is frozen
1	3C5C 3BE2 3C5C 3BE2 ... — two of them
3	3C5C 3BE2 3CD6 3BE2 ... — full cycle of four
5	3C5C 3BE2 3CD6 40A6 4120 ... — into the next chain

The last row explains everything: **a chain has no terminator**. No zero at the end, no length field, no marker — only **ANIMLEN**, and set it one too high and the animation walks into the neighbouring chain.

That is the price of letting chains overlap, and the price is worth paying. **3B92h** is the tail of **3B90h** shifted by one word: two different frame sets sharing the same memory. The saving shows up everywhere in this file, and the discipline it demands — get **ANIMLEN** right and the structure is invisible — is exactly the discipline the rest of the listing displays.

The frame is set at **6D27h**:

```

mov    ax, ds:[bp+di]    ; 6D27 AX -- the frame out of the chain
inc    ax                ; 6D2A step 1 of two: the frame header
inc    ax                ; 6D2B step over the frame header
mov    [bx+si], ax       ; 6D2C pointer to the object's bitmap

```

Two **INCS** instead of **ADD AX,2** — a byte shorter and a cycle faster; by 1984 substitutions like that were made without thinking about them.

12.2. The Advance Does Not Live Where You Expect

All of this is done by **ANIMHIT** (**6CB8h**), a routine with two duties at once: walk the live objects, advance each one's animation frame, and while it is there call the collision check **HITCHK** (the call is at **6D34h**) for everyone except object zero. Animation and collisions in one loop: two separate passes over eight objects would cost twice as much, and the two jobs need exactly the same iteration.

Object 0's advance has conditions on it (**6CD0h**):

```

cmp     si, 0            ; 6CD0 object 0 is the player
jne     LANIMH3          ; 6CD3 not the player
xor     ah, ah           ; 6CD5 AH=0: the whole state now sits in AX
mov     al, PLRST        ; 6CD7 AL -- the player's state
cmp     al, 0            ; 6CDA state 0 -- standing still
je      LANIMH4          ; 6CDC standing -- do not change the frame
cmp     ax, 3            ; 6CDE state 3 -- climbing
je      LANIMH2          ; 6CE1 state 3 -- climbing
cmp     ax, 4            ; 6CE3 state 4 -- hanging on a rope
jne     LANIMH3          ; 6CE6 neither one -- the frame does change
LANIMH2:
cmp     OBJVY, 0         ; 6CE8 is the player moving vertically?
jne     LANIMH3          ; 6CED moving -- the frame does change

```

So: a standing player is reset to frame 0 every frame, and one frozen on a ladder (**PLRST** 3 or 4, **OBJVY** = 0) holds whichever frame he was caught on. His feet do not shuffle on the rungs for nothing.

12.3. Which Chains Live in ANIMTB

The answer comes from **PST_0** (**65D3h**) — a branch on **CMDBITS**, and it is laid out symmetrically:

```

    and    ah, DIR_LF      ; 6669 bit 2 of AH -- left
    jne    LPST8          ; 666C left it is
    jmp    LBELTP2        ; 666E not left -- go and look at right
LPST8:
    mov     ax, OFFSET AN_WALK2 ; 6671 frame chain for walking left
    mov     ANIMPTR, ax        ; 6674 the player's frame chain is set
    mov     ANIMLEN, 3         ; 6677 four frames in the chain
    mov     OBJVX, 0FFh        ; 667D FFh = -1: speed to the left
    ...
    and     ah, DIR_RT      ; 6745 bit 3 of AH -- right
    je      LBELTP4         ; 6748 not right
    mov     ax, OFFSET ANIMTB ; 674A frame chain for walking right
    mov     ANIMPTR, ax      ; 674D the player's frame chain is set
    mov     ANIMLEN, 3       ; 6750 four frames in the chain
    mov     OBJVX, 1         ; 6756 speed to the right

```

So 3B88h is **walking left** and 3B80h is walking right. There is no sprite mirroring machinery at all: both sides are drawn separately and sit in the file as two independent sets of pictures.

That they really are mirror images shows in the pixels:

pair	agreement
F_STAND 3BE0h ↔ F_WALKL0 40A4h	exact mirror, offset by 4 pixels
F_WALK1 3C5Ah ↔ F_WALKL1 411Eh	exact mirror, offset by 4 pixels
F_WALK2 3CD4h ↔ F_WALKL2 4198h	mirror, one pixel does not match

The four-pixel offset is that same byte step: the reflected figure is shifted by one byte within a 24-pixel-wide field. The single divergent pixel sits in row 9, on the belt. It is also the proof that the second half of the set was drawn by hand: a machine reflection would have come out exact. Somebody sat down and mirrored thirty-odd frames pixel by pixel, and finished one pixel away from perfect.

The standing poses sort out the same way:

chain	address	what it is	set from
ANIMTB	3B80h	walking right, 4 frames	674Ah
AN_WALK2	3B88h	walking left, 4 frames	6671h, 6903h
AN_STNDL	3B90h	standing, facing left	668Eh, 6337h
AN_STNDR	3B92h	standing, facing right	676Ah, 634Fh
AN_DIE	3B9Ch	death, 6 frames	71ACh
AN_JUMP	3BA8h	jump and fall	65F7h, 6A17h
AN_FALL	3BB0h	climbing down a ladder	6626h, 698Fh, 6A04h

12.4. The “Tail of ANIMTB” Is Not a Tail

The block 3BC0h...3BDFh looks like the tail of the table — a scattering of bytes after the player’s comprehensible chains. Read as words, it gives sixteen frame addresses: 472Ch 46D6h 472Ch 4782h 47D8h 482Eh 4884h 48DAh 4930h 46D6h 49F6h 4A4Ch 4AA2h 4AF8h 4986h 49BEh. These are the enemy animation chains, and they are pointed at by element codes from ELMTAB:

code	“going that way” chain	“coming back” chain
6Ah	AN_OBJ4 (3BDCh)	AN_OBJ1 (3BD2h)
6Ch	AN_OBJ1 (3BD2h)	AN_ENMYL (3BB2h)
6Eh	AN_ENMYL (3BB2h)	AN_ENMYR (3BC2h)

code	“going that way” chain	“coming back” chain
72h	AN_OBJ2 (3BD4h)	AN_OBJ3 (3BD8h)

PUTOBJ3 takes the word at the code (0E4Ah), and the word at code + 2 (0EEAh) goes to whoever is heading the other way. The entries overlap again: one enemy’s return chain is the next enemy’s outbound chain, because **ELMTAB** is a ribbon rather than an array.

Sixteen bytes saved — and the cost is that this data cannot be read by eye at all: it acquires meaning only from the side of the code that walks it. In a thirty-two kilobyte image that is a bargain, and it is also a reminder of why the listing in this project carries an address on every line.

12.5. Living Tiles

The second animation method changes the picture rather than the pointer. **ANIMTL** (3776h) is called once a frame and copies a new appearance of a tile into **TILES**:

```

mov    si, OFFSET BLK70AT      ; 3849 table of the two phase pictures
mov    si, ds:[bp+si]          ; 384C SI -- picture of this phase
mov    di, OFFSET TILE_70     ; 384F DI -- the live tile 70h
mov    cx, TILE_SZ             ; 3852 bytes to a tile
shr    cx, 1                   ; 3855 MOVSW carries two bytes at a time
cld                                ; 3857 MOVSW will run forward
rep movsw                      ; 3858 the picture is down in the live tile

```

TILE_70 is exactly **TILES** + **70h*10h**, the picture of tile **70h**. The destination is the tile itself: sixteen bytes, and a blinking floor blinks all at once throughout the room, however many cells it fills.

The technique has one flip side, and it is a real one: the tile is spoiled permanently. It can only be restored from a reference copy, and that is exactly why the file contains **TILRST** (05FFh) and duplicate pictures — the subject of [Chapter 13](#).

The phase-0 sets for both blinking tiles are **TILE_00**, the empty tile. That is, phase 0 means “the tile is not visible”:

tile	phase 0	phase 1	period
70h variant A	empty, 6 frames	picture from 3B4Ch , 26 frames	table at 3862h
71h	empty	picture from 3B28h	exactly 10h frames , 3798h

The duty cycle of **70h** is asymmetric, and this is the disappearing floor: it is absent for six frames out of thirty-two. That it is not decoration is clear from who reads the phase: **PH70A** (3757h) is read by **PST_0** and **PST_5**, that is, **the player’s physics consults the blink phase**; **PH71** (3761h) is read by **PLRHIT** — tile **71h** kills in only one of its two phases. Gameplay timing driven straight out of the animation state, with no extra variable to keep in step.

12.6. Two Calls Lifted Out of ANIMTL

The body of **ANIMTL** consists of four identical blocks of “decrement a counter, take a step at zero”. Two of them do nothing:

```

    dec    TMR70          ; 3779 pit timer
    jg     LANIMT1        ; 377D jump lands zero bytes ahead
LANIMT1:
    dec    TMR70A         ; 377F timer of floor 70h, first phase
    jg     LANIMT2        ; 3783 phase is not up yet
    call   BLNK70A        ; 3785 swap the picture of floor 70h
LANIMT2:
    cmp     PH70A, 0       ; 3788 second phase of floor 70h
    je     LANIMT3        ; 378D phase is not armed
    call   BLNK70B        ; 378F swap the picture of floor 70h, second variant
LANIMT3:
    dec    TMR71          ; 3792 timer of floor 71h
    jg     LANIMT4        ; 3796 phase is not up yet
    mov     TMR71, 10h     ; 3798 10h = 16 frames to a phase
    call   BLNK71         ; 379E swap the picture of floor 71h
LANIMT4:
    dec    TMR71C         ; 37A1 scroll timer of tile 71h
    jg     LANIMT5        ; 37A5 phase is not up yet
    mov     TMR71C, 2      ; 37A7 2 frames to a phase
    cmp     PH71, 0       ; 37AD scroll phase
    je     LANIMT5        ; 37B2 jump lands zero bytes ahead
LANIMT5:
    ret                ; 37B4 return: the tiles have been cycled

```

The counters turn, the conditions are tested, and there is nothing to call: the **CALL** has been lifted out of both blocks. Both jumps lead to the very next instruction, zero bytes forward. Every frame, twenty-four times a second, the game decrements two timers and compares them with zero to no effect whatsoever.

Where those calls went is visible from the orphaned routines: **PITANI** (37DCh, scrolling the pit and the torch) writes to **TMR70**, and **BLNK71C** (3892h, four frames of tile 71h from 3B28h) has no references at all. Four tile animations were built, two are in use; the routines themselves are intact and still in place.

Lifting out the **CALL** and leaving the counter is the standard way to switch a piece of the picture off: it switches back on with a single instruction, whereas ripping the whole block out is only worth doing once you are certain it will never be wanted again. Two decrements a frame is a modest price for being able to bring the animation back in a minute — and it is why we can still see, forty years later, exactly what those animations were.

The third counter, **TMR70A** (3755h), looks the same — **ANIMTL** only decrements it — but it is live: **BLNK70A** (3846h) reloads it itself from the table at 3862h. That is the mechanism behind the asymmetric duty cycle.

13. Light and Darkness

The pyramid is deep, and it is dark down there. The requirement sounds expensive: half the room has to vanish, the other half has to stay, the physics must not change, and all of it on a machine where redrawing the screen costs four frames.

The solution — one of the thriftiest in the file — costs four **REP STOSB** instructions. The game touches neither the screen nor the logic. It puts out the **tile pictures**: it zeroes bytes inside **TILES**, and brick becomes an empty square. The room map does not change by a single cell.

13.1. Who Decides Whether It Is Dark

LITCHK (0DA2h) answers the question with one carry flag: **CF = 1** means dark. The rule is four lines long:

```

    mov     al, FLOOR      ; 0DA5 floor of the pyramid
    cmp     al, 9          ; 0DA8 floor 9 -- the treasure chamber, always lit
    jge     LLITCH2        ; 0DAA floor 9 and above -- lit
    clc     ; 0DAC clear the carry ahead of the ADC
    adc     al, LEVEL      ; 0DAD depth plus difficulty
    cmp     al, 9          ; 0DB1 nine -- the threshold of darkness
    jl      LLITCH2        ; 0DB3 shallow or easy -- lit

```

After that comes a search for the torch (**ELM_TRCH**, code 3Fh) in the inventory: found, and it is light.

The formula is short: **depth plus lap**. `FLOOR + LEVEL >= 9` means dark. The lap number acts as a correction on the depth: on the first lap only floor 8 is dark, on the third it is 6, 7 and 8, and every subsequent lap eats another floor of light ([Chapter 17](#)). One addition, and the pyramid gets darker as you get better at it.

`FLOOR = 9` is checked separately above and never darkens — it is not a floor of the pyramid at all but the treasure room ([Chapter 17](#)), and the player is not asked to loot it in the dark.

`LITCHK` puts its result into two cells: `LITMSK` (`10A8h`) and `DARKF` (`07BDh`), both `0FFh/0F0h`. On the first of those, see 13.7.

13.2. What `DARKEN` Puts Out

`DARKEN` (`05C7h`) is four `REP STOSB` runs of zeros, and the addresses are worth writing down:

destination	bytes	what it is
<code>15C5h</code>	<code>20h</code>	tiles <code>01h...02h</code> — floor
<code>15E5h</code>	<code>310h</code>	tiles <code>03h...33h</code> — walls, ladders, items
<code>1CD5h</code>	<code>60h</code>	tiles <code>72h...77h</code> — conveyor belts
<code>38F0h</code>	<code>180h</code>	the working copy of the animated tiles

The first three blocks are obvious — 57 living tiles. The fourth looks superfluous: why put out some working copy that nobody displays? For a very good reason.

`BELTANI` (`37B5h`) copies the conveyor belt over tiles `72h...77h` every frame, taking the current phase from the table at `3A70h`. And that table points **into that very working copy**:

phase 0 -> <code>38F0h</code>	phase 2 -> <code>39B0h</code>
phase 1 -> <code>3950h</code>	phase 3 -> <code>3A10h</code>

Four phases of six tiles at `10h` bytes each is exactly `180h`, exactly the block `DARKEN` clears. Without that fourth block, `ANIMTL` would restore the belts on the very next frame and the conveyors would glow in a dark room: the only luminous stripes in pitch blackness, and precisely where the player is standing. Four `REP STOSB` instead of three, and a bug that would have been maddening to track down never happens.

The reverse operation is done by `TILRST` (`05FFh`) over the same four blocks, only with `MOVSB` from the reference copies at `1D95h`, `1DB5h`, `20C5h`, `2125h`.

13.3. The Physics Does Not Change At All

This is the heart of the design, and it is the dividend on the kilobyte spent on `SCRMAP` back in [Chapter 8](#). What gets put out is the pictures, and all the mechanics read tile numbers. A wall that has become invisible stays solid; a ladder you cannot see is still a ladder; a pit kills exactly as reliably as it does in the light.

There is not one “if dark” branch anywhere in the physics: `DARKF` is read by exactly one routine, and that routine is not physics (see 13.6). Darkening costs four `REP STOSB` on entering a room and zero cycles during play.

You cannot do it more cheaply. You can only not do it at all.

13.4. What Stays Visible

Tiles `34h...71h` and `78h` and above are not put out. What that means for the player can be read off `ELMTAB` — from the four tiles of each item:

item	code	2×2 tiles	in the dark
brown key	4Bh	43h 42h 45h 44h	visible
green key	43h	3Bh 3Ah 3Dh 3Ch	visible
red key	47h	3Fh 3Eh 41h 40h	visible
torch	3Fh	7Dh 7Ch 6Fh 6Eh	visible
sword	37h	25h 24h 27h 26h	not visible
amulet	2Fh	2Dh 2Ch 2Fh 2Eh	not visible
jewel	2Bh	31h 30h 33h 32h	not visible

The line was not drawn by accident, and at this point it stops being economy and becomes design. What remains visible in the dark is **everything you cannot get past without**: the three keys, and the torch that lifts the darkness. Everything that is worth only points — sword, amulet, jewel — vanishes along with the walls.

So a dark room can always be completed, but it can only be stripped bare with the lights on. A player who wants treasure goes looking for the torch. An entire game mechanic that cost not one instruction: it fell out of which tiles happened to land inside the range being zeroed — and somebody noticed that it had, and arranged the tile numbering so that it kept working.

Tiles 70h and 71h also stay visible — the blinking floor and the lethal tile. ANIMTL goes on animating both, because their frame sets live separately (3B6Ch, 3B48h) rather than in the working copy that gets cleared.

Hazards, in other words, do not hide in the dark. This is one of those decisions where the player's interests and the data layout happened to coincide: the frame sets for the blinking tiles are simply not in the block that gets cleared. Noticing a coincidence like that and leaving it alone is work too, and it is the kind nobody thanks you for.

13.5. Darkness Does Not Touch the Sprites

DARKEN writes only into TILES and the working copy. Sprite frames live at 3BE0h...4B9Fh, and DRWSPR (5D31h) puts them straight into video memory through MSKTAB without going near TILES.

So in a dark room the pyramid disappears while Panama Joe, the skulls and the snakes stay hanging in the void in full colour. Technically this is unavoidable — sprites and tiles live in different worlds. What the player gets is the best scene in the game: a black screen, a glowing little man, and three skulls flying at him out of nowhere.

Room 3, non-empty pixels across all of video memory:

room	from tiles	after the sprite	sprite contributed
lit	48,452	48,532	80
dark	732	950	218

The sprite draws its 218 pixels either way; in a lit room you only see 80 of them added, because the rest landed on background that was not empty to begin with. The 732 pixels surviving in the dark are the status panel and the tiles that stayed: in room 3 that is 22 cells out of a thousand.

Put another way, a dark room leaves about one and a half per cent of the picture on screen — and the game remains completely playable, because everything the physics needs is in SCRMAP and not on the screen at all. One and a half per cent of the pixels and one hundred per cent of the game.

13.6. Picking Up the Torch Mid-Room

Everything so far happens on entering a room. But the torch can be picked up while already standing in the dark, and the room has to brighten immediately. That is **LITNOW**'s job (**60DEh**) — a flag that lives for exactly one frame.

PLRHIT clears it in its very first instruction every frame (**6FDBh**) and sets it only here:

```
cmp    DARKF, 0          ; 7167 is the room dark
je     LPLRHI22          ; 716C it is lit
mov    al, LASTITM       ; 716E AL -- what was picked up
xor    ah, ah            ; 7171 AH=0: the whole code now sits in AX
cmp    ax, 3Fh           ; 7173 3Fh -- the torch
jne    LPLRHI22          ; 7176 not the torch
mov    LITNOW, 1         ; 7178 a torch in the dark -- the lighting has changed
```

That is the one and only reader of **DARKF**. Then **ST_PL2**, immediately after **PLRHIT**, sees the flag and orders a full redraw (**624Ch**):

```
mov    MKOBJF, 0         ; 6253 do not create the objects over again
call   ROOMDRW           ; 6258 rebuild the room map and redraw the whole frame
mov    MKOBJF, 1         ; 625B objects are created as usual from here on
```

The brackets around the call matter more than the call. **MKOBJF** (**6CB3h**) is read by **MKROOM** (**0807h**) and **PUTOBJ3** (**0E1Ah**), and at zero the first **does not reset INBITS to 1** and the second **does not create enemies**. So the room is rebuilt from scratch while the live objects stay as they were. Otherwise picking up a torch would resurrect everyone you had killed and put them back in their places — an expensive lamp, that would be. Two instructions of bracketing, and a whole class of bug is closed.

13.7. LITMSK — the Scheme That Lost

LITMSK (**10A8h**) looks like **DARKF**'s twin: it is written by **LITCHK** (**0DCBh**, **0DD9h**) and **PUT2X2** (**0D0Eh**), with the same **0FFh** and **0F0h**. It has no reader.

Three writes, no reads — there is no instruction anywhere in the image that takes **LITMSK** by name. Its neighbour **LITNOW**, alive and working, is read some eighty times in a hundred and fifty frames; **LITMSK**, not once.

The name tells you what it was for. **0FFh** and **0F0h** are a nibble mask, one that would have knocked out every other pixel instead of the whole picture. The darkness would have come out as a haze with things half-visible through it — a lovely effect, and a more expensive one. The cheaper scheme won: zero the tiles outright. The cell belonging to the scheme that lost stayed where it was, and is written to faithfully to this day. It is the clearest surviving evidence in the image that darkness was designed twice.

13.8. The Order Within a Frame

The full redraw is done by **DRAWFRM** (**627Ch**), and the order goes like this:

```
DRAWFRM:
call    ANIMHIT          ; 627C collisions and frame advance for every animation chain
call    DRWALL           ; 627F redraw the whole screen from the map
call    MKSOLD           ; 6282 build the floor-span list by scanning SCRMAP
call    DRWALL2          ; 6285 draw every sprite, saving the background under it
call    NOP0             ; 6288 empty routine: one of eight such calls in the image
cmp     LITNOW, 0        ; 628B did the lighting just change?
jne     LDRAWF1          ; 6290 yes -- put the master tiles back
call    LITCHK           ; 6292 is the room dark
jae     LDRAWF1          ; 6295 CF=0 -- the room is lit
call    DARKEN           ; 6297 dark -- blank the tiles out
jmp     LDRAWF2          ; 629A the lighting did not change

LDRAWF1:
call    TILRST           ; 629D refill the live tiles from their master copies

LDRAWF2:
call    DRWALL           ; 62A0 redraw the whole screen from the map
call    DRWALL2          ; 62A3 draw every sprite, saving the background under it
call    SHOWSCR          ; 62A6 push the finished map out to video RAM
```

The room is drawn **twice**: between the two **DRWALL** calls sits either **DARKEN** or **TILRST**, so the tile pictures change and the second pass rewrites the screen. Nothing in between reads video memory — **MKSOLD** works off **SCRMAP**, and the second **DRWALL2** saves the background into **OBJTAB** again on top of the first.

The cost is known: one full redraw of room 3 is **56,308 instructions**, so entering a room costs twice that, about 150 milliseconds extra. The frame rate is unaffected: **DRAWFRM** runs on entering a room and on **LITNOW**, not every frame. The lighting block was appended to a finished routine rather than woven into it — an edit that cannot break anything already working, made at the one moment in the frame where 150 milliseconds are affordable. Given the choice between elegant and safe here, safe is also correct.

Part IV. The Game

14. Items, Inventory, Doors, Score

Everything you can take in the pyramid comes to seven items, three doors and five inventory slots. Let us see what that machinery cost.

14.1. An Item Is Four Bytes in `ELMTAB`

A collectable occupies a **2×2 tile** square on the map and is described by four bytes in `ELMTAB` (`249Dh`). The element code is an offset within the ribbon, and the anchor is at the top right corner:

```
+1 +0
+3 +2
```

The seven items and what they are worth:

code	what it is	points	what goes into the inventory
4Bh	brown key	50	4Bh
43h	green key	50	43h
47h	red key	50	47h
37h	sword	50	3Bh
2Fh	amulet	100	33h
2Bh	jewel	1000	nothing
3Fh	torch	3000	3Fh

The code substitution for the sword and the amulet is not accidental. `3Bh` and `33h` are the same pictures, but assembled from the **reference** tiles `A1h...A4h` and `A9h...ACh`.

The difference is that the reference tiles are not cleared by `DARKEN` and not overwritten by `ANIMTL`. A sword lying on the floor of a dark room has to vanish along with the walls; a sword drawn in the status panel has to be visible at all times. The same picture, two different tile sets and one extra element code — and the whole problem is solved without a single conditional anywhere in the drawing path.

14.2. Inventory: Five Slots and No Eviction

`ITEMS` (`6CA5h`) is five words, `ITEMCNT` (`6CA4h`) says how many are in use. `TAKEITM` (`725Dh`) opens with a check:

```
mov    al, ITEMCNT      ; 7260  how many we carry already
cmp     al, 5            ; 7263  the inventory holds five slots
jge     LTAKEI5          ; 7265  full -- it will not fit
```

Overflow is handled in the shortest way there is: **the item is simply not picked up**. No eviction, no choice of what to drop, no message, no sound. `CF = 0`, and the thing stays on the floor. Anything else would have needed a selection window, and there is nothing in this game to draw one with.

The check sits **before** the item code is decoded, and that has a side effect. The jewel (`2Bh`) takes no slot at all — it only gives points — but with five things in hand you cannot pick it up either, because control never reaches the decoding. A thousand points lie underfoot and stay there because the counter has hit five.

Three instructions at the top of the routine instead of decoding first and checking after: the exchange costs one item out of seven, saves a handful of bytes in the busiest pickup path in the game, and gives the player a reason to think about what they are carrying. Games have shipped worse inventory rules on purpose.

TAKEITEM can only append to the end of the list (**7288h**), and that is the whole inventory: five words, a counter, and appending to the tail. Nothing more is needed.

14.3. A Door Opens on the Picture, Not on the Code

USEITEM (**7298h**) compares not the item code but **the first byte of its description** — the tile in the top right corner. The door’s colour matches the key’s: **0Fh** green → green key, **16h** red → red, **1Dh** brown → brown.

Comparing pictures rather than codes is the same kind of move as everything else here: do not introduce a “door → key” table when the colour is already drawn into both of them. The artwork is the lookup table.

There is one trap for the reader. The number **3Bh** appears in two different roles: it is the green key’s tile, and it is also the code under which the sword sits in the inventory. The coincidence is accidental, but working from the tables alone you could easily conclude that the green door opens with a sword. It does not.

14.4. The Doorway: Seven Bytes for Closed and Open

A door is described by seven bytes rather than four:

```
+0      closed-door tile, laid out as a 1x4 column by PUTROPE
+1 .. +6 open doorway 3x4, read right to left
```

The open doorway is laid down by **PUTGRID** (**0B2Fh**), and its layout is not obvious: the top row comes from bytes **+3 +2 +1**, and the three lower rows are **identical**, all three taken from **+6 +5 +4**. All three doors are built the same way and differ only in tile numbers: **4Fh** green, **56h** red, **5Dh** brown.

Six bytes instead of twelve. A doorway four cells tall whose three lower rows are copies of one another, and that fact is recorded not in the data but in the loop: the door was drawn from the start with a repeating bottom in mind. Artwork and code designed together, which is what you get when the same small team does both.

14.5. **SEENMAP**: One Bit per Object, Sixteen per Room

What has already been collected is remembered by **SEENMAP** (**10AAh**) — **101 words, two bytes per room**. The bit inside a record is not the item code and not the slot number but **the object’s ordinal position in the room**: **ROOMOBN** counts objects as the room’s bytecode is decoded, and **OBJSEEN** (**09ECh**) shifts the mask by the same number of places:

```
LOBJSE1:
    dec    c1                ; 0A00 one object fewer between us and the one we want
    j1     LOBJSE2           ; 0A02 we have arrived at our bit
    rcl    al, 1             ; 0A04 shift the mask; a carry means we left the byte
    jae    LOBJSE1           ; 0A06 the bit is still inside the first byte
    inc    bx                ; 0A08 we have crossed into the second byte of the record
```

The **ROOMOBN** counter advances **even if the object has already been collected** (**0A0Dh**) — otherwise the numbering would slide and a key you had picked up would rename its neighbour.

Sixteen bits per room is the ceiling, and that ceiling was not picked by eye. The most populated room in the pyramid contains **twelve objects** (rooms 4 and 59, at any difficulty), leaving exactly four bits of headroom.

A seventeenth object would carry the mask into the neighbouring record, and a key collected in one room would go missing in another. There is no check against that: its role is played by the fact that the rooms were laid out by a person who knew about the sixteen-bit limit. In a game where the data author and the code author are the same person, that is a perfectly sound place to put the invariant.

Enemies are marked by the same mechanism, but only while `LEVEL < 5`: above that an enemy is reborn on every entry ([Chapter 17](#)).

14.6. The Amulet: 55 Frames of Invulnerability

`ITMTMR` (`6CAfh`) is set to `37h` when the amulet is picked up (`714Ah`) and decremented by one at the top of `PLRHIT` (`6FF7h`). While it is non-zero, `PLRHIT` at `708Bh` does not examine collisions at all — the player is invulnerable.

`PLRHIT` is called exactly once per game frame, without exception. So `37h` is **55 frames**, and at a frame budget of about 41 ms ([Chapter 6](#)) that is roughly **2.3 seconds** of invulnerability. Long enough for Panama Joe to walk straight through a skull, which is precisely the intended sensation.

14.7. The Panel: `DIGELM` Is a Digit Font

`DIGELM` (`253Eh`) looks like a table of item codes and matches the item codes in not a single row. The answer is in `PUTITM` (`6FA7h`), a routine of ten instructions:

```

mov     si, ax                ; 6FB0 SI -- the same code as an index
mov     al, DIGELM[si]        ; 6FB2 AL -- offset of the digit's element
mov     si, ax                ; 6FB6 SI -- offset of the element within the ribbon
mov     bx, OFFSET ELMTAB     ; 6FB8 BX -- head of the element ribbon
call    CURSP0S              ; 6FBB DI -- the cell, from CMDOP and CMDX
call    PUT2X2               ; 6FBE the icon is laid down as a 2x2 square

```

And this is how `SHOWSCR` feeds it:

```

mov     al, SCORE[di]         ; 6EAE AL -- a byte of the score
and     al, 0Fh               ; 6EB2 the low BCD digit
call    PUTITM                ; 6EB4 the digit is out in the panel
...
mov     al, LEVEL             ; 6EEA AL -- the difficulty level
call    PUTITM                ; 6EED the level digit is out in the panel

```

`PUTITM`'s input is a **decimal digit, 0...9**, and `DIGELM` translates it into the element code holding the picture of that digit. Ten codes, `79h`, `7Dh`, ... `9Dh` at a stride of 4 — a ten-glyph font, 2×2 tiles per glyph, drawn in two colours, brighter on top and darker below.

Which is to say there is no separate font in this game at all. The score digits are drawn out of the same `ELMTAB` elements as the brick in the wall, by the same `PUT2X2` that places keys around the rooms. The status panel is simply the top five rows of the same map the pyramid lives on. One drawing path for the whole game, and the digits came free.

The score is written right to left from `CMDX = 4Dh` at a stride of -2, with leading zeros suppressed (`6E9Dh`); the level is one digit at `CMDX = 34h`.

14.8. Lives, and the Last Three Bytes of `ELMTAB`

The three bytes at `2513h` look like a stray tail at the end of the ribbon: no element code appearing in any room points there. `SHOWSCR` points there — it is element `76h`, the life glyph:

```

LSHOWS9:
mov     CMDX, 79h             ; 6F46 79h -- cell of the first row of lives
mov     ax, 76h               ; 6F4B 76h -- element of the life icon
mov     ELMCUR, ax            ; 6F4E what to draw it from
mov     al, LIVES              ; 6F51 AL -- how many lives there are
cmp     al, 0                  ; 6F54 none at all?
je      LSHOWS12              ; 6F56 nothing to draw
shl     al, 1                  ; 6F58 two cells to an icon
cmp     al, 0Bh               ; 6F5A 0Bh = 11 cells -- the limit of the row
jb      LSHOWS10              ; 6F5C it fits
mov     al, 0Ah               ; 6F5E 0Ah = 10 cells -- five icons
LSHOWS10:
mov     CMDP1, al              ; 6F60 length of the icon strip
call    PR_PATT               ; 6F63 the first row of lives is out

```

249Dh + 76h = 2513h, and what lies there is DB 02h,38h,39h — a pattern of length two made of tiles 38h and 39h. The life glyph is drawn by the same PR_PATT that lays a brick wall in a room: the lives are a strip tiled with a pattern, and the strip's length is the number of lives times two. A row of lives in the HUD costs the same code as a wall, which is exactly the sort of reuse a 32K budget rewards.

The first row holds five lives (cmp a1,0Bh clips it at 0Ah tiles), the second another six. **The panel can show no more than eleven**, however many there are; and more than eleven is entirely achievable, since the counter has no upper bound (Chapter 17). A twelfth life is awarded and announced with a sound, but the panel keeps its own counsel about it.

14.9. ELMTAB Is Mapped End to End

The ELMTAB ribbon is **161 bytes, and every one of them has a reader**: 00h...4Bh are 2×2 squares, 4Fh, 56h, 5Dh are doors, 64h...66h ropes, 6Ah...72h enemy animation chains, 76h the life glyph, 79h...9Dh the digits.

Not one spare byte, not one hole, records overlapping each other to save space. And three kilobytes away sit the two pyramid tables that nobody calls on (Chapter 4), and two kilobytes of background buffer inherited from a machine with two screens (Section 11.9).

There is no contradiction. ELMTAB is worked over to the last byte because every byte in it is multiplied by ninety-nine rooms. Two hundred bytes of tables and two kilobytes of buffer are multiplied by nothing and get in nobody's way, and the whole lot fits on a floppy. Effort goes where effort pays — and knowing the difference is most of what engineering judgement is.

14.10. A Quirk in the Inventory Clear

VIDINI clears the inventory like this (06B5h):

```
mov    al, 0           ; 06B5 AL=0 -- what to fill with
mov    cx, 5           ; 06B7 CX=5 -- that many bytes
mov    di, OFFSET ITEMS ; 06BA DI -- start of the inventory
rep stosb              ; 06BD clears 5 bytes out of the 10
```

ITEMS is declared as five **words** — ten bytes, 6CA5h...6CAeh — and REP STOSB with CX = 5 clears five of them. The upper half of the array keeps whatever the previous session left there.

It makes no difference to anything. ITEMCNT is set to zero, slots are only read up to ITEMCNT, and a slot is always written before it is read. The five high bytes simply never come into play.

It is worth noticing all the same, because of what it says about the design. The inventory is arranged so that an uninitialized slot cannot be reached from anywhere: the counter, not the contents, defines what is in the list. Structures with that property are forgiving, and this one is forgiving here. The line immediately below, clearing SEENMAP, counts in bytes for a word array too — CX = 0CAh = 202 bytes = 101 words — and both lines do exactly what the surrounding code needs them to.

15. Player Physics: Ten States, the Jump, Room Edges

The game has two state machines, built the same way: a state byte, a table of addresses, the same DISPATCH.GSTATE (Chapter 5) decides what is happening on the screen at all; PLRST (651Fh) decides what is happening to Panama Joe.

The whole of his physics — the jump, falling, ladders, conveyors, three ways to die — is ten entries in a table and one table of thirteen signed bytes. No accelerations, no masses, no friction.

15.1. The Dispatcher

PLRSTEP (65A7h) is five instructions:

```
PLRSTEP:
mov     bx, OFFSET PSTTAB      ; 65A7 BX -- table of the ten player states
mov     bp, cs                 ; 65AA BP -- segment of that table, for DISPAT
mov     al, PLRST              ; 65AC AL -- number of the player's state
call    DISPAT                ; 65AF call the handler at PSTTAB[AL]
call    MOVEPLR               ; 65B2 apply the speeds to the coordinates
```

First the state handler sets the velocities, then **MOVEPLR** (6B0Bh) applies them. The separation is strict: **no handler moves the coordinates itself**, it only writes **OBJVX** and **OBJVY**.

The reason is simple: room edges, room transitions and saving state across a transition are all done by **MOVEPLR**, once, in one place. In ten separate handlers that logic would take ten times the bytes, and would be ten times as easy to get subtly wrong.

PSTTAB (65B6h) is ten words, and three pairs lead to one handler:

PLRST	handler
0	PST_0 65D3h
1	PST_1 65CAh
2	PST_2 678Fh
3, 4	PST_34 695Ch
5	PST_5 6805h
6	PST_6 67C6h
7, 8, 9	PST_789 6A5Bh

15.2. What the States Are

The state numbers say nothing about themselves. What each one leaves **PLRSTEP** with, however, says everything: an animation chain and a pair of velocities.

PLRST	ANIMPTR	OBJVX	OBJVY	what it is
0	3B80h 3B82h 3B88h 3B92h	0, ±1	−1, 0	standing, reading input
1	3B80h 3B88h	±1	0	walking
2	3B92h	0, 1	−6...+8	jumping
3	3BA8h	0	−2	climbing up
4	3B96h	0	+2	climbing down
5	3B94h	0	+3, +4	falling after a landing
6	3B88h 3B96h	0	+3, +8	stepped off, dropping
7	—	—	—	killed by an enemy
8	3BACH	0	0	killed by the landing
9	3B9Ah	0	−1	killed by a lethal tile

The **OBJVY** range in state 2 — −6 −4 −2 −1 +1 +2 +4 +8 — is the whole of **JUMPTAB**, and it is also what gives state 2 away as the jump.

Three of the ten states are ways to die, and they are different ways: by an enemy, by hitting the ground, and by a lethal tile. Each death has its own frame chain and its own sound. Three distinct deaths in a 32K game is generosity.

15.3. Who Switches State, and When

There are eight instructions writing to **PLRST** in the whole image, and the important one is a single **MOV PLRST, AL** in **LPST3**. Every physics decision converges there with a ready value in **AL**; the other seven writes are the initial setup, two deaths, and the return afterwards.

One write point instead of a dozen scattered **MOVs**: cheaper in bytes and far easier to debug. Here is where each value comes from.

value	from	condition
3	6603h	LADDER found a ladder, OBJVY = 0FEh (−2)
4	6662h	tile underfoot is 6: descent, OBJVY = +2
2	66C9h	JUMPIDX = 0Ch, OBJVY from JUMPTAB
1	66FBh	OBJVX ≠ 0 and there is support
6	6721h	no support: OBJVX = 0, OBJVY = 8
5	68A2h	landing: OBJVY = 4, sound 2
8	6957h	killed by impact: chain 3BACH, OBJTMR2 = 3Fh
9	6879h	tile from DEADTIL : sound 6, OBJTMR2 = 3Fh
7	719Fh	PLRHIT : an enemy got him, sound 6
SVST	6A7Eh, 6AD2h	return after death

15.4. The Jump

JUMPTAB (6510h) is thirteen signed bytes:

index	:	0	1	2	3	4	5	6	7	8	9	10	11	12
signed	:	+8	+4	+4	+2	+2	+1	0	-1	-2	-2	-4	-4	-6

At the start of a jump **PST_0** sets **JUMPIDX** = 0Ch and takes the rightmost byte, −6. After that **PST_2** runs every frame:

```

PST_2:
    dec     JUMPIDX           ; 678F  jump phase one step back
    jge     LPST19           ; 6793  the arc is not finished
    mov     JUMPIDX, 0       ; 6795  the arc is finished: sit at zero from here on
LPST19:
    mov     si, JUMPIDX       ; 679B  SI -- the jump phase
    mov     bx, OFFSET JUMPTAB ; 679F  BX -- table of jump speeds
    mov     al, [bx+si]      ; 67A2  AL -- Y speed at this phase

```

The index counts down, the table is read right to left, and out comes a parabola:

frame	OBJVY	height
0	−6	−6
3	−2	−16
6	0	−19
9	+2	−14

frame	OBJVY	height
12	+8	+2

Peak at 19 pixels, flight time 13 frames. After that the index sticks at zero and yields +8 forever.

Which is the table’s best property: **its lower end is not part of the jump at all, it is terminal falling velocity**. Thirteen bytes serve as the initial push, as gravity, and as terminal velocity. No $V += G$ every frame, no multiplication, no fractions: read a byte by index, decrement the index. A complete physics engine in thirteen bytes, and it feels right to play — which is the part that cannot be derived from the listing and had to be tuned by hand.

The ceiling. If the player is right at the top ($OB_Y \leq 0Ah$) and the next step would take Y past zero, OBJVY is zeroed (67B9h), while JUMPIDX carries on counting down:

```
starting Y 60h:  -4 -4 -2 -2 -1 +0 +1 +2 +2 +4 +4 +8 +8 ...
starting Y 04h:  -4 +0 +0 +0 +0 +0 +1 +2 +2 +4 +4 +8 +8 ...
```

The rise is clipped, and the fall begins at exactly the moment it would have without a ceiling. There is no special “head bump” in the code: Joe presses against the top edge for a couple of frames and falls on schedule. Handling the bump separately would have cost a dozen bytes and some fiddling to make it look natural; players hit the ceiling rarely, and the budget went somewhere it shows.

A jump cannot be interrupted. PST_2 never reads CMDBITS, and neither does PST_6, which it falls through into. While PLRST = 2, input has no effect on Joe.

That is a deliberate decision. A thirteen-frame jump steerable in flight would need tests in every frame, and the parabola would stop coming out of one table. What the player gets instead is a rule learnable in five minutes: think before you jump. Every platformer of the era that felt precise made the same choice.

PST_2 ends by falling through into PST_6. At 67C5h there is a POP AX, at 67C6h PST_6 begins, and there is no RET between them. So every frame the jump is finished off by the falling code — and that is where catching a ladder or a rope in mid-air comes from.

15.5. Support and Death

PST_5 (6805h) decides whether the fall has ended from the tile underfoot:

```
LPST25:
    mov     al, TILEDN          ; 6810 AL -- the tile underfoot
    xor     ah, ah              ; 6813 AH=0: the whole tile now sits in AX
    cmp     ax, 0               ; 6815 nothing underfoot?
    je      LPST27              ; 6818 nothing -- keep falling
    cmp     ax, T_PASS          ; 681A a walk-through tile: nothing to stand on
    je      LPST27              ; 681D keep falling
    mov     si, 5               ; 681F five tiles in the list
    mov     bp, OFFSET LANDTIL ; 6822 the tiles you land on
    call    INLIST              ; 6825 is AL in the list
    jb      LPST28              ; 6828 CF=1 -- a landing
    call    DOORSID             ; 682A a belt is footing too
    jb      LPST28              ; 682D CF=1 -- landing on a belt
    xor     ah, ah              ; 682F AH=0: the whole tile now sits in AX
    cmp     ax, T_SOLID         ; 6831 solid floor with speckle
    je      LPST26              ; 6834 that one -- go and check the blink phase
    cmp     ax, 0               ; 6836 nothing underfoot?
    jne     LPST29              ; 6839 something there -- go and test the deadly tiles
LPST26:
    cmp     BYTE PTR PH70A, 0   ; 683B blink phase of floor 70h
```

The line at 683Bh confirms [Chapter 12](#) directly: **the physics consults the blink phase**, and the disappearing floor holds the player exactly when it is visible. One comparison, and the animation and the rules can never disagree.

If there is no support and the tile at the player’s position is in DEADTIL (6 entries) — chain 3B9Ah, sound 6, OBJTMR2 = 3Fh, PLRST = 9.

15.6. Conveyor Belts

BELTPUSH (6726h) is fourteen instructions:

```
BELTPUSH:
    mov     al, TILEDN          ; 6726 AL -- the tile underfoot
    call    DOORSID            ; 6729 is it a belt underfoot, and which way does it run
    jae     LBELTP1             ; 672C CF=0 -- not a belt
    mov     al, FRAMEC          ; 672E the belt only works every other frame
    and     al, 1               ; 6731 bit 0 of the frame counter
    je      LBELTP1             ; 6733 even frame -- the belt stands still
    mov     bx, OFFSET BELTVX   ; 6735 the two belt speeds: left and right
    mov     al, cs:[bx+si]       ; 6738 AL -- speed of this belt
    add     OBJVX, al           ; 673B the belt has added its speed to the player
```

BELTVX (6740h) is two bytes, 01h and 0FFh: plus a byte or minus one, with the direction taken from the same index DOORSID returned.

Two subtleties turn this into a conveyor rather than a trap. First, the belt **adds to** the player's own velocity instead of replacing it — so you can walk against the belt, just slowly. Second, it works every other frame, via **AND AL, 1** on the frame counter, so it pulls at half strength.

Half a step per frame against a full step of walking: exactly enough for the player to feel resistance and still be able to beat it. Fourteen instructions and a two-byte table, and the feel of the thing is right.

15.7. Room Edges

All four edges are handled by MOVEPLR after it has applied the velocities:

edge	condition	what it does
right	OB_X ≥ 4Ch	INC ROOMPOS, X := 1
left	OB_X < 1	DEC ROOMPOS, X := 4Bh
bottom	OB_Y/2 > 58h	INC FLOOR, Y := 2
top	OB_Y/2 = 0	if PLRST = 2 — Y := 2 and stay ; otherwise DEC FLOOR, Y := 0B1h

The top edge is the only one with a proviso, and the proviso carries real weight: **you cannot jump up a floor**. Touching the top edge in mid-jump bounces Joe to Y = 2 and he stays in the room.

One comparison of PLRST against two defines the entire topology of the pyramid: you can fall down from anywhere, but you can only go up by ladder or rope. Without that line the floors would be passable by jumping, and the pyramid would stop being a puzzle.

A room transition saves the whole state at once (6B88h):

```
    mov     SVY, al             ; 6B8B keep Y for the new room
    mov     al, [bx+OB_X]       ; 6B8E AL -- the player's X
    mov     SVX, al             ; 6B91 keep X as well
    mov     al, PLRST           ; 6B94 AL -- the player's state
    mov     SVST, al            ; 6B97 keep the state
    mov     ax, ANIMPTR         ; 6B9A AX -- the frame chain
    mov     SVANIM, ax          ; 6B9D keep the chain
    mov     bp, ax              ; 6BA0 BP -- head of the chain
    mov     ax, ANIMIDX         ; 6BA2 AX -- frame number within the chain
    mov     SVIDX2, ax          ; 6BA5 keep the number
    mov     si, ax              ; 6BA8 SI -- the same number as an index
    mov     al, ds:[bp+si]      ; 6BAA AL -- the frame
    mov     FRMSAVE, al         ; 6BAD keep the frame
    mov     ax, ANIMLEN         ; 6BB0 AX -- highest frame number in the chain
    mov     SVLEN, ax           ; 6BB3 keep the length
    call    ROOMDRW             ; 6BB6 build the new room and redraw it
    stc                        ; 6BB9 CF=1 -- the frame was cut short
```

PST_789 (6A6Ch and 6AC0h) restores all of it: after dying, the player reappears in the same pose and in the same place he entered the room in. At the same time 6A81h restores SVANIM, zeroes the velocities, and lights the player's bit in INBITS.

One of the seven cells is not invited to the party. **FRMSAVE** (6550h) is filled in right here, at 6BADh — and nobody touches it after that, for reading or writing. Eight bytes declared, one written, none read.

Saving the animation frame across a transition is a perfectly sensible thing to want: the set “place, pose, frame” is complete only as a trio. In the end the restore turned out to work from the **SVANIM** chain alone, and there is no visible difference on screen. So the third member of the trio stayed declared, written, and quietly unnecessary — which is what a design looks like when it turns out to be simpler than its author expected.

15.8. The Treasure Room Does Not Let You Fall

Twice in **PST_34** there is a test for room 64h:

```
    cmp    ROOMNO, 64h          ; 697D  64h = 100 -- the treasure chamber
    ...
LPST53:
    cmp    ROOMNO, 64h          ; 6A36  64h = 100 -- the treasure chamber
    mov    BYTE PTR OBJVY, 0     ; 6984  the treasure chamber does not let you climb up
    ...
LPST54:
    mov    BYTE PTR OBJVY, 0     ; 6A42  the player is brought to a halt
    ret                                ; 6A47  return: one step along the ladder is done
```

ROOMNO = 64h is set by **TREASR** (0A1Dh) — the treasure room’s pseudo-room, not one of the ninety-nine: **FLOOR** = 9, **ROOMRQ** = 0C8h, inventory cleared, the map built directly through **BLDMAP**, bypassing **MKROOM**.

The point of the test is straightforward: the treasure room has no floor, and ordinary physics would kill the player in the middle of his reward. So instead of falling, **OBJVY** is zeroed, and you cannot come off the rope in there.

Two identical tests in two places in one routine rather than a shared “no falling here” flag is a considered choice: a flag would have to be checked every frame of ordinary play, whereas comparing a room number costs the same and lives in a branch reached once per lap. The cheap thing in the hot path, the slightly redundant thing in the cold one.

16. Enemies: Four Behaviours and the Hit Box

There are up to seven enemies in a room, and every one of them is one of objects 1...7 from [Chapter 11](#): the same six-byte record, the same sprite output, the same background snapshot. Exactly one thing sets them apart from the player: what does their thinking is not

PLRSTEP but **ENEMOVE**.

The whole of enemy behaviour is four handlers: the largest, **BEH_CLMB**, takes 288 bytes; the smallest consists of a single **RET**.

16.1. The Dispatcher and Half Speed

ENEMOVE (74B2h) opens by cutting out on frame parity:

```
    mov    al, FRAMEC           ; 74B9  AL -- the frame counter
    and    al, 1                ; 74BC  bit 0: objects move every other frame
    je     LENEMO3              ; 74BE  even frame -- no turn this time
    mov    al, INBITS           ; 74C0  AL -- one bit per live object
    mov    si, 7                ; 74C3  seven bits: the player does not count here
```

This is not work being spread across frames: on an even frame the routine exits entirely, having processed nobody. It is genuinely **half speed**.

One line solves two problems. The enemies move at half the player's speed with the same four-pixel step, which means you can outrun them. And they cost half as much: seven sprites processed every other frame cost the same as three and a half. In a budget where seven per cent of the frame covers all the work ([Chapter 6](#)), that is a real amount. A game-feel decision and a performance decision turn out to be the same instruction.

The walk runs from object 7 downward and stops at the first:

```
LENEM02:
    dec     si                ; 74CA on to the next object
    jg      LENEM01          ; 74CB slots not used up yet
```

JG, not JGE: at `SI = 0` the loop ends. Object 0 is the player, and he is never an enemy.

16.2. The Behaviour Type Packed into OBJKND

OBJKND (07ADh) is one byte per object, and it holds two fields. PUTOBJ3 puts them there from the primitive's CMDOP:

```
mov     al, CMDOP            ; 0E59 AL -- the opcode
and     al, 7                ; 0E5C bits 2..0 of the opcode -- the enemy type
mov     OBJTYP, al           ; 0E5E OBJTYP -- type of the enemy at hand
shl     al, 1                ; 0E61 type times two
shl     al, 1                ; 0E63 type times four: room made for the subtype
mov     bx, OFFSET OBJKND    ; 0E65 the "slot -> type and subtype" table
mov     [bx+si], al          ; 0E68 the type is stored
mov     al, CMDOP            ; 0E6A AL -- the opcode
rc1     al, 1                ; 0E6D shift 1 of three
rc1     al, 1                ; 0E6F shift 2 of three
rc1     al, 1                ; 0E71 bits 7..5 of the opcode come around through the carry
and     al, 3                ; 0E73 the low two bits are the subtype
or      [bx+si], al          ; 0E75 the subtype goes in beside the type
```

So OBJKND = (type shl 2) or subtype. OBJTYPE (6DA1h) extracts the type with a shift right by two; the subtype is read by the handlers themselves.

OBJTYPE opens with a proviso: if `SI = 0`, the type is set to 4. There is no fifth behaviour in BEHTAB, and no lookup will happen either — object 0 never arrives from ENEMOVE.

16.3. The Four Behaviours

BEHTAB (74F3h) is eight bytes, four addresses. Together with the frames from OBJFRM (06E0h) and the starting chain length from 06F0h, that is the game's entire bestiary:

type	handler	frame	ANIMLEN	what it does
0	7690h BEH_CLMB	4986h	1	walks, and can climb ladders
1	74FBh BEH_NONE	49F6h	1	stationary: a single RET
2	75E1h BEH_WALK	472Ch	7	walks, turns at walls and platform edges
3	74FCh BEH_JUMP	46D6h	0	hops along an arc

That also settles a debt from [Chapter 12](#): the four object frames are tied to types by a table, not by what the picture looks like.

The starting horizontal velocity comes from the four bytes of VXTAB (07B9h) — +1, -1, +2, 0 — and it is chosen by the **subtype** rather than the type. The same walker can be slow, fast, oncoming or standing still. Four behaviours times four subtypes, out of one byte.

BEH_NONE is worth pausing over. It is **one byte** — a RET at address 74FBh — and immediately after it, at 74FCh, BEH_JUMP begins.

A stationary enemy is a full entry in the behaviour table whose code is one byte long. Testing “is this type one?” in the dispatcher would cost more: a comparison and a branch, seven times a frame. Cheaper to call a routine that returns at once. There is something very satisfying about a bestiary in which “does nothing” is implemented as literally nothing.

16.4. The Hop Arc

BEH_JUMP drives OBJTMR2 along JUMPARC (7492h), 21 steps:

```
-9 -8 -4 -4 -2 -2 -2 -1 -1 +0 +0 +1 +1 +2 +2 +2 +4 +4 +8 +9 +1
```

Peak at 33 raster rows. The mechanism is the same as the player’s jump ([Chapter 15](#)), but the table is its own and twice as long: the player gets 13 steps and 19 rows.

The difference is elsewhere. The player’s index **counts down** and sticks at zero, which gives terminal falling velocity. Here the index **counts up** and wraps:

inc	BYTE PTR [bx+si]	; 750C	phase one step on
cmp	BYTE PTR [bx+si], 15h	; 750E	15h = 21 -- length of the jump arc
j1	LBEHJU1	; 7511	the arc is not finished
mov	BYTE PTR [bx+si], 1	; 7513	the arc starts over, from phase 1

The enemy’s hop is closed into a ring: on reaching the twenty-first step the index goes back to the first. A skull neither falls nor lands — it hops until the player leaves the room or dies.

No ground beneath it, no support check, no “in the air” state: a table of twenty-one numbers closed on itself. The game’s most memorable opponent, the bouncing skull, is twenty-one bytes of data and an index that wraps.

The subtype has a say here too: if bit 1 of OBJKND is set, the handler takes the **second step** from the table as well and adds them both (7538h). Such an enemy goes round the arc twice as fast.

16.5. Turning Around

All three mobile behaviours turn on TURNTIL (74AEh) — four tiles: 01h floor and the three doorways 0Fh, 16h, 1Dh. The tiles to the left and right of the object are tested.

BEH_WALK adds a second condition: **the edge of a platform**, turning when the tile underfoot is empty.

Which is why the snakes and spiders never tumble off the beams. An enemy has no notion of “falling” at all: anything that turns one step before the edge cannot fall. The player gets no such check — he falls from any height ([Chapter 15](#)). Two different sets of rules for the same six-byte record, and the asymmetry is exactly what makes the pyramid feel dangerous to you and safe for them.

16.6. The Hit Box

Collisions are computed by HITCHK (6DDFh), and its rule is four comparisons:

```

        mov     al, [bx+0B_Y]           ; 6DF1 the player's Y
        sub     al, [bx+di+0B_Y]        ; 6DF4 minus the object's Y
        cmp     al, 0                   ; 6DF7 is the difference negative?
        jge     LHITCH1                 ; 6DF9 no
        neg     al                       ; 6DFB take the absolute value
LHITCH1:
        shr     al, 1                   ; 6DFD bit 0 picks the field
        cmp     al, 7                   ; 6DFF seven lines -- the vertical limit
        jge     LHITCH3                 ; 6E01 further off than that -- no collision
        mov     al, [bx+0B_X]           ; 6E03 the player's X
        sub     al, [bx+di+0B_X]        ; 6E06 minus the object's X
        cmp     al, 0                   ; 6E09 is the difference negative?
        jge     LHITCH2                 ; 6E0B no
        neg     al                       ; 6E0D take the absolute value
LHITCH2:
        cmp     al, 4                   ; 6E0F four bytes -- the horizontal limit
        jl      LHITCH4                 ; 6E11 closer than that -- a collision

```

The units on the two axes are **different**, and that is where all the asymmetry comes from: **Y** is counted in raster rows (hence the divide by two), **X** in bytes of four pixels. Run an enemy across a grid around a stationary player and the hit box looks like this:

	dX in bytes (4 pixels each)										
dY	-5	-4	-3	-2	-1	+0	+1	+2	+3	+4	+5
-12	.	.	#	#	#	#	#	#	#	.	.
+0	.	.	#	#	#	#	#	#	#	.	.
+12	.	.	#	#	#	#	#	#	#	.	.
+14	.	.	.	.	.	.	.	.	.	.	.

It catches **dY** from **-13** to **+13** — 27 raster rows — and **dX** from **-3** to **+3** — 7 bytes, that is, 28 pixels. In screen pixels the rectangle is almost square, even though the numbers **7** and **4** in the code look nothing alike. Getting a square hit box out of two different units, with no conversion anywhere, is a small piece of arithmetic elegance.

The player's sprite, meanwhile, is 24 pixels wide and 20 raster rows tall (**PLR_W** = 6 bytes, **PLR_H** = 14h). So the hit box is **larger than the sprite on both axes**: 27 rows against 20, 28 pixels against 24. A skull kills Panama Joe without touching him — a third of his height above his head and a pixel or two to the side. The reason is price, not balance: two comparisons against constants instead of a rectangle intersection. Playing it, the generous box mostly reads as the skulls being frightening, which is not the worst outcome.

HITCHK decides nothing — it only accumulates bits in **HITMSK** (6C60h): bit 0 for “a collision happened at all”, the rest marking who with.

16.7. Which One **PLRHIT** Picks

Before it can get to the fight, **PLRHIT** has to decide **who** the player collided with: bit 0 of **HITMSK** only says that something happened. The scan runs from object seven down to zero:

```

        mov     al, HITMSK              ; 709B AL -- the frame's collision mask
        and     al, 0FEh                ; 709E clear bit 0: it is not about any one object
        mov     si, 7                   ; 70A0 SI=7 -- count the slots from the top down
        cld                               ; 70A3 clear the carry ahead of the RCL
LPLRHI9:
        rcl     al, 1                   ; 70A4 next bit out into the carry
        jnb     LPLRHI11                ; 70A6 bit set -- this is the one he ran into
LPLRHI10:
        dec     si                       ; 70A8 on to the next slot
        cmp     si, 0                   ; 70A9 are the slots used up?
        jge     LPLRHI9                 ; 70AC slots not used up yet
        RESTALL                          ; 70AE macro
        ret                               ; 70B5 return: there were no interactions
LPLRHI11:
        cmp     al, 0                   ; 70B6 are there no other collisions?
        je      LPLRHI12                ; 70B8 none -- deal with this one
        call    NEARCHK                 ; 70BA is the player close to object SI?
        jae     LPLRHI10                ; 70BD CF=0 -- too far off

```

The branch at **70B6h** is the whole point of **NEARCHK**. While more than one object is flagged, every candidate gets a proximity test: $abs(dY)/2 < 12$ and $abs(dX) < 5$. But **the last remaining candidate is accepted without a test**: by that point **AL** is already empty, and the branch skips past the call.

So if there is only one enemy in the room, **NEARCHK** is never asked. The logic is sound: if exactly one object flagged itself and there is nobody else, that is who you hit. The test would cost twelve instructions to confirm the obvious in the overwhelmingly common case.

16.8. The Sword Against Death

The branch that follows is short:

```
mov    al, ITMTMR          ; 7094 AL -- the amulet timer
cmp    al, 0               ; 7097 is the amulet still in force?
jne    LPLRHI14            ; 7099 in force -- the enemies cannot touch him
...
cmp    al, 1               ; 70C9 type 1 -- the skull
je     LPLRHI13            ; 70CB a skull is not taken with the sword
call   USESWRD             ; 70CD try the sword
jb     LPLRHI15            ; 70D0 CF=1 -- the enemy went down to the sword
LPLRHI13:
call   KILLPLR             ; 70D2 no sword -- the player is the one killed
```

A stationary enemy cannot be taken with the sword at all. Type 1 goes to **KILLPLR** bypassing **USESWRD**, and that is the only distinction between types anywhere in the game’s combat logic.

There is sense in it: a stationary enemy goes nowhere and chases nobody, and you can only walk into one through your own inattention. Mobile enemies are worth **3000** points for type 0 and **2000** for the rest (**70ECh...70F9h**), plus sound 4.

A kill is marked in **SEENMAP**, but only while **LEVEL < 5** (**70D7h**). From the fifth lap on, an enemy’s death becomes temporary: the next entry into the room brings him back whole.

16.9. How a Dead Enemy Leaves the Screen

Here is a subtlety without which the picture would break. **PLRHIT** clears the dead object’s bit in **INBITS**, but remembers it first:

```
mov    DEADBIT, al         ; 7128 bit of the killed object: its sprite still gets erased
xor     INBITS, al         ; 712B the object is taken off the screen
```

And **ST_PL2**, before erasing the frame, temporarily puts the bit back:

```
mov    al, INBITS          ; 6218 AL -- one bit per live object
push   ax                 ; 621B AX comes back at 6226
or     al, DEADBIT         ; 621C fold in the bit of the object just killed
mov    INBITS, al         ; 6220 so that its sprite gets erased along with the rest
call   ERSALL             ; 6223 erase every sprite, putting the background back
pop    ax                 ; 6226 AL -- the live-object bits as they were
mov    INBITS, al         ; 6227 the bits are back the way they were
mov    DEADBIT, 0         ; 622A the killed-object bit is cleared again
```

Without this, **ERSALL** would skip the dead one and its sprite would stay on the screen forever, because there would be nobody left to restore the background from **OBJTAB** ([Chapter 11](#)). The cell lives for exactly one frame, like **LITNOW** in [Chapter 13](#). Two one-frame flags in the whole program, both solving the same class of problem the same way — the file has a house style, and it holds.

16.10. One More Single-Frame Marker

Its neighbour **D_6CB1** (**6CB1h**) is set to one in the same place the dead object’s bit is cleared (**712Fh**) — and is never read: one write, no reads. There is nothing to name it after, because the reader that would have given it meaning is not in the image.

A byte raised at the moment an enemy dies looks like a marker saying “this has already happened” — the sort of thing you set while you are building the death sequence and watching it in the debugger. Markers like that outlive the code that read them, and the write stays behind: taking it out of a working routine is riskier

than leaving it. This is the second such case after `LITMSK` ([Chapter 13](#)), and like `LITMSK` it is a small window onto how the routine was built.

17. Progression and Score: Pyramid Laps, the Treasure Room, the Bonus Life

There is no finish line in this game: it does not end, it comes round again.

Having descended to the bottom of the pyramid, the player lands in the treasure room, gathers whatever he can in the half minute he is given, and finds himself back at the top. The same pyramid, the same Panama Joe, only darker and with enemies that no longer stay dead. Nine laps of that, after which the difficulty stops rising and the game goes round and round until the last life.

The session ends when the lives run out, and only then — which is exactly how every arcade cabinet of the period worked, and which suits this pyramid better than a closing screen would.

17.1. The Score Is Three BCD Bytes

`SCORE` lives at `6C84h` and takes three bytes, high one first:

```
6C84h  ten-thousands and hundred-thousands
6C85h  hundreds and thousands
6C86h  units and tens
```

Each byte is a **packed binary-coded decimal number**: one nibble per digit. Six digits, maximum 999,999.

The choice is not about saving space: three binary bytes would hold sixteen million rather than one. It is about output. To display a binary number you have to divide by ten six times, and a divide on the 8088 costs the better part of a hundred and eighty cycles — a thousand cycles to draw the score, every frame. In BCD the digit is simply sitting there in a nibble: shift, mask, draw. Picking a number format to suit the slowest instruction on the machine is the sort of decision that separates code that runs from code that runs at twenty-four frames a second.

The three bytes are not three numbers but one six-digit one: `ADDSCR` treats them as `SCORE+1` and `SCORE+2`, and `SHOWSCR` indexes `SCORE[si]` directly. There would be no sense in splitting them into separate cells — the carry runs straight through.

17.2. `PTSTAB` — What Is Worth What

`PTSTAB` (`6C61h`) is seven pairs of bytes, also BCD, low byte first:

offset	award	event
0	50	key, sword
2	100	amulet
4	3000	torch
6	300	opening a door
8	1000	jewel
10	2000	killing an enemy, type ≠ 0
12	3000	killing an enemy, type 0

The offset arrives in **DI**. For items it is not taken directly but through two tables lying right behind: **6C6Fh** holds seven element codes (**4Bh 43h 47h 37h 2Fh 2Bh 3Fh**), **6C76h** the corresponding offsets into **PTSTAB**. Doors and enemies set **DI** as a literal in the code.

Not one award is written as a number in the code: even the 3000 points for the torch is an offset of **4** in **DI** and two bytes in a table. Retuning the whole economy of the game is a data edit.

17.3. The Addition

ADDSCR (**6E3Bh**) adds in a column, bottom up:

```
mov    al, PTSTAB[di]      ; 6E42 low byte of the points, in BCD
adc    al, SCORE2          ; 6E46 plus the low byte of the score
daa    ; 6E4A the BCD fixup after the addition
mov    SCORE2, al         ; 6E4B low byte of the score
mov    al, PTSTAB[di+1]    ; 6E4E high byte of the points, in BCD
adc    al, SCORE1          ; 6E52 plus the middle byte of the score, with the carry
daa    ; 6E56 the BCD fixup after the addition
mov    SCORE1, al         ; 6E57 middle byte of the score
jae    LADDSC2            ; 6E5A no carry out into the high byte
```

DAA after every **ADC** is the whole of the decimal arithmetic. The high byte is touched only on a carry out of the middle one, that is, once every 10,000 points; the rest of the time **ADDSCR** works with two bytes.

The first ADC runs without a CLC. Whatever carry flag you arrive with goes straight into the score: with **CF = 1** a door would pay 301 points rather than 300. There are three call sites, and in all three the flag is clear by construction. Two clear it as a side effect — **CMP AL,0** at **70F4h** and **SHL DI,1** at **7158h**. In the third (**73BFh**, opening a door) it comes from **PUTGRID**, where the last instruction to touch the flags is **ADD DI,25h** (**0B56h**), and **DI** there is walking through **SCRMAP** (**11CDh...15B4h**), nowhere near overflow.

So no player will ever collect a spare point. It is a nice illustration of how this code is written: the invariant is held by the memory map rather than by an instruction, and the two bytes that a defensive **CLC** would have cost went somewhere else.

17.4. The Bonus Life

And here is what the carry is actually being checked for:

```
mov    al, 0              ; 6E5C AL=0 -- only the carry gets added
adc    al, SCORE          ; 6E5E high byte of the score, with the carry
daa    ; 6E62 the BCD fixup after the addition
mov    SCORE, al         ; 6E63 high byte of the score
cmp    LEVEL, 5          ; 6E66 level 5
j1     LADDSC1            ; 6E6B below the fifth -- a life every time
mov    al, SCORE          ; 6E6D AL -- high byte of the score
and    al, 1             ; 6E70 the low BCD digit
cmp    al, 0             ; 6E72 is it even?
jne    LADDSC2            ; 6E74 odd -- no life this time
LADDSC1:
call   XLIFE              ; 6E76 the bonus life
```

A life is awarded **on every crossing of 10,000 points**. From the fifth lap on, a condition is added — “the ten-thousands digit must be even” — which coarsens the threshold to 20,000. The extra life comes out of the same carry the addition already produced, so the whole reward system costs one comparison.

XLIFE (**6FC9h**) is three instructions: **INC LIVES**, sound 11, **RET**. There is no upper bound on the number of lives, though the panel will only show eleven ([Chapter 14](#)).

17.5. Drawing the Score

SHOWSCR (**6E84h**) hands nibbles to **PUTITM** right to left, starting at **CMDX = 4Dh** and stepping the position back by two per digit — a digit occupies a **2x2** cell.

Leading zeros are suppressed like this: first the loop at 6E9Dh finds the first non-zero score byte and puts its index in DX, then the high nibble is printed only if it is non-zero **or** the byte is not the first significant one. The floor is one digit: at a score of zero the panel shows 0.

The level digit (CMDX = 34h) and the inventory item glyphs (CMDX = 29h) are drawn the same way.

17.6. LEVEL Is a Lap Number, Not a Menu Choice

From Chapter 10 LEVEL is known as the difficulty that F2 cycles in the menu. That is only half the story:

```
LSTPL14:
    inc     LEVEL                ; 6307 difficulty level one step on
    cmp     LEVEL, 0Ah           ; 630B 0Ah = 10 -- the ceiling
    jl      LSTPL15             ; 6310 not reached
    mov     LEVEL, 9             ; 6312 it never climbs past the ninth
```

The menu offers 1, 2 or 3, and after that the number grows by itself — one per pyramid completed, up to a ceiling of nine.

So “difficulty” in this game is not a setting but a **lap counter**. Choosing three in the menu does not make the game harder; it starts you on the third lap. On the way back to the menu, ST_TITLE pushes the number back down to three (647Eh), or the lap number would carry over into the next session and it would begin on lap nine. One variable doing two jobs, and both of them honestly.

LEVEL is read in eight places, and nearly all of them are about difficulty:

address	what it does
07FEh	MKROOM: adds FLOOR+LEVEL into AL and does not use it
08F7h	BLDMAP: the bytecode branch when LEVEL < CMDX (Chapter 10)
0AABh	TREASR: window length for the second treasure table
0DADh	LITCHK: dark when FLOOR+LEVEL >= 9 (Chapter 13)
6E66h	ADDSCR: bonus life threshold
6EEAh	SHOWSCR: the level digit on the panel
70D7h	PLRHIT: whether the enemy that killed the player disappears
7D32h	menu: highlighting the selected level

The first line is a trace of code that has moved on: MOV AL, FLOOR / ADD AL, LEVEL at 07FBh...0801h computes a sum that nobody reads. AL is overwritten at 0813h and the flags die on the CMP at 0807h. The same sum is computed by LITCHK when deciding whether a room is dark — which is where the darkness test ended up living, and where it belongs.

The seventh line is a kindness to the player. When an enemy kills the player, KILLPLR returns CF = 1, and on laps 1...4 control goes to 70FCh, where the killer is marked in SEENMAP and does not reappear in the rebuilt room: the spot where you died becomes a little more passable. From the fifth lap that help is switched off. A difficulty curve implemented as the gradual withdrawal of mercy, in one comparison.

Darkness grows in the same order: on the first lap only the eighth floor is dark, on the third the sixth and below, and on the ninth everywhere except the treasure room (FLOOR = 9).

17.7. The Treasure Room

The hundredth room is the only one in the whole game that is not in bytecode. Ninety-nine are described by programs (Chapter 9), and this one is built by code: TREASR (0A1Dh). You get there through MKROOM:

```

call    ROOMID          ; 07DF AL -- room number, worked out from FLOOR and ROOMPOS
xor     ah, ah          ; 07E2 AH=0: the whole room number now sits in AX
mov     ROOMNO, al      ; 07E4 the number is kept
cmp     al, 62h         ; 07E7 62h = 98 -- an ordinary room
je      LMKR002         ; 07E9 that one is built from byte code
shr     al, 1           ; 07EB number halved
cmp     al, 31h         ; 07ED 31h = 49: AL/2 >= 49 means AL >= 98
jb      LMKR001         ; 07EF lower -- an ordinary room
call    TREASR          ; 07F1 99 and above -- the treasure chamber

```

ROOMID computes the number from floor and position; anything giving 99 or more goes to **TREASR**. The test reads in two moves: **AL/2 >= 49** means **AL >= 98**, and 98 was cut off by the line above comparing against 62h. The shift damages **AL** in the process, so 07F5h takes the number from **ROOMNO** again.

TREASR's first act is to declare the room special: **ROOMNO = 64h** (100), **FLOOR = 9** — a floor that does not exist in **ROOMBAS**, which is why it is never dark in here. The inventory is cleared completely (**ITEMCNT = 0**): everything you were carrying stays outside the door. The tune is picked by the **MUSSEL** counter, cycling through the four entries of **MUSTAB**.

17.8. The Treasure Room Layout: Two Windows

Then come two **BLDMAP** passes over two tables, and in both of them what is read is not a numbered record but a **window** whose start is computed arithmetically.

Geometry. Table 3362h, three-byte records:

```

mov     ax, OFFSET TRSMAP ; 0A61 table of treasure chamber geometry
mov     RDPTR, ax         ; 0A64 read from here
xor     ah, ah            ; 0A67 AH=0: the whole number now sits in AX
mov     al, FRAMEC        ; 0A69 the frame counter picks which chamber it is
and     al, 7             ; 0A6C eight geometry variants
shl     al, 1             ; 0A6E n * 2
shl     al, 1             ; 0A70 n * 4
mov     bx, ax            ; 0A72 BX = 4n
clc     ; 0A74 clear the carry ahead of the ADC
adc     ax, bx            ; 0A75 AX = 8n
adc     ax, bx            ; 0A77 AX = 12n: records run 12 bytes apiece
adc     ax, RDPTR         ; 0A79 plus the start of the table
mov     RDPTR, ax         ; 0A7D read from here
mov     ENDPTR, ax        ; 0A80 ENDPTR -- the same point for now, it moves on below
adc     ENDPTR, 24h       ; 0A83 24h = 36 bytes -- a window of three records

```

The start is **3362h + 12*(FRAMEC and 7)**, the length 36 bytes, which is exactly 12 records. Eight windows at a stride of four records, and they **overlap**: neighbouring variants share eight records out of twelve. The same trick as the **ELMTAB** ribbon in [Chapter 14](#) — variants packed densely into one sequence.

Treasures. Table 33F2h, two-byte records, window stride three records:

```

mov     ax, OFFSET TRSGEM ; 0A8B table of the gem scatter
mov     RDPTR, ax         ; 0A8E read from here
xor     ah, ah            ; 0A91 AH=0: the whole number now sits in AX
mov     al, FRAMEC        ; 0A93 the frame counter picks which scatter it is
and     al, 0Fh           ; 0A96 sixteen variants
mov     bx, ax            ; 0A98 BX = n
shl     ax, 1             ; 0A9A AX = 2n
clc     ; 0A9C clear the carry ahead of the ADC
adc     ax, bx            ; 0A9D AX = 3n
shl     ax, 1             ; 0A9F AX = 6n: records run 6 bytes apiece
adc     ax, RDPTR         ; 0AA1 plus the start of the table
mov     RDPTR, ax         ; 0AA5 read from here
mov     ENDPTR, ax        ; 0AA8 ENDPTR -- the same point for now, it moves on below
mov     al, LEVEL         ; 0AAB AL -- the difficulty level
cmp     al, 5             ; 0AAE levels past the fourth count as the fourth
jl      LTREAS1           ; 0AB0 the level is below the fifth
mov     al, 4             ; 0AB2 past the fourth no more gems are added
LTREAS1:
xor     ah, ah            ; 0AB4 AH=0: the whole level now sits in AX
shl     ax, 1             ; 0AB6 level * 2
shl     ax, 1             ; 0AB8 level * 4
clc     ; 0ABA clear the carry ahead of the ADC
adc     ax, 0Eh           ; 0ABB plus 0Eh bytes: the higher the level, the wider the window
adc     ENDPTR, ax        ; 0ABE read up to here

```

The start is `33F2h + 6*(FRAMEC and 0Fh)`, and the window length depends on the lap: `4*min(LEVEL,4) + 14` bytes, that is, from 9 records on the first lap to 15 on the fourth and thereafter. **The higher the lap, the more treasure.**

Counting by lap: the first pass always takes exactly 12 records, the second 9, 11, 13, 15 and thereafter 15 — two records more per lap, capped at the fourth. The number of tiles actually painted does not grow by exactly eight, because the `2x2` squares overlap one another.

Both masks — `and 7` and `and 0Fh` — come from the low nibble of `FRAMEC`, so there are exactly **sixteen** different treasure rooms rather than eight times sixteen: the wider mask adds nothing new. And sixteen is plenty, since you see one every time round the pyramid.

A third, short pass picks the exit side: if `ROOMPOS <= 8` the position becomes 8 and the bytecode `346Ah...346Dh` is laid down; if `>= 0Ah`, position 10 and `346Dh...3470h`. Two instructions, door on the left or door on the right.

17.9. Eight More Layouts, Ready and Waiting

The first table's index tops out at seven, so the last window is `33B6h...33DAh`. But the records carry on to `33F1h`: **24 bytes, eight complete three-byte records that nothing reaches.** Same format, same opcodes — these are fully formed treasure room geometries that no player will ever see.

Eight windows instead of twelve: the `and 7` mask does not span the whole table. Which came first, the mask or the last records, the image does not say. What it does say is that the treasure room was designed with headroom, and that widening the mask by one bit would light up another eight layouts tomorrow.

The second table, by contrast, lines up exactly: the last window at `LEVEL >= 4` ends at `346Ah`, precisely where the exit bytecode begins. Not a byte to spare.

17.10. The Lap Closes

The treasure room lives in a state machine state of its own — `ST_PL3`, state 4. `ST_PL2` prepares the entry:

```
LSTPL7:
    cmp    ROOMNO, 64h          ; 6262 64h = 100 -- the treasure chamber
    jne    LSTPL8               ; 6267 some other room
    mov    FRAMEC, 1           ; 6269 frame counter starts over at one
    mov    TREATMR, 1          ; 626E treasure chamber timer
    inc    GSTATE              ; 6273 on to the next state
```

`FRAMEC` is a one-byte frame counter, so it wraps every 256 passes of the main loop. `ST_PL3` counts those wraps:

```
    cmp    FRAMEC, 0           ; 62BB has the frame counter come around to zero?
    jne    LSTPL9               ; 62C0 no
    dec    TREATMR              ; 62C2 treasure chamber timer
    jl     LSTPL14              ; 62C6 timer ran out -- the level is over
```

`TREATMR` starts at one, so the treasure is on offer for two turns of the counter — about 511 frames, on the order of twenty-one seconds. The only way out early is to die.

Twenty-one seconds to collect everything the pyramid owes you. They are measured not by a timer and not by a clock but by the overflow of a one-byte frame counter: reached zero twice, time's up. There is nothing else to hand, and nothing else is needed.

The seam between laps is at `6307h`:

```

LSTPL14:
    inc     LEVEL                ; 6307 difficulty level one step on
    cmp     LEVEL, 0Ah           ; 630B 0Ah = 10 -- the ceiling
    jl      LSTPL15             ; 6310 not reached
    mov     LEVEL, 9             ; 6312 it never climbs past the ninth
LSTPL15:
    mov     LAPTMR, 2            ; 6317 room-change timer
    mov     ax, 0                ; 631C AX=0 -- not a single bit
    mov     cx, 65h              ; 631F 65h = 101 words in the seen map
    mov     di, OFFSET SEENMAP   ; 6322 DI -- the seen map
    rep stosw                    ; 6325 everything picked up is forgotten again

```

65h words is 202 bytes, two for each of the 101 rooms: **all memory of what was collected and killed is wiped by a single REP STOSW**. Items return to their places, doors close again, enemies live again. Then the player is placed at the edge of the pyramid, given the jumping state, and GSTATE goes off to redraw.

That is the entire world reset: an increment, two hundred bytes zeroed, and a jump. The pyramid is not rebuilt — it is simply no longer remembered. Nine laps of content out of one REP STOSW is as good a return on an instruction as this book has to offer.

The whole progression fits into those two dozen instructions: harder geometry, darker floors, rarer bonus lives, dead enemies that no longer stay dead — and, by way of compensation, two more records' worth of treasure waiting at the bottom. The game does not change. It becomes **less forgiving**, and for a game without an ending that is exactly enough.

Part V. Peripherals

18. Sound: the PIT, the Sequencer, the Scores, the Priorities

The IBM PC of 1984 has no sound device. It has a speaker wired to one bit of port 61h, and a timer channel that can toggle that bit at a given frequency. One square wave, with no volume, no envelope, no noise, and no second voice.

On the machines next door this was the job of a four-voice chip: send a note and forget about it. Here you have to send the note yourself, a hundred times a second, in an interrupt handler, between the frames of the game, and without letting the frame sag.

Out of that come twelve tunes, a glissando and a priority system. All the scores together take 455 bytes.

18.1. The Timer Moved to 100 Hz

SNDINI (058Eh) is called once, from **START**:

```
mov    bx, es:[di]          ; 0598 old offset out of the vector
mov    OLD080, bx           ; 059B old offset is saved away
mov    bx, es:[di+2]        ; 059F old segment out of the vector
mov    OLD08S, bx           ; 05A3 old segment is saved away
mov    ax, OFFSET SNTICK    ; 05A7 AX -- offset of the new handler
cld                                ; 05AA STOSW will run forward
stosw                               ; 05AB offset half: INT 08h -> SNTICK
mov    ax, cs                ; 05AC CS -- segment the handler lives in
stosw                               ; 05AE segment half of the vector
mov    al, PIT_M3            ; 05AF 36h: channel 0, mode 3, both bytes of the divisor
out    PORT43, al            ; 05B1 PIT control word
mov    bx, PIT_DIV           ; 05B3 divisor 2E9Ch = 11932 ticks
mov    al, bl                ; 05B6 low byte of the divisor into AL
out    PORT40, al            ; 05B8 low byte of the divisor out to channel 0
mov    al, bh                ; 05BA high byte of the divisor into AL
out    PORT40, al            ; 05BC high byte: the timer now runs a hundred times a second
in     al, PORT21             ; 05BE IRQ mask of the 8259 controller
and    al, 0FCh              ; 05C0 clear bits 0 and 1
out    PORT21, al            ; 05C2 timer IRQ0 and keyboard IRQ1 are open
```

The divisor 2E9Ch is 11,932, and 1,193,182 / 11,932 gives exactly **100.00 Hz**. A sequencer tick is 10 ms, and all the duration arithmetic is counted in those ticks.

SNDINI also saves the old **INT 08h** vector into **OLD080** and **OLD08S**. **And never thinks of it again**. Each of those two cells has exactly one reference in the image — that write. **SNTICK** sends its own **E0I** to port 20h and does not chain to the old handler.

So from the game's first frame the machine's time-of-day clock stands still: 40:6Ch stops advancing. The game runs from a bootable floppy and has nowhere to go back to, so the time of day is none of its business.

Saving the vector anyway is a two-line habit that costs four bytes and buys the option of putting the machine back the way you found it. Every program that hooks a vector should do it; this one does it and then turns out not to need it, which is the happiest possible outcome.

18.2. The Score Format

The twelve scores lie back to back, 77B2h...7978h. There are four kinds of record:

DW divisor, DB duration	note, 3 bytes
DW 0, DB duration	rest, 3 bytes
DW 1, DW f0, DW step, DW count, DW duration	sweep, 10 bytes
DW 0FFFFh	end, 2 bytes

The frequency in hertz is $1,193,182 / \text{divisor}$, the duration in ticks of 10 ms. A rest is a note with a zero divisor: the speaker simply goes quiet for the stated time.

A sweep occupies five words, and that shows both in the data and in the code. `SND_02` (77F2h) is a single record, `DW 1, 1560, 7, 225, 1`, ten bytes, with the terminator immediately after at 77FCh. And 0507h adds six to the pointer when the steps run out — because 0544h has already moved it four bytes forward, onto the “step” field:

```
mov    cx, 6                ; 0507 a sweep record is 10 bytes, 4 of them already stepped over
add    SNDPTR, cx           ; 050A pointer on to the next record
```

The step duration takes a word but is read as a byte: 0557h loads `es:[bx+4]` into `CX`, and only `CL` goes on into `SNDTCK`.

A note record is three bytes and is read as a word. At 052Dh:

```
LSNDTI4:
mov     bx, SNDPTR          ; 052D SNDPTR -- start of the next record
mov     ax, es:[bx]         ; 0531 note divisor, or a record marker
mov     cx, es:[bx+2]       ; 0534 only CL is wanted, but a whole word is read
add     bx, 3               ; 0538 a note is three bytes
```

The duration takes one byte but is read as a word, together with the first byte of the next record, which is thrown away along with `CH`. One instruction against one extra byte read: a word read costs no more than a byte read on this processor, so the byte is free.

The consequence is visible at the very end of the data. Score `SND_03` finishes with the word `FFFFh` at 7977h, and on the terminator `CX` is filled from 7979h...797Ah — **that is, from the first bytes of `PLAYSND` itself**. The game reads its own code and briefly considers playing it as a note duration. Nothing comes of it: on the terminator `CX` is zeroed immediately (0569h) and what was read goes nowhere. It is a good demonstration of how little slack there is between the data and the code in a 32K image — and of how carefully the terminator path was written.

18.3. The Sequencer

`SNDTICK` (04D6h) hangs on `INT 08h` and runs at all times, even when the main loop is busy redrawing:

```
cmp     SNDACK, 1           ; 04E1 sound switched off -- nothing to do but the EOI
je      LSNDTI2             ; 04E6 a sound is running -- carry on
LSNDTI1:
jmp     LSNDTI9             ; 04E8 off to the end: send the EOI and leave
LSNDTI2:
mov     al, SNDTCK          ; 04EB ticks left until the next record
dec     al                 ; 04EE one tick has gone by
mov     SNDTCK, al         ; 04F0 the counter is down
cmp     al, 0              ; 04F3 have we counted down to zero?
jne     LSNDTI1            ; 04F5 no -- the note is still sounding
```

`SNDTCK` is the count of ticks until the next record. `SNDAPI` sets it to one, so the first note is taken on the very first tick. After that a note of duration `d` occupies exactly `d` ticks.

The branch on record type is at 055Fh: zero is a rest (`SPKROFF`), `FFFFh` is the end (`SNDACK = 0, SPKROFF`), anything else is a note (`SNDFRQ = AX, SPKRON`).

`SPKRON` (048Bh) reprograms channel 2 from scratch each time: mode `B6h` to `43h`, divisor to `42h` in two bytes, then gate and data to `61h` (`OR AL, 3`). `SPKROFF` (04A1h) clears the same two bits.

Neither a rest nor the end writes a divisor to the port — channel 2 keeps the previous number. Silencing the speaker is enough: while the gate bit is down, the timer affects nothing. Two bits instead of a reprogram, on a path taken by every rest in every tune.

18.4. The Frequency Sweep

The one record type that outlives a single note. Its marker is the divisor **0001**, impossible for a real note:

```
SND_02: DW SND_SWEEP,1560,7,225,1 ; 225 steps of +7
SND_04: DW SND_SWEEP,3279,65461,20,1 ; 20 steps of -75
```

It is set up at **0544h**: **SNDPTR** is advanced to **+4** (the step field), **SNDRPF** is raised, the count goes into **SNDRPC**, and the starting frequency is played. From then on the upper branch runs on every tick:

```
mov bx, SNDPTR ; 051B SNDPTR is parked on the "step" field
mov ax, es:[bx] ; 051F the sweep step, signed
add SNDFRQ, ax ; 0522 the divisor moves on by one step
mov cx, es:[bx+4] ; 0526 how long this step lasts
```

When **SNDRPC** reaches zero, **SNDPTR** advances by 6 (**050Ah**) — $4 + 6 = 10$, exactly the record length — the flag comes down, and the next record is read in the same tick.

The step is added to an unsigned word, so a descending sweep is written as **65461**, which is **-75** in two's complement. Sound 2 is a rising whistle 2.25 s long, sound 4 a short descending one of 0.20 s. Two sound effects that a dedicated chip would have needed an envelope generator for, out of five words of data.

18.5. The Twelve Scores

The twelve entries of **SNDTAB** are the entire audio of the game:

sound	address	events	time	role	called by
0	7827h	5	0.22 s	starting to run	PST_1 (65CDh)
1	780Ah	10	0.18 s	jump	PST_0 (6697h)
2	77F2h	226	2.25 s	stepping off, rising whistle	PST_5 (689Fh)
3	796Bh	5	0.17 s	landing	PST_5 (68B4h)
4	77FEh	21	0.20 s	enemy killed by sword	PLRHIT (70E9h)
5	7928h	10	0.27 s	doorway opened	OPENDOOR (73C5h)
6	7945h	13	0.12 s	death	PST_5, KILLPLR
7	77B2h	10	0.86 s	item picked up	PLRHIT (7164h)
8	7835h	24	1.84 s	treasure room tune	TREASR
9	787Ch	24	1.84 s	treasure room tune	TREASR
10	78C3h	34	2.64 s	treasure room tune	TREASR
11	77CFh	12	1.14 s	bonus life	XLIFE (6FD0h)

The last three rows are easy to mistake for level background music. They are started by one routine only, **TREASR**, picking by **MUSSEL** and 3 from **MUSTAB** (**07BFh**, values **10**, **9**, **8**, **10**). **There is no continuous background music in this game at all**: a tune plays only in the treasure room, once per lap, and the laps cycle 10, 9, 8, 10, 10, 9, 8, 10... The rest of the time the pyramid is silent, and the sound works as a response to events rather than as a bed.

Given one voice and one speaker, that is the right call twice over. Silence keeps the effects legible, and it means the tune, when it finally arrives, feels like a reward.

18.6. Priority Is the Sound Number

There is no separate priority table. **PLAYSND** (7979h) asks its own API whether anything is playing, and if so compares numbers:

```
mov    ax, 1          ; 797F AX=1 -- the "is it playing" query
int     62h           ; 7982 OUR OWN API: sound
cmp     al, 1         ; 7984 AL=1 -- a sound is running
pop     ax            ; 7986 AX -- sound number
jne     LPLAYS1       ; 7987 silence -- play it, no questions asked
cmp     SNDPRI, ax    ; 7989 number of the sound now playing
jb      LPLAYS1       ; 798D the new one outranks it
jmp     LPLAYS2       ; 798F the one playing outranks it -- leave it alone
```

SNDPRI holds the number of the sound currently playing, **AX** the new one. Only a strictly greater number gets through — and that is the entire rule, for all one hundred and forty-four combinations of twelve sounds.

One comparison, and the game’s whole audio policy is settled without a priority table anywhere. The numbers in **SNDTAB** are arranged so that “greater” and “more important” coincide:

- the jump (1) overrides starting to run (0), the landing (3) overrides the falling whistle (2);
- death (6) overrides any movement;
- picking up an item (7) beats death — the fanfare plays even if the player is dying at that moment;
- the treasure room tune (8...10) is interrupted by nothing except the bonus life (11), which interrupts everything.

The numbering has nothing to do with the order of the data in memory: the scores are interleaved (7827h for sound 0, 77B2h for sound 7). So the numbers were assigned separately, to produce the hierarchy that was wanted — and the whole hierarchy fits into one **JB**. Encoding policy in the choice of identifiers is a trick worth stealing.

18.7. Its Own **INT 62h**

SNDAPI (04A8h) is the handler installed by **START**. Two modes, selected by **AX**:

- **AX = 1** — return **SNDACT** in **AL**: playing or not;
- otherwise **AX** is the offset of a score and **DS** its segment. **SNDPTR**, **SNDSEG**, **SNDTCK** = 1, **SNDACT** = 1, **SNDPRPF** = 0 are set up.

Why an interrupt where a **CALL** would do? The answer is in one instruction: **SNDAPI** takes the caller’s **DS** (**MOV BX, DS** at 04ADh) and puts it into **SNDSEG**, and the sequencer later reads the score through **ES = SNDSEG**.

Which is to say the sound was written on the assumption that scores would live **in somebody else’s segment**. A foreign segment is what a resident driver called from another program through an interrupt vector sees — and that is the case this subsystem was built for.

That case never arose. In a **.COM** image everything lives in one segment, the only client is **PLAYSND**, and its **DS** is the same one. The mechanism works perfectly, passing a segment from a variable into the same variable.

It is the same handwriting as **INT 67h** in [Chapter 7](#): what was being written was not a game but a set of services, out of which a game is then assembled. For a shop putting the same title on seven machines in a year, that is how you avoid writing the sound driver an eighth time. The vector entry stayed with the services, along with a few extra cycles per sound — and forty years later it is why the sound driver in this file can be read, understood and lifted out on its own.

19. Input: Keyboard, Joystick, and Its Self-Calibration

There are two devices, and one reader for both: all the rest of the code sees only **CMDBITS** and has no idea where the bits came from.

The technique is familiar; the execution is its own thing. Usually one

would invent a format of one's own and convert both devices to it. Here there is no format of one's own: **the keyboard pretends to be a joystick**, right down to the bit layout of the **201h** port byte. The game code reads the format of hardware that may not be in the machine at all — and reads it identically either way.

19.1. One Format for Both Devices

INPREAD (5E19h) branches on **INPDEV** and in both cases returns **AX** in the same shape: **AH** the direction bits, **AL** the button byte of port **201h**.

```
cmp    INPDEV, 0          ; 5E1A INPDEV -- the input device now selected
jne    LINPRE1            ; 5E1F not the keyboard -- read the joystick
jmp    KEYLOOK            ; 5E21 keyboard: pick LASTKEY apart
LINPRE1:
mov     bx, 102h          ; 5E24 BH=1, BL=2 -- X and Y axis bits of port 201h
call    JOYCAL            ; 5E27 take a count off both axes
in      al, dx            ; 5E2A AL -- the port 201h byte with the buttons
push    ax                ; 5E2B AX comes back at 5E2F
push    ax                ; 5E2B AX comes back at 5E2F
call    JOYDIR            ; 5E2C counts -> direction bits
pop     ax                ; 5E2F AL -- the button byte
mov     ah, JOYBITS       ; 5E30 AH -- the direction bits
```

Note the **IN AL,DX** with no reload of **DX**: **JOYCAL** left **201h** there, and the buttons are read from the same port. Three bytes saved, at the cost of the call order now being fixed — **JOYCAL** has to run first.

The joystick's is the native format. **The keyboard fakes it**. **KEYTAB** (5DFDh) is seven four-byte records: a scan code word and a reply word, already in **INPREAD** format:

address	scan code	key	reply	direction	button
5DFDh	4838h	Num 8	0210h	up	released
5E01h	5032h	Num 2	0110h	down	released
5E05h	4B34h	Num 4	0410h	left	released
5E09h	4D36h	Num 6	0810h	right	released
5E0Dh	4737h	Num 7	0400h	left	pressed
5E11h	4C35h	Num 5	0000h	—	pressed
5E15h	4939h	Num 9	0800h	right	pressed

The scan code is compared as a whole word, ASCII byte included: **4838h** is **48h** (arrow up) and **38h** (the digit 8). That combination comes from the numeric keypad **with NumLock on**; with it off the low byte is zero and **KEYLOOK** will not find the key.

The key to the whole construction is bit 4 of the low byte. In port **201h** the buttons are active low, so **10h** means “button 1 released”. The jump keys (7, 5, 9) differ from the walk keys (4, 8, 6) in exactly that bit being absent. A key not in the table yields **AX = 10h**: no direction and no button.

KEYLOOK (5FE2h) is a linear scan of the seven records, a **LOOP** on **CX = 7**. There can be more than one match only if the player assigns the same key twice; then the first in the table wins and the second goes quiet.

Seven keys, seven replies, and **10h** for anything else — that is the entire keyboard driver, and it plugs into the joystick path without one line of adaptation.

19.2. The Joystick Reading: Counting Port Reads

The PC has no analogue input. Port **201h** has a one-shot per axis whose pulse length is proportional to the potentiometer's resistance. Writing to the port fires them all, after which you have to measure how long the bit stays high.

JOYCAL (**5F99h**) counts **port reads** rather than cycles:

```
JOYCAL:
    cli                ; 5F99 the count is taken with interrupts off
    mov     JOYXRAW, 0  ; 5F9A X axis count starts out empty
    mov     JOYYRAW, 0  ; 5FA0 Y axis count starts out empty
    mov     cx, 40h     ; 5FA6 40h = 64 -- top of the scale
    xor     si, si      ; 5FA9 SI -- count of reads
    mov     ax, si      ; 5FAB AX=0 -- what goes out to the port
    mov     dx, PORT201 ; 5FAD the joystick port
    out     dx, al      ; 5FB0 the write fires the one-shots
LJOYCA1:
    in      al, dx      ; 5FB1 AL -- status byte of the axes and buttons
    ...
LJOYCA4:
    inc     si          ; 5FDA read counter one step on
    loop    LJOYCA1     ; 5FDB the scale is not used up
LJOYCA5:
    cmp     si, 40h     ; 5FDD did we run all the way to the top of the scale
    sti     ; 5FE0 interrupts back on
    ret      ; 5FE1 return: the counts are in JOYXRAW and JOYYRAW
```

Each axis has a scale of **0...63**. The **CLI** for the duration of the measurement is essential: a timer tick in the middle of the loop would stretch the reading and pull the stick sideways.

BX = 102h sets the masks: **BH = 1** for the X axis, **BL = 2** for Y, that is, joystick A. The second joystick is never read.

Measuring an analogue axis by counting your own **IN** instructions is a lovely piece of improvisation, and it is self-calibrating by construction: however fast the machine, the ratio between “centre” and “full deflection” stays the same.

A timeout test whose answer turned out not to be needed. **CMP SI,40h** at **5FDDh** sets the flags to say precisely whether the measurement finished in time — and then come **STI** and **RET**, and in both callers the very next instruction overwrites the flags (**IN AL,DX** leaves them alone, but **CALL JOYDIR** follows; in **NEWGAME** it is **SUB AX,3**).

The consequence is this: if an axis does not finish inside 64 probes, the reading stays **zero**, and zero to **JOYDIR** means “stick hard over”, left and up at once. A missing joystick and a stick jammed into a corner look the same to the game.

Catching that, though, would have been shooting at a target that is not there. The game boots from its own floppy and has nobody to ask whether a joystick is fitted, and the player selects the input device himself with **F1** in the menu. The flags are set correctly and truthfully; it simply turned out that nothing needed to read them.

19.3. A Dead Zone One Count Wide

JOYDIR (**5E36h**) turns two readings into four bits of **JOYBITS**: **1** down, **2** up, **4** left, **8** right. The thresholds are four words: **JOYXLO**, **JOYXHI**, **JOYYLO**, **JOYYHI**.

The trigger bands are deliberately **asymmetric**:

- below — exactly three counts: the direction is given when **si <= JOYXLO + 2**;
- above — **si >= JOYXHI - ((JOYXHI shr 3) or 2)**, so the margin grows with the threshold itself: 2 counts at **JOYXHI = 23**, 6 at 32, 7 at 63.

NEWGAME (5ECCh) sets the thresholds from a fresh rest reading: **JOYXLO** = rest - 3, **JOYXHI** = rest + ((rest shr 3) or 2) + 1. At a rest value of 20 that gives 17 and 23, and **the dead zone comes out exactly one count wide** — the value 20 itself. Left from 19 down, right from 21 up.

Sixty-four divisions across the axis, exactly one of which means “the stick is at rest”. The controls come out very responsive: a small deflection is enough to set Joe walking, which is what you want in a game where a jump has to be committed to before it starts.

19.4. A Ratchet: the Thresholds Creep Outward and Stay

ADJXLO...ADJYHI (5F41h...5F98h) are four identical routines of twenty-odd bytes each:

```

mov    ax, ACCXLO          ; 5F42  AX -- accumulator of the X low threshold
sub    ax, JOYXLO          ; 5F45  take the current threshold out
add    ax, si              ; 5F49  add the new count in
mov    ACCXLO, ax         ; 5F4B  accumulator brought up to date
mov    cl, 3              ; 5F4E  CL=3 -- shift by three places
shr    ax, cl              ; 5F50  accumulator divided by eight
mov    JOYXLO, ax         ; 5F52  new low threshold of the X axis

```

The accumulator **ACCXLO** holds the threshold times eight. So **ACCXLO** - **JOYXLO** + reading is 7·threshold + reading, and a shift by three gives the new threshold (7·threshold + reading)/8. Ordinary exponential smoothing with a coefficient of 1/8, done without a single multiply.

When it is called matters. **ADJXLO** runs only if the reading is **below** the low threshold, **ADJXHI** only if it is above the high one. The movement is always outward; there is no way back in the code.

Hold the stick hard left (reading 5) and the threshold slides from 17 to 6 in sixteen frames and to 5 in twenty-four. Bring it back to centre and wait a minute: the threshold **does not move by one count**.

It behaves like a ratchet: once it clicks, it does not let go. The point is that the game is adapting to the real travel of the particular stick in front of it: the “left” band ends where this joystick actually stops, rather than where **NEWGAME** guessed from a single rest reading. This is auto-calibration in 1984, in about eighty bytes, for hardware whose electrical characteristics varied wildly between manufacturers. The cost is that the adaptation accumulates over a session and resets only on the next **NEWGAME**, from **ST_TITLE** at the start of the next game (6496h).

19.5. Jumping on the Edge

READCMD (6D81h) puts the reply into **CMDBITS** and adds one thing to it without which the game would be unplayable:

```

and    al, BTN_JUMP        ; 6D87  bit 4 -- the jump button: 0 held, 1 released
cmp    JMPPREV, BTN_JUMP   ; 6D89  was the button released in the previous frame
je     LREADC1             ; 6D8E  it was -- this press counts
cmp    al, BTN_JUMP        ; 6D90  is it released right now
jne    LREADC2             ; 6D92  held, and it was held in the previous frame too
LREADC1:
mov     JMPPREV, al        ; 6D94  the state of the button is kept
jmp     LREADC3            ; 6D97  off to the common exit
LREADC2:
or      CMDBITS, BTN_JUMP   ; 6D9A  bit 4 forced on: holding the button kills the jump

```

If the button was down last frame too, the “released” bit is forced. The jump fires **on the leading edge**; holding the button gives exactly one jump — six frames of holding is one jump, three taps is three. Edge detection in seven instructions and one byte of state.

There is a subtlety right at the start. **JMPPREV** sits in the image as zero, meaning “button pressed”. Until it is released at least once, the **LREADC1** branch cannot be reached, and there will be no jump: a game started with the button already held eats the first jump. The second works.

19.6. The Keyboard Outside the Game

Game input goes through `INT 16h` inside `KEYPOLL` and `WAITF5`, and `MAINLP` keeps a poll of its own — and they do not interfere with each other, because they run at different times:

```
call    DISPAT      ; 0448 call the handler at STTAB[AL]
mov     LASTKEY, 0   ; 044B the frame's key starts out empty until one is read
xor     al, al       ; 0451 AL takes no part here, cleared along the way
mov     ah, 1        ; 0453 AH=1: is a key waiting in the buffer
int     16h         ; 0455 keyboard BIOS: is there one
je      MAINLP      ; 0457 the buffer is empty -- straight on to the next frame
xor     ax, ax       ; 0459 AH=0: take the key out of the buffer
int     16h         ; 045B keyboard BIOS: one key per frame, no more
mov     LASTKEY, ax  ; 045D scan code and ASCII are handed on to the state handlers
```

`LASTKEY` is filled in **after** the frame and read by `KEYLOOK` on the next one. So there is always exactly one frame between a press and the response, and in exchange nobody steals a character from anybody: the BIOS buffer is drained in exactly one place.

`MAINLP` handles two keys itself: `F5` returns to the menu (`GSTATE = 8`), `F10` pauses, that is, the loop at `0477h` that spins until a second `F10`. The other function keys are handled by `MENUKEY` (`7BAAh`) from the menu: `F1` toggles `INPDEV`, `F2` cycles `LEVEL` 1-2-3, `F3` goes to the key redefinition screen.

Two more keys were planned, and one of them left a receipt. Among the interface strings sit `F4 - Pause` and `F7 - Reset Game`; they never reach the screen ([Chapter 20](#) shows the menu in full) and there is no handler for them — `MENUKEY` knows only `F1`, `F2` and `F3`.

The filter in front of the `MENUKEY` call in `ST_MENU`, though, lets the whole range from `F1` to `F7` through:

```
cmp     ah, HIGH KEY_F1 ; 64C8 F1 -- bottom of the function key range
jl      LSTMEN1         ; 64CB below F1 -- keep waiting
cmp     ah, HIGH KEY_F7 ; 64CD F7 -- top of the range
jg      LSTMEN1         ; 64D0 above F7 -- keep waiting
call    MENUKEY         ; 64D2 deal with the function key
```

The upper bound is `F7`'s scan code rather than `F3`, where the decoding ends. The bound was set for a decoder that has room to grow, and `MENUKEY` simply returns on anything it does not recognize, so nothing has to be tightened for the game to be correct. Pause ended up on `F10`, and the reset was not needed: the menu is one keystroke away at all times through `F5`.

19.7. The Key Redefinition Screen

The `F3` screen, on the other hand, is a working feature, and by 1984 standards not a trivial one. `SCRKEYS` draws a heading and seven lines, calling `KEYLINE` for each, which shows the current assignment, waits for a new one, and **writes the result straight into `KEYTAB`**:

```
mov     ax, KEY_UP    ; 7E18 key now assigned to the action
call    KEYLINE       ; 7E1B put out the line and wait for a key
mov     KEY_UP, ax     ; 7E1E new key for the "up" action
```

The order of the lines on screen is UP, DOWN, RIGHT, LEFT, JUMP UP, JUMP RIGHT, JUMP LEFT — and it independently confirms the meaning of the table entries: under the caption `JUMP UP` it is `5E11h` that gets redefined, that is Num 5, and under `JUMP RIGHT` it is `5E15h`, Num 9. You can work out what each `KEYTAB` record means from the bits of its reply, or you can simply read it off the `F3` screen; both agree.

Fully remappable controls, stored in the same table the input path already reads, at a cost of no extra indirection whatsoever. That is a feature many games of the decade did not have.

You can only get here from the menu, via `F3`. There is a separate entry point `GOKEYS` (`7E97h`) — a single `JMP` to the same place — with no references to it: an unused door into a working room, the twin of `GOTTL` ([Chapter 5](#)).

19.8. Controls on a Modern Machine: a Queue, Not a State

The most noticeable difference when this game is run on modern hardware is that holding a direction key makes Panama Joe keep running for a while after it is released, so you end up playing in short taps. The cause is neither the player physics nor the animation; it lies entirely in where the direction comes from — and, as we will see, on the machine the game was written for it does not happen at all.

Joe has no inertia. In `PST_0`, on a frame with no direction bit, the velocity is zeroed by the very first instruction:

```
LBELTP4:
    mov     OBJVX, 0           ; 6773 the player comes to a stop
    mov     BYTE PTR OBJVY, 0 ; 6778 the player is not moving vertically
```

Get Joe running and release the key, and on the first frame with an empty buffer `CMDBITS` arrives with no direction bit at all and `OBJVX` becomes zero. He runs for exactly as many frames as he is given a direction, and not one more.

The direction comes out of a queue rather than from the keyboard's state. The game never reads port `60h` and never asks `INT 16h` with `AH = 2`. There are ten `INT 16h` calls in the whole image, and all of them are `AH = 0` and `AH = 1`.

There is the reason. The game does not know whether a key **is** down; it knows only that a key **was** pressed. The gap between those two questions is the lag: while unprocessed events sit in the buffer, Joe goes on taking orders from presses that ended some time ago.

One key is taken from the queue per frame. `MAINLP`:

```
    mov     LASTKEY, 0        ; 044B the frame's key starts out empty until one is read
    xor     al, al            ; 0451 AL takes no part here, cleared along the way
    mov     ah, 1             ; 0453 AH=1: is a key waiting in the buffer
    int     16h               ; 0455 keyboard BIOS: is there one
    je      MAINLP            ; 0457 the buffer is empty -- straight on to the next frame
    xor     ax, ax            ; 0459 AH=0: take the key out of the buffer
    int     16h               ; 045B keyboard BIOS: one key per frame, no more
    mov     LASTKEY, ax       ; 045D scan code and ASCII are handed on to the state handlers
```

There is no loop here. Fifteen presses sitting in the buffer will be decoded over fifteen frames, one per frame, strictly in order.

And what fills the queue is the BIOS typematic repeat. While a key is held, the controller sends it again and again, and the BIOS stacks the presses into the ring at `0040:001Eh` — 16 words, of which 15 are usable (one slot is spent distinguishing empty from full). On overflow the BIOS beeps and drops the excess.

From there it is simple arithmetic between two rates.

- While the repeat rate exceeds the frame rate, the queue grows. When the key is released, a backlog remains, and Joe honestly works through it at one press per frame.
- The ceiling on the lag is those same **15 frames**: no more will fit in the ring.
- How long that is in seconds depends only on the frame rate.

The frame rate is set by `FRMWAIT`: a budget of 270 wait units, which on a 4.77 MHz 8088 is about 41 ms, that is, **24 frames per second** ([Chapter 6](#)). The number is approximate — how many cycles a turn takes depends on the prefetch queue — but it agrees from the other direction: the treasure room is allotted 511 frames, and that is about 21 seconds.

frames/s	full buffer	repeat 10/s	repeat 30/s
5	3.00 s	grows	grows
8	1.88 s	grows	grows
12	1.25 s	steady	grows

frames/s	full buffer	repeat 10/s	repeat 30/s
24	0.63 s	steady	grows
30	0.50 s	steady	steady

On the machine the game was written for, there is no lag. The PC/XT typematic rate is about ten presses a second; the frame rate is about twenty-four. The queue does not grow at all: the buffer empties faster than it fills, and the delay between a press and a step is exactly one frame. Read against the hardware of 1984, the input path is not merely adequate — it is well matched, and it costs six instructions.

It comes apart in two situations, and neither of them is 1984:

- **the repeat rate is raised.** `MODE CON RATE=32 DELAY=1` under DOS, an AT-BIOS setting, or, under an emulator, the host system's own repeat rate, typically 30 a second. At 30/s against a 24/s frame the queue gains six presses a second and hits the ceiling after two and a half seconds of holding;
- **the frame is stretched.** Under an emulator the speed is usually turned down deliberately so the game does not race. But the slower the frame, the worse it gets from both ends: the queue starts growing even at the original ten presses a second, and the same fifteen frames of lag stretch into seconds. At eight frames a second a full buffer is nearly two seconds of coasting.

The author knew about the hazard. At the tail of `DRAWFRM` sits a **buffer drain** — the same loop we met back in [Chapter 5](#):

```
LDRAWF3:
    xor     al, al           ; 62A9 AL takes no part here, cleared along the way
    mov     ah, 1           ; 62AB AH=1: is a key waiting in the buffer
    int     16h             ; 62AD keyboard BIOS
    je      LDRAWF4         ; 62AF the buffer is empty
    xor     ax, ax          ; 62B1 AH=0: take the key out of the buffer
    int     16h             ; 62B3 keyboard BIOS
    jmp     short LDRAWF3    ; 62B5 keep draining until the buffer is dry
```

The loop reads the keyboard dry: a full buffer goes entirely, all fifteen presses. But `DRAWFRM` is a full room redraw, and it is called only by `ROOMDRW` (entering a room, picking up the torch) and `TTLROOM` (the attract screen). There is no drain in an ordinary game frame.

The logic is clear: a redraw costs four frames ([Chapter 7](#)), the player has time to hammer the keys, and the queue has to be thrown away so that Joe does not bolt from the doorway. The drain is placed exactly where it was needed on the target hardware, and nowhere else, because nowhere else did it cost anything.

The same line in `MAINLP`, ahead of the key fetch, would remove the lag on modern setups entirely: the queue would be limited to one last press, and the delay would never exceed a frame. Six instructions that are not there because on a 1984 machine there was nothing to cure. It is a good illustration of how tightly software of this period is fitted to its hardware — and of how much of what looks odd today is simply the hardware having changed underneath it.

19.9. Ten Zeros, Placed by the Linker

`5DE6h...5DEFh` are ten zero bytes in front of the axis thresholds. They look like a forgotten calibration table, and there is **not one reference to them**, for reading or writing. Nor was there: `5DF0h` is a paragraph boundary, and those ten bytes are the distance from the end of the sprite output to that boundary. The linker put them there, not a person.

They are worth knowing about because they are a fingerprint of the build. Fourteen of the fifteen anonymous padding runs in this image end exactly on a paragraph, which is what told us the original source was assembled as separate modules rather than one enormous file — and that, in turn, is what made a byte-exact reconstruction possible.

20. Text, Menus, Settings Screens

Three screens — the title, the main menu and key redefinition. All of

it together comes to twenty-odd lines of code, a ribbon of text with no separators in it at all, and not one font of its own.

The entire user interface, control remapping included, cost less than a single skull sprite — and the proportion is right: the player sees the menu twice a session.

20.1. There Is No Text Mode

The obvious guess is that the game switches to mode 3 for the menu and back. It does not: `INT 10h` with `AH = 0` appears in the image **twice** — in `START` (`03F6h`) and in `VIDINI` (`0648h`) — and both times it sets mode 4.

So the letters in the menu are not letters but pictures of letters, which the BIOS paints as dots on the same graphics page where the pyramid will appear a second later.

The letters are drawn by the BIOS teletype call, `INT 10h AH = 0Eh`. In graphics mode it takes glyphs from the 8×8 ROM font, hence the **40×25** grid: 320/8 by 200/8. And hence `BL`: in graphics mode the teletype reads it as the character colour, where in text mode it would be ignored.

The palette is set in `START`: `AH = 0Bh`, `BH = 1`, `BL = 0` — palette 0 of mode 4, green, red, brown. Three colours, and the whole interface is coloured with them.

`CLRTXT` (`7BF2h`) clears the screen through the BIOS as well — `AH = 6`, scrolling the window `0,0...19h,28h` by zero lines, which blanks it. Exactly seven instructions and not one access to video memory. Getting a complete text layer for free out of a ROM you already paid for is thrift of the best kind.

20.2. Strings Without Terminators

All the text lies in a continuous ribbon, `7A31h...7B98h`, with no zeros and no lengths. `PUTSTR` gets the length in `CX` and the address in `SI`:

```
PUTSTR:
    mov     ah, 2          ; 7CDB AH=2: the "set cursor position" call
    int     10h           ; 7CDD video BIOS
LPUTST1:
    push    cx            ; 7CDF CX comes back at 7CEA
    mov     cx, 1          ; 7CE0 put out one character at a time
    mov     ah, 0Eh        ; 7CE3 AH=0Eh: the "write character" call
    mov     al, [si]       ; 7CE5 AL -- next character of the string
    int     10h           ; 7CE7 video BIOS
    inc     si            ; 7CE9 on to the next character
    pop     cx            ; 7CEA CX -- how many are left
    loop    LPUTST1       ; 7CEB string is not finished
```

The `PUSH CX / POP CX` around the call is because the teletype wants `CX` too (the repeat count), while the outer `CX` holds the loop counter.

A ribbon like this saves twice over: no terminator byte per string, and pieces may overlap. Twenty-three `PUTSTR` calls from five routines (`SCRCTL`, `SCRMENU`, `MENUDEV`, `MENULVL`, `KEYLINE`) slice it into substrings. Every call looks the same — a start address and a length:

```
    mov     si, OFFSET TITLE2 ; 7C13 SI -- the string "MONTEZUMAS REVENGE"
    mov     cx, 13h          ; 7C16 13h = 19 characters
```

`TITLE2` is the heading `MONTEZUMA'S REVENGE`, `13h = 19` characters. The same point in the ribbon is printed on the title screen and in the menu; only the coordinates and the colour change.

20.3. Title Screen and Main Menu

SCRCTL (7C02h) lays out this:

```
+-----+
5|      Parker Brothers Presents      |
10|      MONTEZUMA'S REVENGE          |
12|      copyright 1984                |
23|      Press <return> for Menu Screen|
+-----+
```

And SCRMENU (7C4Ch):

```
+-----+
2|      MONTEZUMA'S REVENGE          |
4|      Main Menu                    |
8| F1 - Keyboard / Joystick          |
10| F2 - Game Level  1  2  3         |
12| F3 - Define Keys for Keyboard Control|
14| * F5 - Return to Main Menu       |
16| * F10 - Pause                    |
20| * functions valid while game in progress|
23|      Press <return> to start      |
+-----+
```

The lines are on alternate rows — even ones — and the asterisk marks what works during play rather than in the menu. The F3 line is exactly 40 characters, flush with the screen edge.

20.4. Highlighting the Selection with Colour

No inverse video, no box: the selected item is simply recoloured. MENUDEV (7CFEh) prints the words **Keyboard** and **Joystick** over the already-drawn line, substituting BL:

```
mov    bl, 3                ; 7CFF color 3 -- the chosen one
cmp     INPDEV, 1            ; 7D01 is the joystick chosen
je      LMENUD1              ; 7D06 yes -- "Keyboard" will not come out bright
mov     bl, 2                ; 7D08 color 2 -- not chosen
```

The logic reads inverted: at INPDEV = 1 (joystick) the word **Keyboard** gets colour 3 and **Joystick** colour 2. So **the selection is drawn in colour 2, red, and the unselected item in colour 3, brown**. On screen it looks like this:

```
row 8 3:' F1 - Keyboard / ' 2:'Joystick'
row 10 3:' F2 - Game Level ' 2:'1' 3:' 2 3'
```

MENULVL (7D31h) is cleverer and deserves a second look. There are three digits, in columns 14h, 17h, 1Ah, and it recolours **exactly two**: the selected one in colour 2, and the one selected before it in colour 3.

LEVEL	in colour 2	in colour 3
1	1, column 14h	3, column 1Ah
2	2, column 17h	1, column 14h
3	3, column 1Ah	2, column 17h

The one that gets extinguished is always the previous one round the cycle — because F2 can do nothing except cycle 1 -> 2 -> 3 -> 1. The third digit needs no touching: it is already colour 3, left over from the line SCRMENU printed. One call saved by reasoning about what can actually happen, which is exactly the sort of reasoning that keeps a 32K program inside 32K.

The defaults are visible from here too: INPDEV = 1 (7A30h) is joystick, LEVEL = 1 is the first level.

20.5. The Menu Turns NumLock On

The first thing SCRMENU does is reach into the BIOS data area:

```

mov ax, 40h          ; 7C4D 40h -- segment of the BIOS data area
mov es, ax           ; 7C50 ES -- the BIOS data area
mov BYTE PTR es:[17h], 20h ; 7C52 0040:0017 -- keyboard flags, bit 5 is NumLock

```

0040:0017h is the shift status byte, whose bit 5 means “NumLock on”. It is written with a **MOV** rather than an **OR**: Caps, Scroll, Insert and both shifts are cleared in the same stroke, and NumLock is set.

The previous state is not saved: enter the menu, and the machine’s keyboard is configured the way the game needs it. Nobody will put it back, but there is no way out of the game either ([Chapter 2](#)).

This is a direct continuation of [Chapter 19](#). **KEYTAB** compares a **word** — scan code together with the ASCII byte — and 4838h (arrow up plus the digit 8) is only produced by the keyboard with NumLock on. Without this one line the numeric keypad would give 4800h, **KEYLOOK** would find nothing, and keyboard control would not work at all. One instruction in the menu underwrites the entire keyboard driver in [Chapter 19](#), which is a nice piece of joined-up thinking across two subsystems.

20.6. The Key Redefinition Screen

SCRKEYS is seven identical blocks of three instructions: put the current value in **AX**, call **KEYLINE**, write the result back. **KEYLINE** (7ECBh) prints the caption on the left, puts the cursor in column 1Eh (**CURS30**), shows the current key and waits for a new one:

```

KEYLINE:
  push ax          ; 7ECB AX comes back at 7ED6
  mov bl, 3        ; 7ECC color 3
  mov dl, 2        ; 7ECE column 2
  call PUTSTR      ; 7ED0 action caption is on the screen
  call CURS30      ; 7ED3 cursor to column 30
  pop ax           ; 7ED6 AX -- key now assigned to the action
  call PUTCHR      ; 7ED7 show it on the screen
  call CURS30      ; 7EDA cursor to column 30
  call WAITKEY     ; 7EDD wait for the new key
  add dh, 2        ; 7EE0 two rows down

```

PUTCHR (7EBAh) prints the low byte of **AX**, the key’s ASCII. If it is zero (a key with no ASCII: a function key, or an arrow with NumLock off), the scan code is taken instead and clipped to 3Fh, and an arbitrary glyph appears. Only digits look meaningful, which is exactly what the default assignments are.

The screen in full, with the default assignments:

```

+-----+
1| Define Keys for Keyboard Control |
5| UP                8             |
7| DOWN              2             |
9| RIGHT             6             |
11| LEFT              4             |
13| JUMP UP           5             |
15| JUMP RIGHT        9             |
17| JUMP LEFT         7             |
+-----+

```

The order of the lines on screen is an independent caption for the **KEYTAB** table: [Chapter 19](#) works out what the records mean from the bits of their replies, and the **F3** screen simply names them in words. The two agree.

There is no validation of what is pressed. You can assign anything, including Enter, or a key already used by another action; on a clash the action earlier in the table wins and the other goes quiet.

Leaving via F5 is done by surgery on the stack. **WAITKEY** (7EA3h), on catching **F5**, redraws the menu and removes two return addresses from the stack:

```

  call CLRTXT      ; 7EB1 wipe the text off the screen
  call SCRMENU     ; 7EB4 bring the menu back
  pop si           ; 7EB7 return address 1 of two
  pop si           ; 7EB8 two levels of return pulled off the stack
LWAITK1:
  ret              ; 7EB9 return: the key is in AX

```

The first **POP** removes the return into **KEYLINE**, the second the return from **KEYLINE** into **SCRKEYS**. The address left on top leads outside the settings screen altogether, into **ST_MENU**. So **F5** abandons a half-finished screen whole, without waiting for the remaining lines. Where the addresses are popped to does not matter, and **SI** was chosen simply as a register not needed here: two **POP SI** is two bytes against three for **ADD SP,4**. A non-local exit in two bytes, in a language with no exceptions.

20.7. Four Captions Waiting in the Ribbon

The text ribbon is 360 bytes, and 65 of them never reach the screen:

address	contents
7A31h...7A58h	forty spaces
7B38h...7B41h	F4 - Pause
7B6Ah...7B78h	F7 - Reset Game
7DF9h...7DFCh	DONE

Forty spaces at the head of the ribbon are exactly one screen width: a ready-made “blank the line” that turned out not to be wanted, since **CLRTXT** blanks the whole screen through the BIOS more cheaply.

F4 - Pause and **F7 - Reset Game** are not merely unhandled ([Chapter 19](#)); they are **not printed** either. Of the seven menu items typed into the text, **SCRMENU** prints five — and those five are exactly the five that work. Dropping a **PUTSTR** call is a one-line change, whereas editing a ribbon in which everything is packed end to end and every offset is counted from the start is not. The menu the player sees is exactly the menu the code implements, which is the property that actually matters.

DONE is the caption for an eighth line on the **F3** screen. There are seven lines there and no eighth **KEYLINE** call: after **JUMP LEFT** control goes straight to **CLRTXT** and **SCRMENU**.

Three traces of the same story, typed into one ribbon: a pause, a game reset, and a “done” button on the settings screen. The captions are written, their places in the ribbon are laid out, and the key filter in **ST_MENU** already passes the right range — each of them is a few instructions short of being live. Pause found a home on **F10** instead, and with **F5** returning to the menu at any moment the reset was not missed.

Sixty-five bytes of text in a file of 32,228 — two tenths of one per cent to leave three finished captions where they are. Recomputing offsets in a packed ribbon to delete them would have cost more than keeping them, and forty years later they are one of the few places in the image where you can see what the menu was going to be.

20.8. How the Player Moves Between Screens

Putting [Chapter 5](#) and this one together:

```
start: GSTATE = 8
8 ST_TITLE -> SCRCTL, waits for Enter -> NEWGAME, GSTATE = 9
9 ST_MENU -> CLRTXT, SCRMENU, waits for:
    Enter -> GSTATE = 0
    F1, F2, F3 -> MENUKEY, the screen stays
0 ST_INI -> VIDINI, TTLROOM, GSTATE = 1
1 ST_TTL -> Joe walks into the pyramid himself (AUTOCMD), then play
...
7 ST_X7 -> GAME OVER: PUTBAR and the wait
```

The order takes some getting used to: the starting state is eight, followed by nine, and only then zero. The machine is numbered by the game frame rather than by the player’s route, and the title and menu ended up at the tail of the table simply because they were written last.

ST_X7 — the **GAME OVER** screen — holds a branch worth writing out in full:

```

LSTX1:
    cmp     LEVEL, 1           ; 6433 level 1
    jne     LSTX2             ; 6438 past the first -- no continue is offered
    cmp     RESTUSE, 0        ; 643A has the continue been used already?
    jne     LSTX2             ; 643F yes -- there is no second one
    call    READCMD           ; 6441 read the input device into CMDBITS
    mov     ax, CMDBITS        ; 6444 AX -- the control command
    and     ax, BTN_JUMP      ; 6447 bit 4 -- the button
    jne     LSTX2             ; 644A not held -- no continue
    inc     RESTUSE           ; 644C the continue is spent
    mov     LIVES, 5          ; 6450 five lives all over again
    mov     al, 0             ; 6455 AL=0 -- what to fill with
    mov     cx, 3             ; 6457 the score is three bytes of packed BCD
    mov     di, OFFSET SCORE  ; 645A DI -- start of the score
    rep stosb                 ; 645D the score is zeroed
    mov     GSTATE, 3         ; 645F state 3 -- play
    call    ROOMDRW           ; 6464 rebuild the room map and redraw the whole frame
    ret                      ; 6467 return: the game carries on

LSTX2:
    call    WAITF5            ; 6468 wait on a key; CF=1 if it was F5
    jae     LSTX1             ; 646B CF=0 -- F5 was not pressed
    mov     GSTATE, 8         ; 646D state 8 -- the main menu

```

The button will start you again straight from the **GAME OVER** screen, bypassing the menu — but only at **LEVEL = 1** and only **once**: **RESTUSE** is raised here and cleared only in **VIDINI**. And **LEVEL**, as [Chapter 17](#) showed, grows with every lap of the pyramid and returns to one only through the menu. So the quick restart is available precisely to a player who died on the first lap at the first difficulty and has not used it yet. Everyone else gets **F5** and the menu.

It is a small, well-judged piece of generosity: the one player who most needs a second chance is the one who has just started.

Part VI. Appendices

What follows is reference material rather than reading: the map of the image module by module, a list of the questions the file declines to answer, and the vocabulary the rest of the book assumes.

21. The Complete Map of the Image

Everything there is in `MX4C.COM` – 32,228 bytes, `0100h...7EE3h`. The game never had one byte beyond this.

The image was assembled out of twenty-one modules, and this is what each of them contributed.

module	range	bytes	share	what is inside
<code>BOOT.ASM</code>	<code>0100h...06D4h</code>	1493	4.6%	entry, main loop, sound driver, initialization
<code>ROOMBLD.ASM</code>	<code>06E0h...0FF3h</code>	2324	7.2%	object table and the room builder
<code>PUTMAP.ASM</code>	<code>1000h...108Ch</code>	141	0.4%	tile map output, INT 67h ISR
<code>SCRDAT.ASM</code>	<code>1090h...260Fh</code>	5504	17.1%	map variables, SCRMAP, tiles, elements, panel
<code>ROOMDAT.ASM</code>	<code>2610h...369Fh</code>	4240	13.2%	descriptions of all 99 rooms
<code>RECTS.ASM</code>	<code>36A0h...3749h</code>	170	0.5%	partial redraw and the floor-span list
<code>TILANI.ASM</code>	<code>3750h...38EFh</code>	416	1.3%	cycling the animated tiles
<code>ANIMDAT.ASM</code>	<code>38F0h...3BDFh</code>	752	2.3%	working tiles and animation tables
<code>FRAMES.ASM</code>	<code>3BE0h...5C9Fh</code>	8384	26.0%	sprite frames, background buffer, masks
<code>SPRDRW.ASM</code>	<code>5CA0h...5DE5h</code>	326	1.0%	sprite output
<code>JOYSTK.ASM</code>	<code>5DF0h...5FFDh</code>	526	1.6%	joystick, keys, new-game reset
<code>SCANTIL.ASM</code>	<code>6000h...60B1h</code>	178	0.6%	neighboring tiles
<code>STATES.ASM</code>	<code>60C0h...6503h</code>	1092	3.4%	the game state machine
<code>PLAYER.ASM</code>	<code>6510h...6C5Bh</code>	1868	5.8%	player state and physics
<code>ITEMS.ASM</code>	<code>6C60h...7485h</code>	2086	6.5%	score, items, collisions, panel
<code>ENEMY.ASM</code>	<code>7490h...77A1h</code>	786	2.4%	object turns and the four behaviors

module	range	bytes	share	what is inside
SOUND.ASM	77B0h...79BCh	525	1.6%	tunes and the sound dispatcher
TIMING.ASM	79C0h...7A2Dh	110	0.3%	frame length
TEXT.ASM	7A30h...7DA9h	890	2.8%	strings, menu, screens
KEYSCR.ASM	7DB0h...7EE3h	308	1.0%	the define-keys screen
ELMCOD.ASM	—	0	—	constants only, not one byte

22. What the Image Does Not Tell Us

The file is a conscientious witness but not a talkative one. It shows everything the program does, and says not one word about why any of it was done.

There is a fair amount in these thirty-two thousand bytes that affects nothing: the pause between the death and the banner, declared as a state and one instruction short of being connected; the `LITMSK` cell belonging to a second darkening scheme; two tile animations whose calls have been lifted out while the routines stand intact; eight spare treasure room geometries; the `D_6CB1` byte raised at the moment an enemy dies; the saved `INT 08h` vector that nobody returns to; the `F4` and `F7` lines in the menu text.

Stacking all of that up and labelling it “ran out of time” is the easiest thing to do, and [Section 11.9](#) shows what it costs. Eight `NOP0` calls, two spare kilobytes of `OBJTAB` and the `PAD03CD` cell look exactly like “ran out of time” — and they are in fact precise fingerprints of another machine, one with two video pages. Not the leavings of the work but its load-bearing structure, with half of it sawn off on CGA because the other half had nowhere to stand.

How many more such places there are in the file, the file does not say. The game was shipped on seven machines inside a single year and the versions were written side by side; some of what is seen here may have been common to several ports, and some of it working scaffolding that nobody uprooted, because nobody in their right mind disturbs working code before a ship date.

So the list above is not a catalogue of unfinished business. It is a catalogue of questions. Several of them already have answers, and the answers came out in favour of the people who wrote this game. Behind the rest there is presumably a reason just as sound — there is simply nobody left to tell it.

23. A Short Glossary

Everything in this list is used somewhere in the book. Nothing in it is specific to this game; it is the vocabulary of a 1984 IBM PC.

term	what it means here
8088	the processor in the IBM PC: 16-bit inside, 8-bit bus, 4.77 MHz
BCD	binary-coded decimal: one decimal digit per half-byte, so a score can be printed without dividing

term	what it means here
bytecode	a small made-up instruction set, interpreted by the game itself; here it describes rooms
CGA	the Color Graphics Adapter: 320×200, four colours, 16 KB of its own memory
cycle	one tick of the processor clock; at 4.77 MHz there are 4,770,000 a second
dirty rectangle	a patch of screen known to have changed, redrawn on its own instead of the whole picture
field	one of the two halves CGA memory is split into: even screen rows in one, odd in the other
flag	a single bit of processor state left over from the last instruction; the carry flag is the one used most here
frame	one pass of the main loop: read input, move everything, draw. This game aims at about 24 a second
hit box	the rectangle around a figure inside which a collision counts
interleave	the CGA habit of storing every other screen row in a different half of memory
interrupt	a signal that makes the processor drop what it is doing and run a handler; used here for the timer and the keyboard
IRQ	an interrupt raised by hardware: IRQ0 is the timer, IRQ1 the keyboard
ISR	interrupt service routine: the handler an interrupt runs
mask	a pattern of bits used to knock holes in another pattern; here it makes sprites transparent
nibble	half a byte, four bits — on this display, two pixels
opcode	the number that says which operation an instruction performs
paragraph	sixteen bytes; addresses that are multiples of sixteen are said to be paragraph-aligned
PIT	the 8253 programmable interval timer: three counters, one of which drives the speaker
pixel	one dot of the picture; two bits on this display, so four to a byte
prefetch queue	the few instruction bytes the processor reads ahead, which is why exact cycle counts are never quite exact
PSP	program segment prefix: the 256 bytes DOS puts in front of a loaded program
raster	the path the electron beam takes across the screen, line by line
register	one of the processor's eight on-chip storage slots; the fastest place to keep a number, and always in short supply
scan code	the number the keyboard sends for a key, before anything decides what letter it is

term	what it means here
segment	a 64 KB window of memory; a <code>.COM</code> program lives entirely inside one
sprite	a moving figure drawn over the background — on this machine, entirely in software
stack	the region the processor uses for return addresses and saved registers, growing downward
tile	an 8×8 square of picture, sixteen bytes; the pyramid is built from 235 of them
typematic repeat	the keyboard's habit of sending a key over and over while it is held down
vector	an entry in the table at the bottom of memory that says where an interrupt handler lives
VRAM	video memory: on CGA, the 16 KB at segment <code>B800h</code> that the screen is painted from
word	two bytes, sixteen bits

Part VII. The Plates

The pictures that follow were not photographed off a screen. Every one of them was produced by running the game's own code: the rooms are drawn by the image's own room builder, the tiles and frames are the bit-maps lifted straight out of the file at the addresses this book quotes. What you are looking at is the data, rendered at the size the page allows rather than the size the screen allowed.

24. The Tile Set

Two hundred and thirty-five tiles, sixteen bytes each, and the whole pyramid is built out of them. [Chapter 7](#) explains the format; [Chapter 9](#) explains how ninety-nine rooms are assembled from these squares without a single room being stored anywhere.

Read them by number. [00h](#) is empty space, [01h](#) and [02h](#) are the brick floor, [28h...2Bh](#) are the ladder pieces, [72h...77h](#) the six phases of a conveyor belt, [70h](#) the floor that blinks in and out. Tiles [34h](#) and above survive a dark room ([Chapter 13](#)), which is why the keys and the torch are drawn out of that range and the sword, amulet and jewel are not.

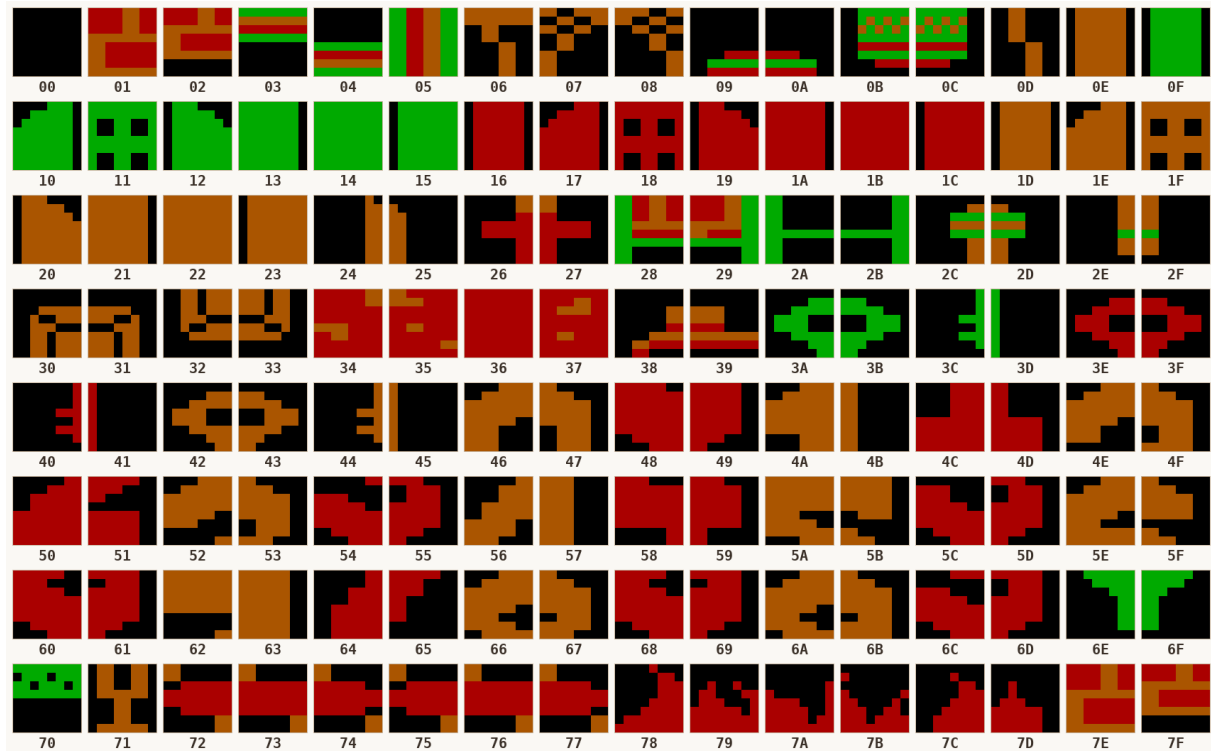
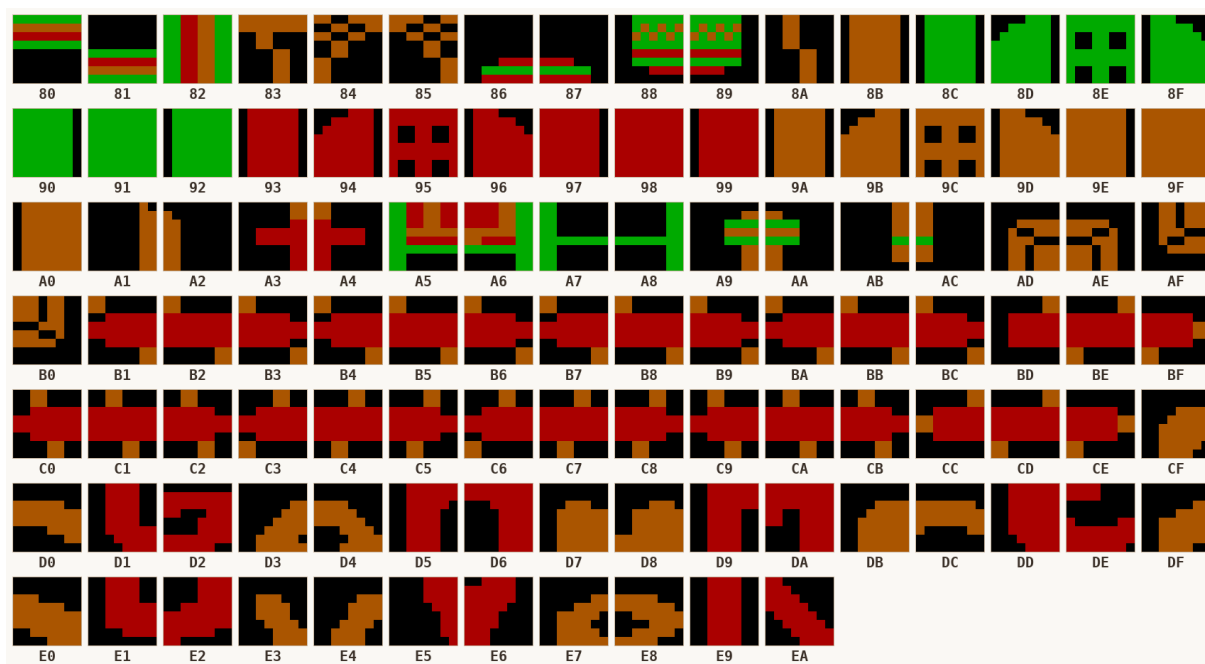


Plate 1. Tiles [00h](#) to [7Fh](#).



*Plate 2. Tiles 80h to EA. The upper range holds the reference copies — the untouched originals that **TILRST** restores after a dark room, and the ones the status panel draws from so that a sword in your inventory stays visible when the sword on the floor does not.*

25. Sprite Frames

Thirty-seven frames, and out of them comes every moving thing in the game: Panama Joe walking, jumping, climbing and coming apart, and four kinds of enemy. [Chapter 11](#) covers how they are drawn, [Chapter 12](#) how they are strung into chains.

Two things are worth looking for. First, the walking-left frames are not the walking-right frames flipped by the machine — they are drawn separately, by hand, and one pixel on Joe’s belt does not quite match its mirror image. Second, none of these figures contains the background colour anywhere it is meant to be seen: a zero in a sprite means “the wall shows through here”, so the artist had three inks and no black.

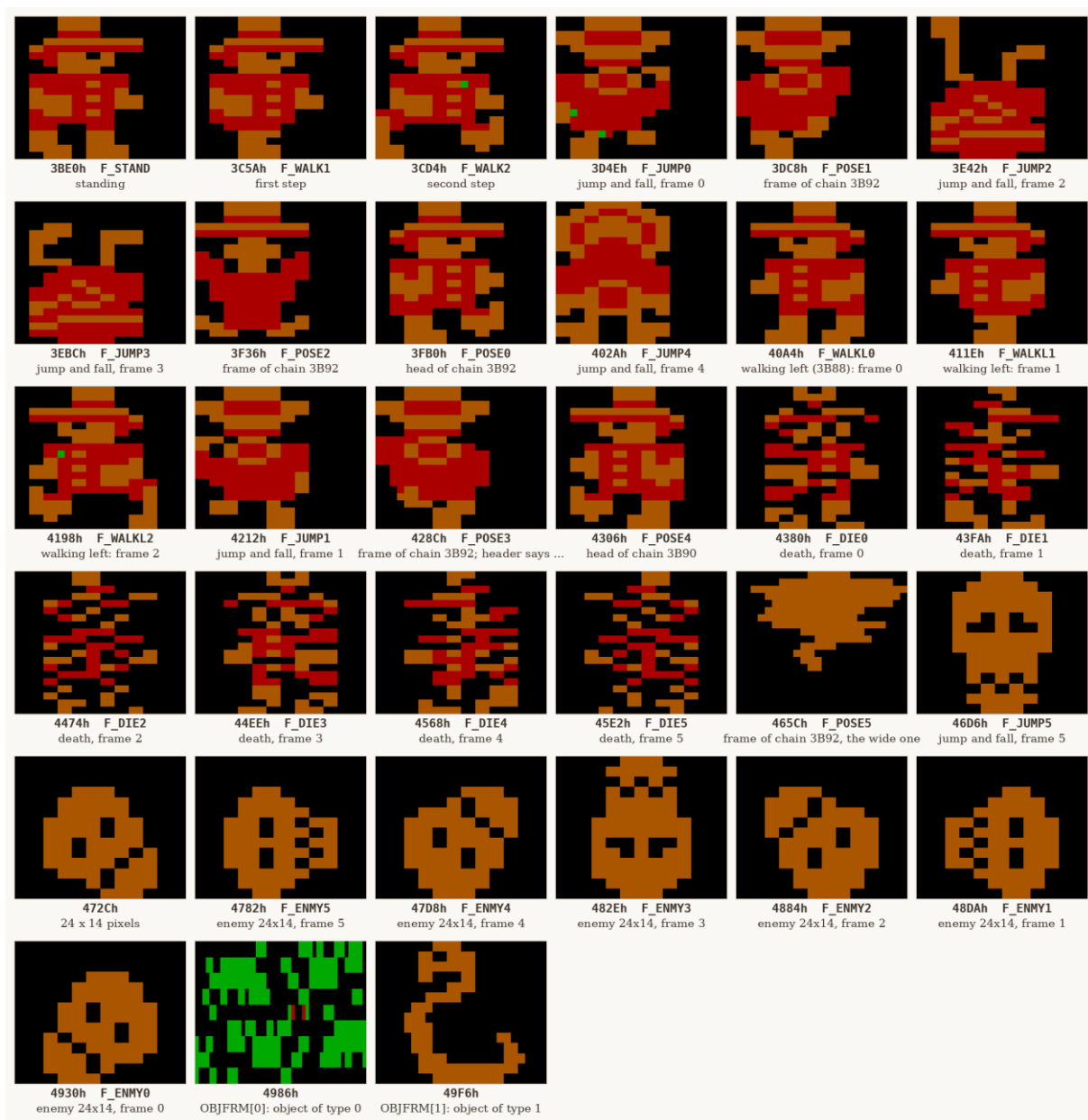


Plate 3. Every sprite frame in the file, with the address it lives at and the name it carries in the listing.

26. Items and the Status Panel

Seven things can be picked up in the pyramid. Each is a 2×2 square of tiles and four bytes of description in the ELMTAB ribbon; [Chapter 14](#) takes them apart.

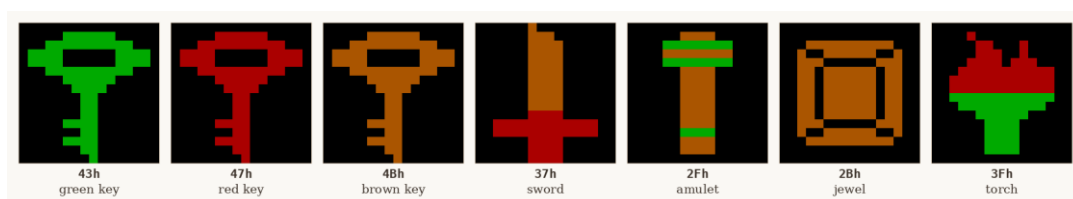


Plate 4. The seven items, with their element codes. Three keys, a sword, an amulet, a jewel and the torch that lifts the darkness.

The status panel is the top five rows of the same map the pyramid is drawn on, and it is built from the same parts: the digits are elements of `ELMTAB`, the row of lives is a strip tiled by the same primitive that lays a brick wall. There is no separate font anywhere in this game.

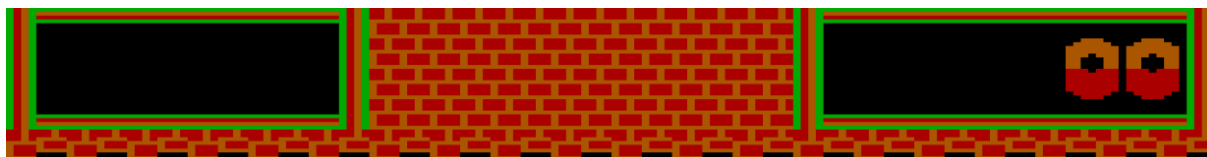


Plate 5. The status panel: lives on the left, the current item and the difficulty digit in the middle, the score on the right.

27. The Ninety-Nine Rooms

None of these rooms exists in the file. Each is a short program in the bytecode of [Chapter 9](#) — thirty-five bytes on average — and what you see below is what the game’s own interpreter draws when it runs them. The pyramid is nine floors deep and widens by two rooms a floor: three, five, seven, and so on to nineteen.



Plate 6. The pyramid in numerical order, ten rooms to a row. The hundredth cell is empty: there are ninety-nine rooms, and the treasure room at the bottom is the one place not written in bytecode at all.

Instead of an Afterword

Thirty-two thousand two hundred and twenty-eight bytes. Ninety-nine rooms that are not in the file. Seven items, four enemy behaviours, twelve tunes on a single bit, two input devices one of which pretends to be the

other. A jump made of thirteen bytes. Darkness for the price of four instructions. A frame measured out by subtracting one from a register.

And half a kilobyte of zeros for the stack, because there was no time to work out the exact figure — and because half a kilobyte of zeros was the right answer anyway.

The game was written in the year the industry was folding underfoot: the publisher would leave video games within twelve months, the original author had already moved on, and the game has no ending and never will. It shipped on time. Bytes were saved where saving them paid and working code was left alone where it did not. All perfectly ordinary.

And what came out of it was something a whole generation played, and something that forty years later is worth taking apart byte by byte — in this very book.

Not a bad result for thirty-two kilobytes.

On the Reconstructed Source

The game's assembly source ships alongside this book. Not a retelling and not an approximate reconstruction: **assembled with MASM 1.25, it produces a file identical to the original MX4C.COM byte for byte, all 32,228 of them.** Every address this book refers to can be opened in the listing, where you will find exactly the instruction described.

The reconstruction was checked in the only way that means anything here: assemble and compare. Anything that did not match to the last byte was treated as wrong and done again. The names of routines and variables were invented during the analysis — nobody kept the originals — but the bytes are authentic.

On Copyright

This project consists of reconstructed source code and assets for *Montezuma's Revenge* (IBM PC, 1984), prepared purely for educational, historical-research and archival purposes. The work was done out of respect for the people who wrote this game, and it pursues neither commercial gain nor any intent to infringe anyone's rights.

Robert Jaeger — the original designer and programmer of *Montezuma's Revenge*, and the principal at Normal Distribution LLC — has been informed of this project and is supportive of it as a non-commercial effort. By his own account Parker Brothers held the game under licence and the rights reverted to him after a few years; he publishes new versions of it to this day.

The IBM PC conversion examined in this book was not his work: Parker Brothers commissioned the conversions to other machines and made the programming changes themselves. This book makes no claim to have cleared whatever separate rights may subsist in that particular conversion. Should Mr Jaeger, or any other party holding rights in the game or in this conversion of it, consider that these materials ought to be taken down, they will be taken down.